

RIT VEXU Software Engineering Notebook

2023-2024

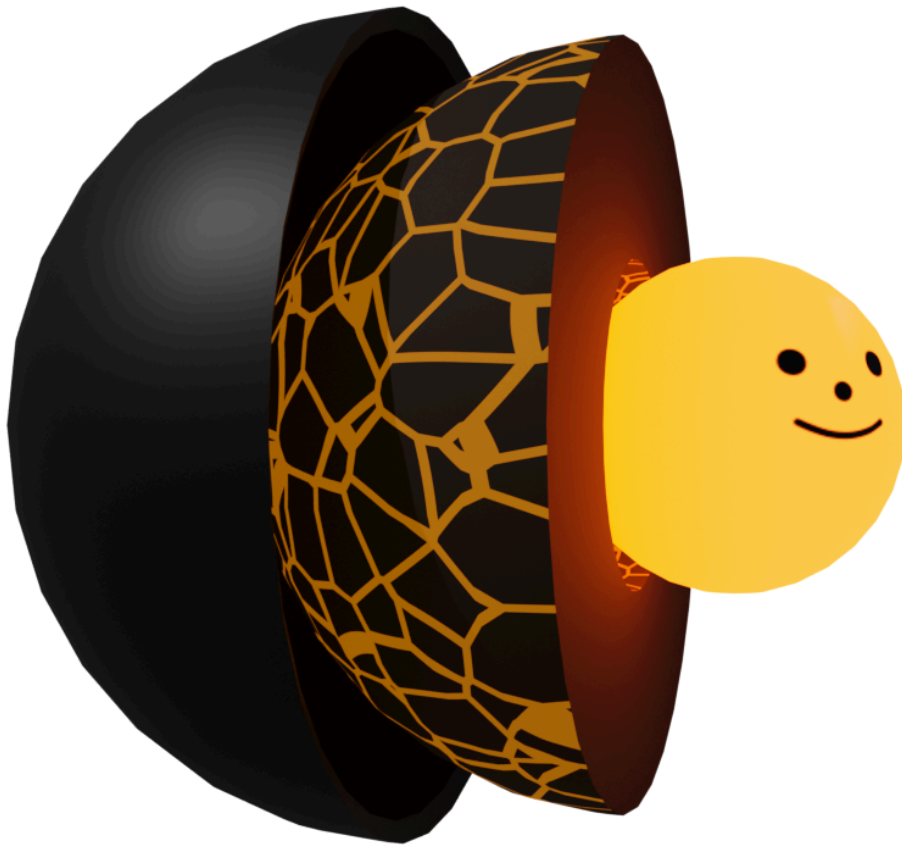


Table of Contents

RIT VEXU Software Engineering Notebook.....	1
Table of Contents.....	2
Overview.....	3
Core API.....	3
Open Source Software.....	3
Project Structure.....	4
Git Subrepo.....	4
Github Project Board.....	4
Github Actions.....	5
Auto-Notebook.....	5
Clang-Tidy.....	6
Wiki.....	6
Core: Fundamentals.....	7
Odometry.....	7
Drivetrain.....	9
Tank Drivetrain.....	9
Control Loops.....	14
Auto Command Structure (ACS).....	16
Serializer.....	18
Screen Subsystem.....	19
Catapult System.....	22
Vision.....	23
Core: Ongoing Projects.....	26
Rust.....	26
N-Pod Odometry.....	28
V5 Debug Board.....	34

Overview

Core API

All of the robot code we use is built on top of our own custom library, called the Core API, which itself is built on top of the official VEX V5 library. This API contains template code for common subsystems such as drivetrains, lifts, flywheels, and odometry, and common utilities such as vector math and command-based autonomous functions. This code remains persistent between years and is constantly updated and improved. The library can be found at github.com/RIT-VEX-U/Core

The Core codebase is abstracted in a way that allows for simple use during a hectic build season, and creates a solid foundation for future expansion. Subsystems are divided into layers following an object-oriented approach to software development.

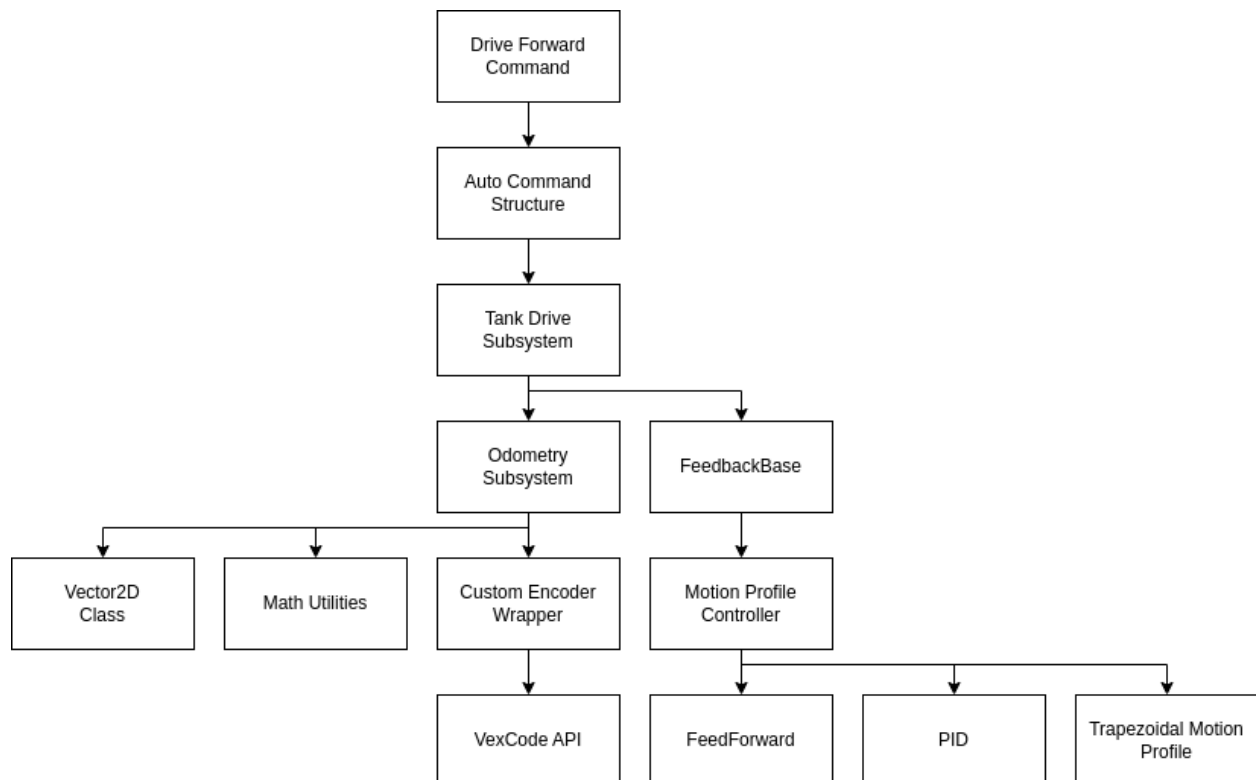


Figure 1 - Example of Object-Oriented Programming

Open Source Software

The Core API is under the MIT open-source license, and is open for other teams to use and improve upon via pull requests. This system was modeled after the Okapi library from the Pros ecosystem, and offers similar functionality for the VexCode ecosystem. Teams that use this API are also encouraged to open source their software.

Project Structure

During the season, there are three repositories (repos) that are actively developed. Two repositories for the two competition robots, and one for the Core API. Development and code building occurs in the robot repos, and any changes to shared code (drivetrain, math utilities, major subsystems) are merged with the Core repo. This method reduces redundant code and development time.

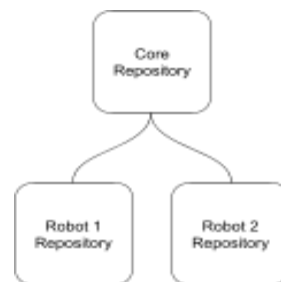


Figure 2 - Project Structure

Git Subrepo

The Core API uses a unique type of version control called Git Subrepo (github.com/ingydotnet/git-subrepo). This allows users to simply clone the repository into an existing VexCode project to have instant access to all the tools. It also allows users to instantly receive updates by pulling from the main branch, and makes sharing code between two robot projects easier with git code merges.

Before choosing Subrepo, the team experimented with using Git Submodules to incorporate the Core API into projects. This however made Core development cumbersome and difficult for anyone unfamiliar with Git submodules specifically. Subrepo made inter-project merges more streamlined, and simplified development.

Github Project Board

In order for our software team to collaborate together with these projects, we use the Github Projects kanban-style project board. This allows us to create and assign tasks, link it to a repository and additionally notify the assigned programmer through a slackbot.

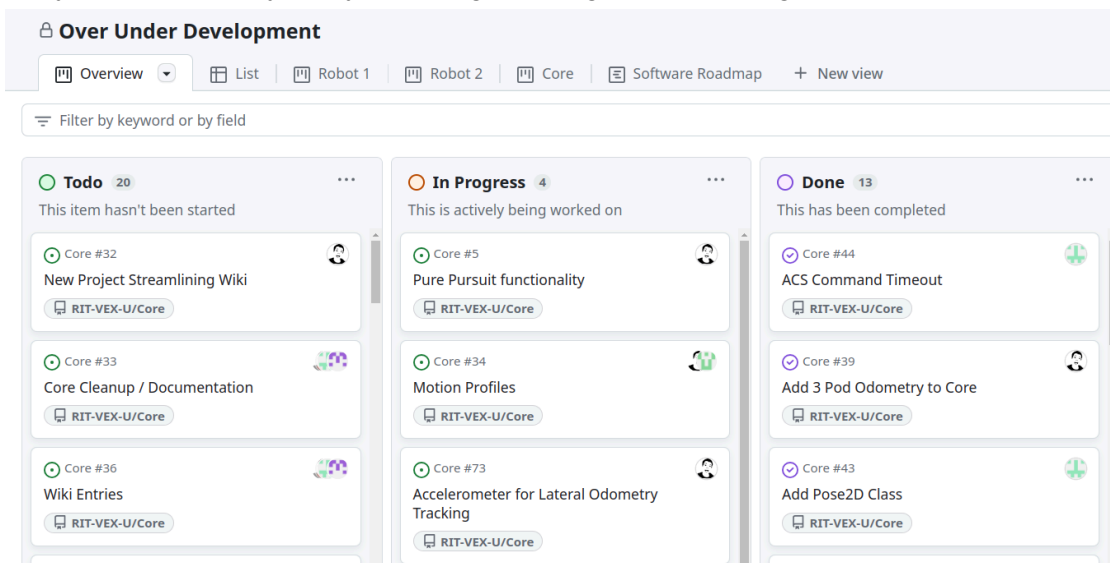


Figure 3 - Software Project Board

Github Actions

This year, our team enhanced our workflow by integrating GitHub Actions into our software development process. One notable addition was an action to build our C/C++ code in the appropriate Vex environment. This automated process involves a series of steps, including checking out the repository, downloading and unzipping the Vex Robotics SDK and toolchain, and compiling the code using a Makefile. A key feature of this GitHub Action is its ability to send a Slack notification to our team channel whenever a build fails, ensuring prompt awareness and response. Furthermore, it helps maintain code integrity by preventing the merging of pull requests with failing builds. This complements our other GitHub Action for building Doxygen documentation and deploying it to GitHub Pages, allowing for seamless documentation and code management. This systematic approach aligns with our commitment to maintaining a neat, organized, and efficient engineering process.

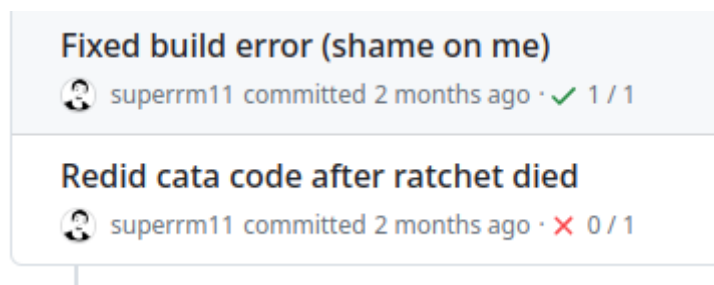


Figure 4: Continuous Integration directly improves the quality of our code.

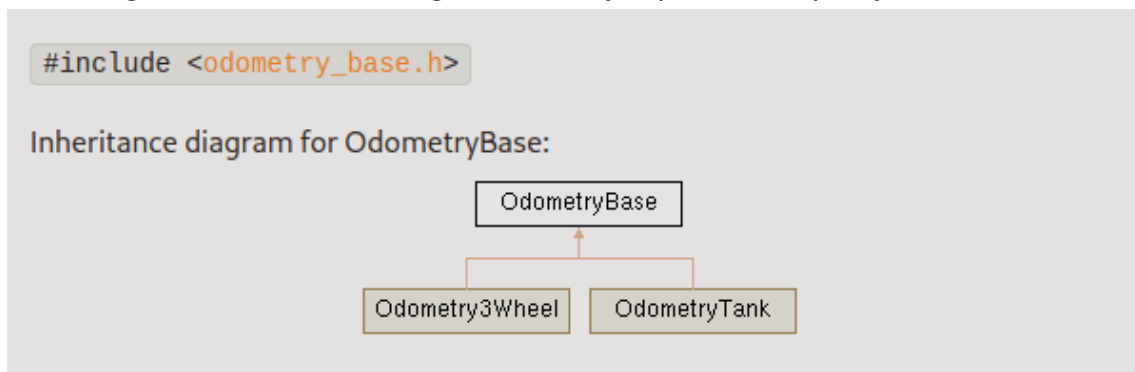


Figure 5: Automatically generated documentation.

Auto-Notebook

Alongside the automatic documentation, whenever Core is updated or we manually trigger it, a Github Action copies the reference manual, exports the most up to date version of our written notebook document, stitches them together, and deploys to a webpage. This is publicly available for any person wishing to see our software development process. The most valuable effect, though, is automating most of the formatting work for our notebook, work that used to require a team member to use valuable pre-competition time to sit down, append, format and export the notebook.

Clang-Tidy

In an effort to improve the quality, reduce headaches, and make our code easier to read, write, and understand, we enabled many more warnings than what is supplied with the default Vex project Makefile. These warnings deal with uninitialized variables, missing returns, and other simple code errors that nonetheless have the tendency to introduce tiny, hard to track down bugs. However, sometimes these warnings do not explore deep enough and another tool must be used. We integrated clang-tidy, a c++ linter developed by the clang compiler project, to inspect our code. With a simple switch of a variable in the Makefile, we run clang-tidy during builds which gives many insights into the code that plain compiler warnings do not. Though it does increase compilation times, it tells us about code that is bug prone or poor for performance and tests many other checks developed and validated by the wider C++ community.

Wiki

Whenever a new feature is added to Core, we create a Wiki page on the Core Github repository that provides documentation on what the function does, how to use it, and some examples of how it can be used. This documentation is easily accessible as it can be found online within the Core repository itself. This allows for new members to get acquainted with Core faster and easier than before. This allows us to speed up our training process and allow new members to start developing sooner rather than later. In addition it provides us and anyone using Core great documentation that not only goes into method signatures and descriptions, but also detailed explanations of what different methods, classes or functions do.

Opcontrol

This class provides two ways of driving the robot with a controller: Tank drive and Arcade drive. Drivers can choose what they're most comfortable with.

Tank Drive - The left joystick controls the left-side motors, and the right joystick controls the right-side motors

Arcade Drive - Acts somewhat to how modern racing video games are controlled. The left joystick controls the forward / backward speed, and the right joystick controls turning left / right.

Both functions also have an optional parameter called `power`, and refers to how the joystick is scaled to the motors. The higher the power is, the more control you have over low-speed maneuvers. Because the scaling is non-linear, it may feel weird to those who aren't used to it.

Method Signatures

```
void drive_tank(double left, double right, int power=1);  
void drive_arcade(double forward_back, double left_right, int power=1);
```

Usage Examples

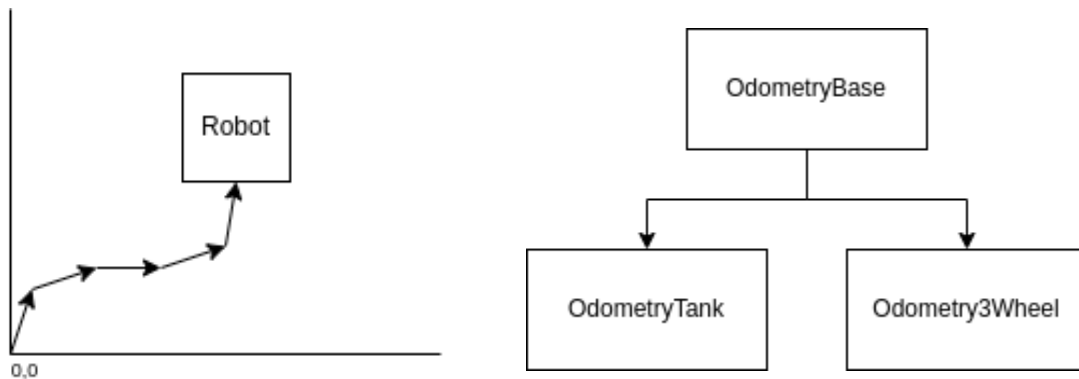
```
drive_system.drive_tank(controller.Axis3.position() / 100.0, controller.Axis2.position() / 100.0);  
drive_system.drive_arcade(controller.Axis3.position() / 100.0, controller.Axis1.position() / 100.0);
```

Figure 6: A screenshot of the Core Wiki

Core: Fundamentals

Odometry

In order for the robot to drive autonomously, it needs to know where it is, and constantly monitor changes to sensors. The Odometry subsystem takes inputs from encoders, and using vector math and previous position data, calculates the position and rotation of the robot on the field as a point in space (X, Y), and heading (deg).



The Odometry subsystem is broken down into an OdometryBase class, which controls the asynchronous behavior and getters/setters, and OdometryTank and Odometry3Wheel classes, which both extend OdometryBase and implement a two-encoder algorithm and a three-encoder algorithm, respectively.

GPS + Odometry

In order to fit an 8-motor drivetrain into the 15" size requirement, the robots could not fit non-powered odometry wheels, leaving only the drive encoders to be used for position tracking. This isn't ideal, since sudden changes in acceleration and wheel slippage can easily cause the tracking to drift a substantial amount. To combat drifting, we looked to the GPS sensor for localization.

The GPS sensor uses a tag-based approach for localization, using a coded strip around the perimeter of the field to estimate position. Between pose estimates, the integrated IMU provides inertial information to estimate changes in position and heading for a constant flow of data, presumably using some sort of onboard Kalman filter. The pose (X, Y, Heading) data is sent back to the Brain over the smart port. In addition, the GPS provides a "quality" value, which is a percentage that increases when the camera can see a large amount of tape, and decreases when the camera is blocked and the IMU detects change in position over a period of time.

To properly characterize the GPS sensor, X/Y/Heading data points were gathered at different positions around the field, facing different headings. The following graphs show the data points on the 12' x 12' field grid. Distance error (in inches) to the actual measurements is shown by color.

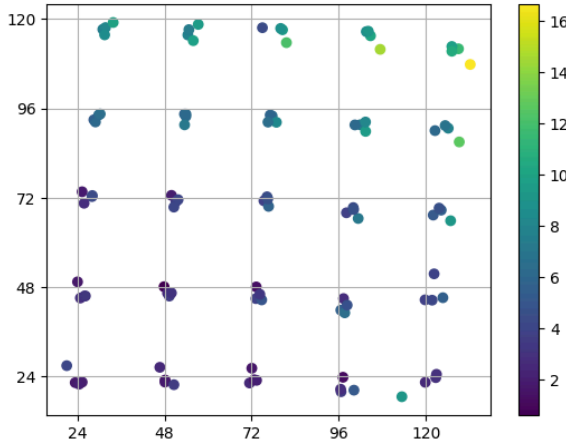


Figure 7 - Raw Data Points

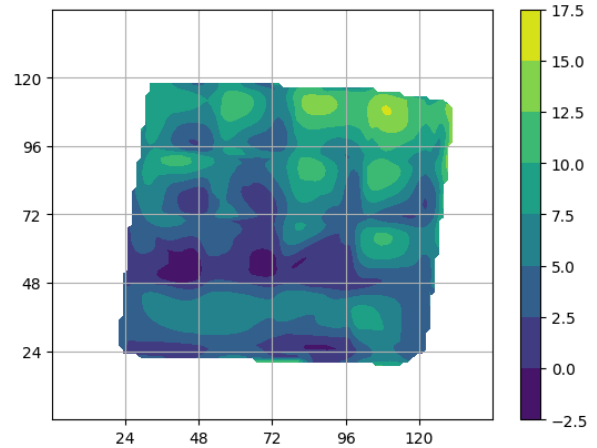


Figure 8 - Error Heat Map

There were some other errors with the GPS sensor, including issues localizing the robot when the sensor could not see enough of the coded tape. To take full advantage of the GPS sensor's localization capability, we'd need a way to perform sensor fusion alongside traditional ground-based odometry. To do this, a complementary filter was chosen - a filter that mixes two sets of data based on a proportional scalar α (α). the equation for a complementary filter is shown below:

$$out = \alpha * s_1 + (1 - \alpha) * s_2$$

where s_1 is *sensor 1*, s_2 is *sensor 2*, and alpha scales between the two.

To calculate alpha, first the X,Y position of the sensor and heading is taken into account. Since the robot will generally have a more accurate position when it's close to the wall and facing away from it, the following formula will report a score between 0 and 1 for the filter:

$$\alpha = \left(\frac{\vec{c} - \vec{p}}{\|\vec{c} - \vec{p}\|} \cdot \begin{bmatrix} \cos \theta \\ \sin \theta \end{bmatrix} \right) * \frac{\|\vec{c} - \vec{p}\|}{\|\vec{c}\|} * q$$

where $\vec{c}$ is the constant vector of a point in the center of the field ($x=72in$, $y=72in$), $\vec{p}$ is the robot's position as a vector (x , y), θ is the robot's heading, and x,y is again the robot's position (0 to 144 inches). The left side is the dot product of the normalized vector pointing from the robot to the center with the direction the robot is facing (gives 1 when the directions are aligned and -1 when opposite smoothly changing in between), and the right side is the sensor's distance to the center as a scalar percentage of the distance from corner to corner (between 0 and 1). Finally, q is the GPS's reported quality. We then remap this from the range $[-1, 1]$ to $[0, 1]$ when we do our mixing.

Drivetrain

A drivetrain class has two functions: To control the robot remotely, and autonomously. In the Core API, the TankDrive class allows the operator to control the robot using Tank controls (Left stick controls the left drive wheels, right controls the right), and Arcade controls (Left stick is forward / backwards, right stick is turning). This means drivers can tailor their controls to whichever feels more natural.

Tank Drivetrain

Brake Mode

A VEX driver has many things to keep track of during a match. From game element position, match load status, and partner robot condition there is a great deal going on. Defense is another layer on top of the mental load of playing the game. To ease this burden, we implemented a brake mode on our drive train. It is a multi-modal system that can either bring the robot to a stop or hold the robot in a specific location on the field. We use the motion profiles we developed for auto programming to decelerate the robot when requested and use our auto driving functions to hold the robot's position. We implement a smarter form of position holding than just motor braking as we can return to the exact location on the field. Additionally, we combine our deceleration control with position holding such that we do not immediately “lock the brakes” and skid away thus losing the position we attempt to hold and making driving incredibly difficult.

Autonomous Driving

For autonomous driving, the TankDrive class has multiple functions:

- `drive_forward()`:
 - Drive X inches forward/back from the current position
 - Signature: `drive_forward(double inches, directionType dir, double max_speed=1)`
- `turn_degrees()`:
 - Drive X degrees CW/CCW from the current rotation
 - Signature: `turn_degrees(double degrees, double max_speed=1)`
- `drive_to_point()`
 - Drive to an absolute point on the field, using odometry
 - Signature: `drive_to_point(double x, double y, vex::directionType dir, double max_speed=1);`
- `turn_to_heading()`
 - Turn to an absolute heading relative to the field, using odometry
 - Signature: `turn_to_heading(double heading_deg, double max_speed=1)`

Generally, it is better to use `drive_to_point` and `turn_to_heading` to avoid compounding errors in position over relative movements. These functions implement the `FeedbackBase` class, so any control loop can be used to control it.

Drive To Point

The defining feature of a drive to point function is the ability for a robot to calculate a relative direction and distance between its own position and the target position, and navigate to it using tuned control loops. The steps taken for our implementation are listed below.

1 - Gather information

To drive towards a specific point, the robot must know the change in angle between the robot's heading and the target, and the distance to the target. To get this, we first grab the robot's current position and heading and create a positional difference vector between this and the new point.

```
pose_t current_pos = odometry->get_position();
pose_t end_pos = {.x = x, .y = y};

point_t pos_diff_pt =
{
    .x = x - current_pos.x,
    .y = y - current_pos.y
};

Vector2D point_vec(pos_diff_pt);
```

Using this information, grab the distance to the target (using a function in the Odometry subsystem). An issue with the pure distance between points is that it does not represent how far the robot has to travel to be considered "on target" in the control loop. In order to properly reach its target, the robot should report its "aligned distance", and ignore the lateral error, as per Figure 9. This should only hold true when the robot is close to the target, or inside a given radius that is tuned by the user.

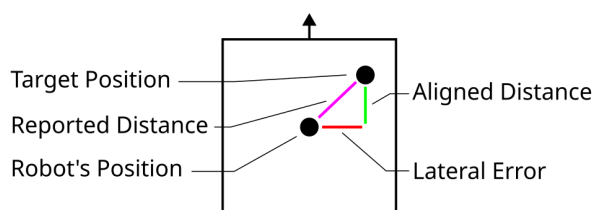


Figure 9 - Distance Modification

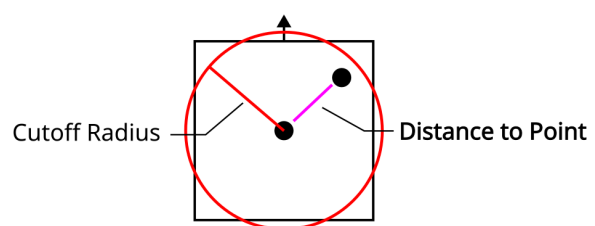


Figure 10 - Correction Cutoff Circle

```
double dist_left = OdometryBase::pos_diff(current_pos, end_pos);

if (fabs(dist_left) < config.drive_correction_cutoff)
{
    dist_left *= fabs(cos(angle * PI / 180.0));
}
```

The next data needed is the difference in angle between the robot's current heading and the vector between the robot's position and the target. This is calculated by using the arctangent of the difference vector, and subtracting it from the robot's current heading. The angle is then wrapped around 360 degrees.

```
double angle_to_point = atan2(y - current_pos.y, x - current_pos.x)
    * 180.0 / PI;

double angle = fmod(current_pos.rot - angle_to_point, 360.0);
if (angle > 360)
    angle -= 360;
if (angle < 0)
    angle += 360;

double heading = rad2deg(point_vec.get_dir());
double delta_heading = 0;
if (dir == directionType::fwd)
    delta_heading = OdometryBase::smallest_angle(current_pos.rot, heading);
else
    delta_heading = OdometryBase::smallest_angle(current_pos.rot
        - 180, heading);
```

The last piece of information needed is whether the robot should be moving forwards or backwards. Since the distance is calculated as $\sqrt{x^2 + y^2}$, the sign is lost when squaring. Re-implement the sign based on the angle and initial driving direction.

```
int sign = 1;
if (dir == directionType::fwd && angle > 90 && angle < 270)
    sign = -1;
else if (dir == directionType::rev && (angle < 90 || angle > 270))
    sign = -1;
```

2 - Setting Control Loops

In this section, the robot takes the above information and sets its feedback loops. Since the function takes in a FeedbackBase abstract class, any feedback can be used to drive the robot's correction and linear movements. The most common situation is a trapezoidal motion profile for linear distance with PD for heading correction. Once the robot is close enough to the target point, the correction feedback is ignored to avoid issues with last-minute heading changes.

```
correction_pid.update(delta_heading);
feedback.update(sign * -1 * dist_left);

double correction = 0;
if (is_pure_pursuit || fabs(dist_left) > config.drive_correction_cutoff)
{
    correction = correction_pid.get();
}

double drive_pid_rval;
if (dir == directionType::rev) {
    drive_pid_rval = feedback.get() * -1;
} else {
    drive_pid_rval = feedback.get();
}

double lside = drive_pid_rval + correction;
double rside = drive_pid_rval - correction;

lside = clamp(lside, -max_speed, max_speed);
rside = clamp(rside, -max_speed, max_speed);

drive_tank(lside, rside);
```

Finally, when the linear feedback reports its on target, stop, return and report that the movement is over.

```
if (feedback.is_on_target())
{
    if (end_speed == 0) {
        stop();
    }
    func_initialized = false;
    return true;
}
```

Pure Pursuit

Pure Pursuit is a method of autonomous robot driving that allows the robot to autonomously drive through a set of waypoints without stopping and turning.

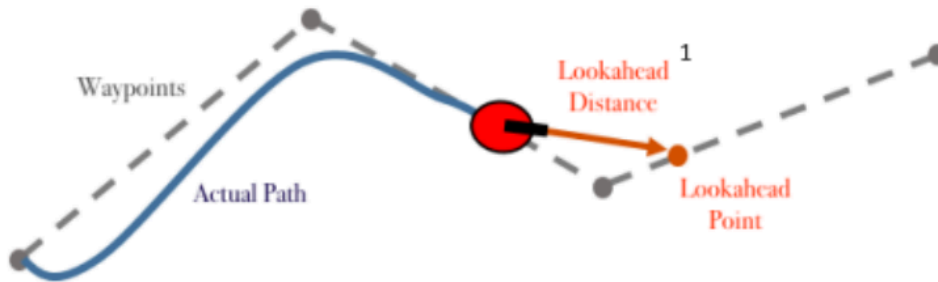


Figure 11 - Pure Pursuit Example

This is accomplished by taking a list of points (x,y), connecting them, and then choosing a "lookahead point" along the lines that the robot will attempt to follow. This will inherently cause the robot to smooth out the point map, and follow without sudden changes in direction.

The lookahead point is chosen by iterating along the path created by connecting the points and finding the furthest point that intersects a circle centered on the robot, with a set radius tuned by the programmer. Increasing this radius smooths the path, while decreasing it ensures the robot more closely follows the path.

The pure pursuit implementation in Core can either use the Autonomous Command System (ACS), or be called directly through the TankDrive class. See the sample code below for examples.

```
// Autonomous Command Controller
CommandController cmd{
    drive_sys.PurePursuitCmd(PurePursuit::Path({
        {.x=19, .y=133},
        {.x=40, .y=136},
        {.x=92, .y=136},
    }, 8), directionType::rev, .5)->withTimeout(4),
};
cmd.run();

// Standalone
while(!drive_sys.pure_pursuit(PurePursuit::Path({
    {.x=19, .y=133},
    {.x=40, .y=136},
    {.x=92, .y=136}}, 8), directionType::fwd, 0.5))
{
    vexDelay(20);
}
```

Control Loops

In order for the Autonomous Command Structure to function, we need a way to tell the robot how we want it to move. There are two broad categories of telling a robot to achieve a requested position - Feedback and Feedforward. Feedback relies on sensors and adjusts the output of the robot according to the error between where it is and where it wants to be. On the other hand, a feedforward controller takes a mathematical model of the system and creates outputs based on what it calculates to be the necessary output to achieve the goal. Additionally, there are simpler methods like Bang-Bang or Take Back Half. These adjust the outputs based on the current position relative to the target, where Take Back Half gradually refines the output until it settles at the desired position. These controller types work for many applications, but a combination of them can achieve an even better control over robot actuators.

PID

A PID controller is perhaps the most common type of Feedback control. It uses measurements of the error at its current state (proportional), measurements of how the error was in the past (integral) and measurements of how the error changes over time(derivative). The controller acts accordingly to bring the errors towards 0. We implemented a standard PID controller but made some alterations to fit our needs. The most important of these are custom error calculations. The standard error calculation function (*target - measured*) works for many of our uses but causes problems when we use a PID controller to control angles. Since angles wrap around at 360 degrees or 2π radians we wrote our own error calculation function that gives the error that accounts for this wrapping.

Feedforward

A feedforward controller differs from a feedback controller in that it does not rely on any measurement of error to command a system. Instead, built into a feedforward controller is a mathematical model of the domain. When a target is requested by the controller, the model is queried to figure out what the robot actuators must output to achieve that target. A key advantage of this form of control is that instead of waiting for an error to build up in the system, the controller acts directly to achieve the target and can reach the target much faster.

Bang-Bang

Bang-Bang control is a straightforward control methodology where the output to the system is either fully on or fully off, with no intermediate states. It's used for systems where fine control isn't necessary or possible. In this method, when the process variable is below the setpoint, the controller output is set to maximum; when above, it's set to minimum. This approach is simple and often used for systems with high inertia or where the precise control of the variable isn't critical. However, it can lead to oscillations around the setpoint and isn't suited for systems requiring precise regulation.

Take Back Half (TBH)

The Take Back Half (TBH) method is an iterative approach used to refine control in systems where overshoot is a concern. This method adjusts the output by taking back half the value of the output each time the controlled variable overshoots the target. The adjustment continues until the system settles close to the desired setpoint. TBH is particularly useful in scenarios where a fine balance between responsiveness and stability is needed, as it reduces the oscillation or overshoot often seen in simpler control methods. It's a practical choice for systems where a PID controller might be too complex or unnecessary. TBH controllers only have one tuning parameter which allows for an incredibly easy tuning experience.

Generic Feedback

Different control systems work best in different environments. Because of this, we found ourselves switching control schemes often enough that rewriting the code each time was time consuming and often led to rushed, worse quality code. To solve this problem we implemented a generic feedback interface so that none of our subsystem code needs to change when we use a different control scheme. Instead, the subsystem reports to the controller where it wants to be, measurements from its environment and some information about the system's capabilities and the controller will report back the actions needed to achieve that target. This allows for much faster prototyping as well as cleaner, less tightly coupled code.

Motion Profile

As we learn from each event, our team has evolved our approach to robot control systems, transitioning from a simple PID controller to a more sophisticated Motion Profile controller. The PID system, while fundamental, had its drawbacks, such as limited speed specification, poor response to wheel slipping, and slower reaction times. These limitations highlighted the need for a more advanced control mechanism.

Our Motion Profile controller represents a significant upgrade. It integrates precise control over position, acceleration, and velocity, allowing for optimized performance of our robot's subsystems. Unlike the PID controller, which reacts only to discrepancies between actual and desired states, our Motion Profile controller proactively manages the robot's movements. It anticipates the required actions, thereby reducing response lags. Moreover, it avoids the rigidity of a pure feedforward controller by adapting dynamically to changing conditions in competition scenarios.

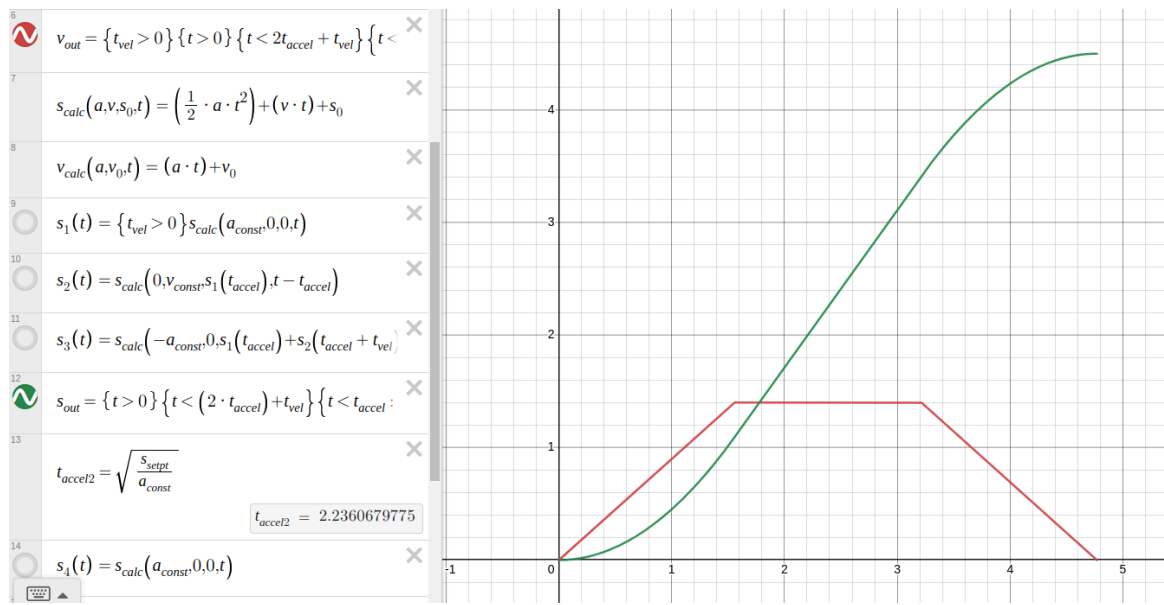


Figure 12: Trapezoidal motion profile

A key feature of the Motion Profile controller is its ability to handle varying accelerations. This functionality enables our robot to accelerate efficiently without wheel slipping, always maintaining optimal acceleration. This year, we've further refined our Motion Profile to accommodate non-zero starting and ending velocities. This enhancement allows for the seamless chaining of complex movements, ensuring smoother transitions and more fluid motion during competition tasks.

Auto Command Structure (ACS)

Principle

A recent addition to our core API was that of the Autonomous Command Structure. No more will our eyes glaze over staring at brackets as we trawl through an ocean of anonymous functions nor lose our way in a labyrinthine state machine constructed not of brick and stone but blocks of ifs and whiles. Instead, we provide named Commands for all the actions that our robot can execute and infrastructure to run them sequentially or concurrently. The API is written in a declarative way allowing even programmers unfamiliar with the code to see a step-by-step, annotated guide to our autonomous path while keeping the procedures of how to execute the actions from hurting the readability of the path.

```

CommandController auto_non_loader_side(){
    int non_loader_side_full_court_shot_rpm = 3000;
    CommandController non_loader_side_auto;

    non_loader_side_auto.add(new SpinRPMCommand(flywheel_sys, non_loader_side_full_court_shot_rpm));
    non_loader_side_auto.add(new WaitUntilUpToSpeedCommand(flywheel_sys, 10));
    non_loader_side_auto.add(new ShootCommand(intake, 2));
    non_loader_side_auto.add(new FlywheelStopCommand(flywheel_sys));
    non_loader_side_auto.add(new TurnDegreesCommand(drive_sys, turn_fast_mprofile, -60, 1));
    non_loader_side_auto.add(new DriveForwardCommand(drive_sys, drive_fast_mprofile, 20, fwd, 1));
    non_loader_side_auto.add(new TurnDegreesCommand(drive_sys, turn_fast_mprofile, -90, 1));
    non_loader_side_auto.add(new DriveForwardCommand(drive_sys, drive_fast_mprofile, 2, fwd, 1));
    non_loader_side_auto.add(new SpinRollerCommand(roller));

    return non_loader_side_auto;
}

```

Figure 13: ACS code from the 2023 competition season

Updates

This season, we found ourselves annoyed with having to repeat basic things such as `path.add(...)` and having to write `new ThingCommand(...)` over and over again. Our first solution to this was “shortcuts”. These were member functions of subsystems that would allocate, initialize and return an auto command for that subsystem. So, instead of `path.add(new DriveForwardCommand(drive_sys, drive_fast_mprofile, 20, fwd))` we could simply write `path.add(drive_sys.DriveForwardCommand(20, fwd))`. This reduced a great deal of typing but still left us with some issues.

The most hazardous, rather than the simply annoying downside of last year's system, was the memory unsafety of this system. Since our auto commands must use virtual functions, they must be on the other end of a pointer. So, they must be allocated using `new` or they must be initialized statically before we write the path which is a terrible user experience (Though, if constrained by an embedded system where allocating on the heap was deemed dangerous, the system could work with this). This became a real issue when we began to write more complicated constructs such as branching, asynchronous, and repeated commands as it became dangerously unclear who was responsible for deallocating these objects. As a solution for this, we developed an RAIL wrapper for the Auto Command Interface. Inspired by C++'s `std::unique_ptr`, this wrapper provides a memory safe, value based way of using auto commands while still maintaining their adaptability. We used C++'s ideas of move semantics and ‘Resource Allocation Is Initialization’ to practically solve memory management so programmers (and even non programmers) can focus on writing paths.

```

CommandController cmd{
    odom.SetPositionCmd({.x = 16.0, .y = 16.0, .rot = 225}),
    // 1 - Turn and shoot preload
    {
        cata_sys.Fire(),
        drive_sys.DriveForwardCmd(dist, REV),
        DelayCommand(300),
        cata_sys.StopFiring(),
        cata_sys.IntakeFully(),
    },
    // 2 - Turn to matchload zone & begin matchloading
    drive_sys.DriveForwardCmd(dist + 2, FWD, 0.5)
        .with_timeout(1.5),

    // Matchloading phase
    Repeat{
        odom.SetPositionCmd({.x = 16.0, .y = 16.0, .rot = 225}),
        intakeToCata.with_timeout(1.75),
        cata_sys.Fire(),
        drive_sys.DriveForwardCmd(10, REV, 0.5),
        cata_sys.StopFiring(),
        cata_sys.IntakeFully(),
        drive_sys.TurnToHeadingCmd(load_angle, 0.5),
        drive_sys.DriveForwardCmd(12, FWD, 0.2).with_timeout(1.7),
    }.until(TimeSinceStartExceeds(30))
};

```

Figure 13: ACS code going into the 2024 competition season

Now that we were free to use auto commands without fear for leaking memory or messing with currently running commands, we began to create more powerful constructs such as branching on runtime information, timeouts so the robot can decide what to do based on how much time is left in the auto or skills period, fearless concurrency (driving and reloading at the same time), and a much much nicer user interface. This declarative, safe, and straightforward method of writing auto paths lets us spend less time writing and debugging custom code and more time exploring and optimizing auto paths.

Serializer

One pain point we found last year was configuring auto paths, color targets, path timeouts, and other parameters that changed often but for the most part should be persistent. Commonly, we found ourselves redeploying code at the last minute before a match. To solve this, we wrote a class that takes control of a file on the SD card to which users can read and write values at runtime using a simple key-value interface. This keeps us from having to change a value, redeploy, repeat which cost us valuable time in the past.

Screen Subsystem

Principle

One of the most powerful elements of the V5 Brain is the fairly substantial touch screen. However, its simple drawing API limits its utility as one person's part of the code will draw over another since there is no larger abstraction controlling who draws when. We have many different subsystems on our robot to observe and debug and many parameters that can be tuned at run time and the screen provides a way to do this. We provide an API that provides a 'page' interface that can be inserted into a slideshow-like interface. Each 'page' provides two functions, an update and a draw. The update runs more frequently allowing touch input and data collection at a reasonably fast rate while the draw function runs less frequently to not cause too much overhead on the system. At startup, users provide the screen subsystem a list of pages and the screen subsystem handles orchestration and input in a background thread while other robot code runs unaffected.

```
pages = {  
    new AutoChooser({"Auto 1", "Auto 2", "Auto 3", "Auto 4"}),  
    new screen::StatsPage(motor_names),  
    new screen::OdometryPage(odom, 12, 12, true),  
    cata_sys.Page(),  
};  
  
screen::start_screen(Brain.Screen, pages);
```

Figure 14: Configuration for the screen subsystem

Pages

Odometry Page

The odometry page has proved incredibly useful in writing and debugging auto and auto skills paths. It shows a picture of the robot on the field as well as a print out of the actual x,y coordinates and heading of the robot. Since we write our autos with respect to the coordinate system of the field, having a map to look at makes development much simpler.

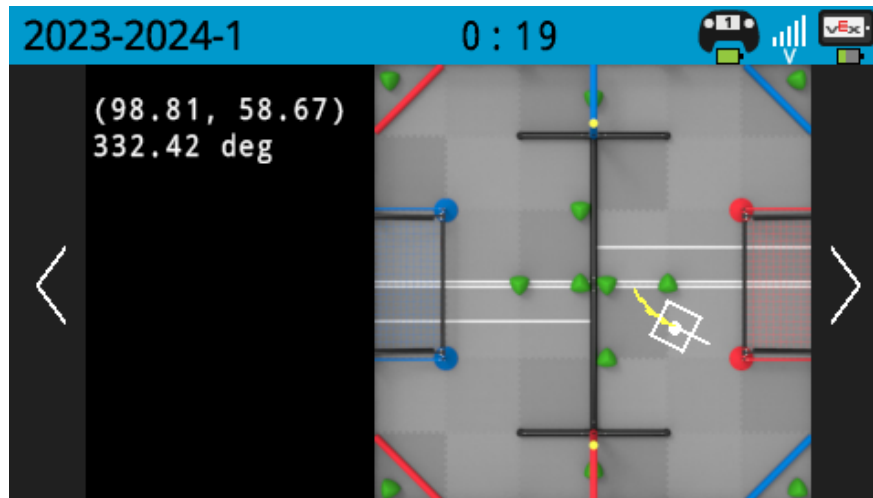


Figure 15: A field display for the Over Under season

PID Tuner

PID controllers are integral to many subsystems on our robots. Our drive code uses them for turning and forward motion, our catapult uses them for reloading, and subsystems across seasons require them for precise control. Tuning them, however, can be incredibly tedious. Changing one value, redeploying, and repeating over and over again is time consuming and unnecessary. Since we have a wonderful touchscreen, we simply added a series of sliders for PID parameters and we can now easily adjust a PID tuning in seconds rather than minutes saving a great deal of time on an already time consuming part of robot development.

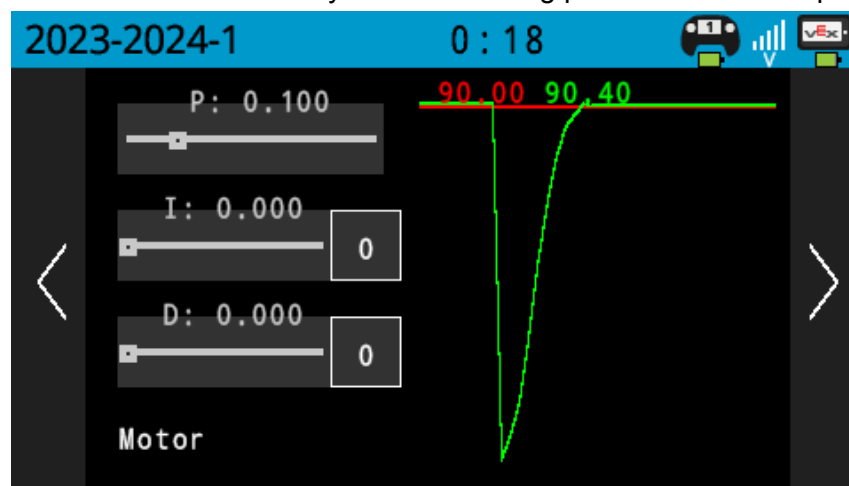


Figure 16: Tuning a motor for reaching an angle

Motor Stats

One would think it an easy step to remember to plug motors in, and yet multiple times this season we have been bewildered and hindered by an unplugged motor. This page was written to continuously display that the motor had been unplugged and was not cancellable like the built-in VEX alert. This screen also displays what port to plug it into as well as a color coded

temperature displaying when the robot needs to cool down. This tool proved extremely useful as we discovered an alarmingly high number of dead or nonfunctioning ports on the brain.



Figure 17: Motor Stats screen from our 2023 robot

Cata System Page

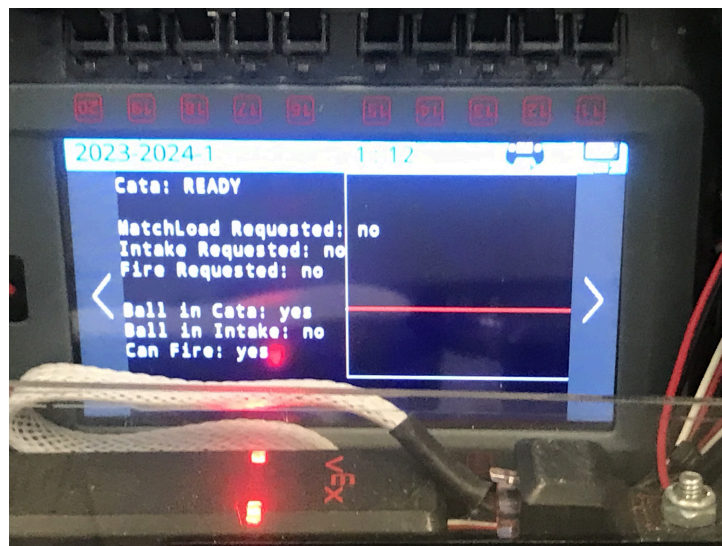


Figure 18: The catapult status page. Includes a graph of Catapult PID values.

Catapult System

Motivation

Vex's Over Under game requires the effective utilization of the fascinatingly shaped triball. After much deliberation, our team decided on a catapult to launch the game element across the field and a reversible intake for picking up and scoring the triball. This system gives us a great deal of flexibility and power for strategy but does increase the system's complexity. This complexity mostly stems from the orchestration of intaking with catapult reloading such that we do not jam our catapult and never intake multiple game pieces leading to a disqualification.

Initial Design

We implemented a state machine that receives inputs from the controller, a distance sensor in the intake, a distance sensor in the catapult, and a potentiometer for watching the catapult's position. The system runs the appropriate motors to either intake to hold the triball, reset the catapult, intake into the catapult, or shoot depending on its state. Because we have so many sensors, we can determine when intaking would lead to disqualification and simply not honor the intaking command.

These messages are a simple enum that one passes to `CataSys::send_command()`. This was originally intended to make writing multi-threaded code less error-prone as there was one thread-safe and simple way to interact with the subsystem, rather than many disparate methods some of which are meant for internal usage of the class on the running thread and some accessors and setters meant to be used from the user thread. Although it started for implementation ease, it naturally brought about a very simple interface for auto. Instead of sending a command on a button press, we simply send a command at a certain point in our auto path and the system reacts accordingly.

Successor

Though the idea of the state machine modeled the intake and catapult system well, our haphazard implementation (very large and complicated switch statement on a worker thread) made changes exceedingly difficult. As we began competing, we identified changes we wished we could make to make driver and autonomous control easier and handle unforeseen hardware faults. However, our system was hard to read, modify, and all but impossible to prove correct.

Inspired by TinyFSM and other off the shelf C++ libraries for this problem, we created a generic state machine class that handles state transitions, background thread execution, and observability. While this tradeoff led to more code overall, its explicitness and separation of concerns allowed members to make changes in the behavior of the system without fear of deadlocking the threads or unknowingly modifying other states. The generic StateMachine class will remain in our Core library and can be reused year to year to achieve these benefits for any other stateful subsystem.

```

struct Reloading : public CataOnlySys::State {
    void entry(CataOnlySys &sys) override {

        sys.pid.update(sys.pot.angle(vex::deg));
        sys.pid.set_target(cata_target_charge);
    }

    CataOnlySys::MaybeMessage work(CataOnlySys &sys) override {
        // work on motor
        double cata_deg = sys.pot.angle(vex::deg);
        if (cata_deg == 0.0) {
            // adc hasnt warmed up yet, we're getting zero results
            return {};
        }
        sys.pid.update(cata_deg);
        sys.mot.spin(vex::fwd, sys.pid.get(), vex::volt);

        // are we there yettt
        if (sys.pid.is_on_target()) {
            return CataOnlyMessage::DoneReloading;
        }
        // otherwise keep chugging
        return {};
    }

    CataOnlyState id() const override { return CataOnlyState::Reloading; }
    State *respond(CataOnlySys &sys, CataOnlyMessage m) override;
};

```

Explicit separation of states allows simpler, more readable code

Due to the message passing interface to the catapult and intake system, this change was able to be made with very few modifications to the external interface of the system meaning driver code and autonomous paths did not have to be rewritten to use the advantages of the new system - a saving grace as we made this modification in the middle of competition season.

Vision

With the unpredictable way triballs roll across the field, our robots need a way to repeatably track the game objects during the autonomous period. And so, a vision sensor is placed inside the intake subsystem on the front of the robot.

The Vex Vision sensor is notorious among teams for being unreliable, being highly dependent on field lighting conditions and often sensing random objects, sending the robot off course. Our team explored different methods of filtering and lighting to combat these issues, and are now successfully tracking triballs in our autonomous programs.

Filtering Vision Objects

The first issue to address was filtering - making sure the robot tracks the correct objects. Currently, we run a filtering algorithm that removes all vision objects that don't follow a strict criteria:

- Minimum area (object width * height)
- Maximum area
- Minimum aspect ratio (object width / height)
- Maximum aspect ratio
- Min / Max X value
- Min / Max Y value

Finally, the filtered objects array is sorted by area, so that the largest objects are easily accessible at the start of the array. Here's an example of how it's used:

```
vision_filter_s filter{
    .min_area = 2000,
    .max_area = 100000,
    .aspect_low = 0.5,
    .aspect_high = 2,
    .min_x = 0,
    .max_x = 320,
    .min_y = 0,
    .max_y = 240,
};
vector<vision::object> obj_list = vision_run_filter(filter);
```

Standardizing Lighting

In past competitions, we've found differing lighting conditions can spell an unfortunate end for autonomous programs using vision. Spotlights, windows and even the color temperature of the overhead lights caused slight differences which would cause the color profile to be off. We were able to completely eliminate this by adding a custom light to the robot - a board that uses two high-powered LEDs switched with a MOSFET over the three wire ports. Here's the schematic:

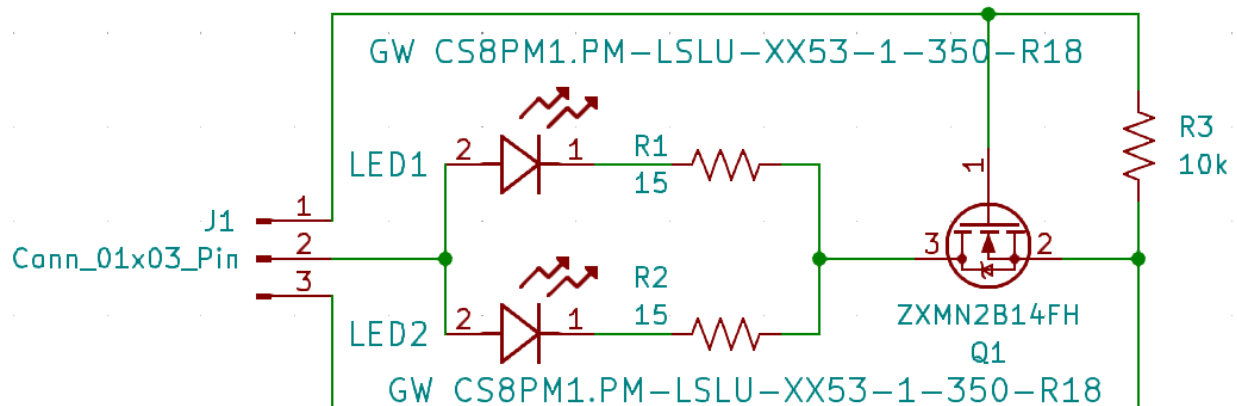


Figure 19 - LED Board schematic

The low-side FET switches power via the signal pin, allowing the programmer to use PWM to dim the lights as needed.

Here's the final PCB built for competition:

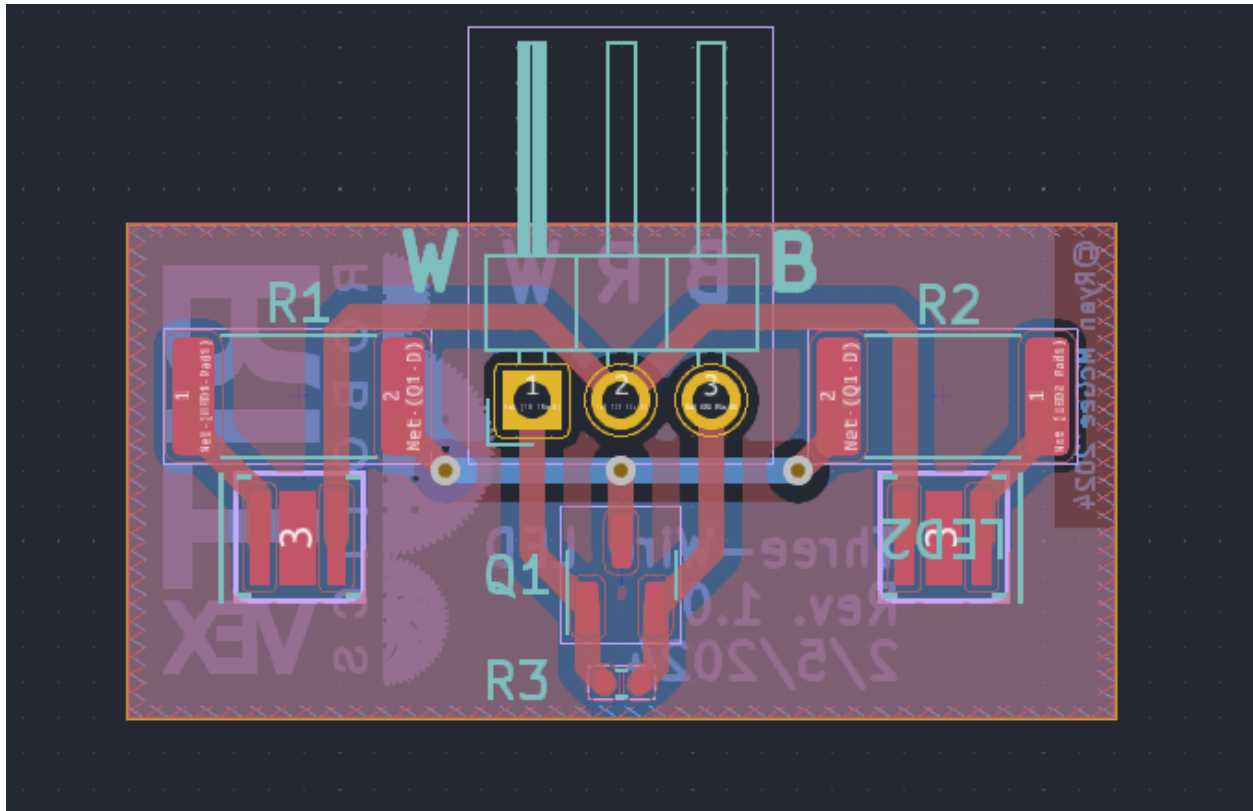


Figure 20 - LED Board PCB design

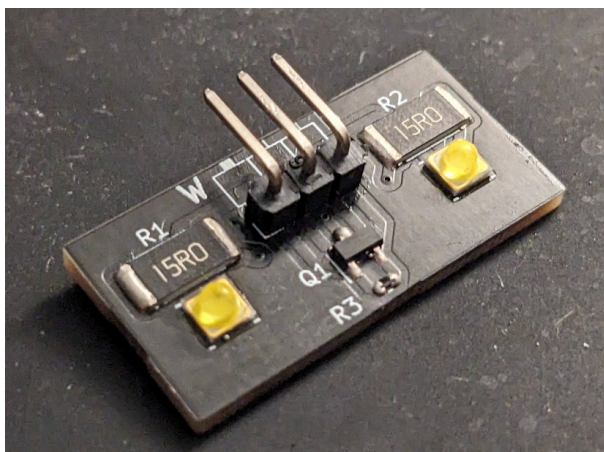


Figure 21 - LED Board, Front

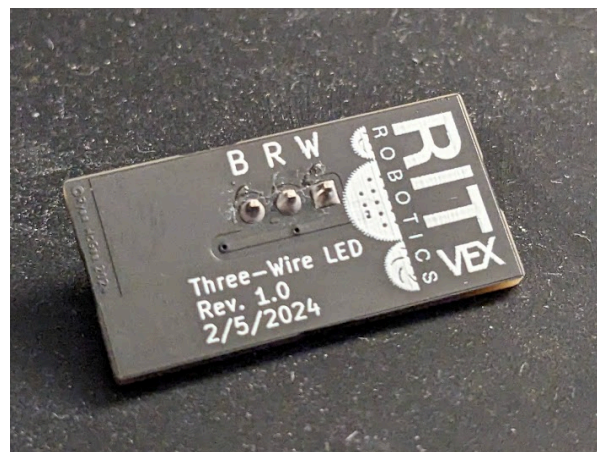


Figure 22 - LED Board, Back

The lighting system was tested and successfully used at the West Virginia tournament, where the robot was able to score five triballs in one autonomous program using this vision system.

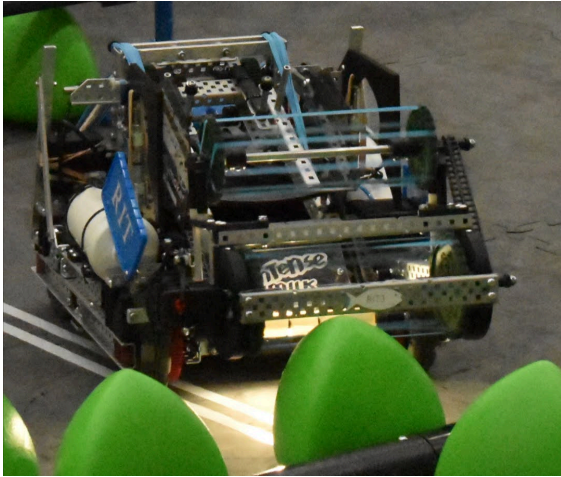


Figure 23 - Vision Tracking in Auto



Figure 24 - LED board at full power (~2 Watts)

Core: Ongoing Projects

Rust

Over the course of the year, we have experimented with rewriting our Core API in Rust, a multi-paradigm programming language focused on performance and safety. Rust offers several potential advantages over C++:

Motivation

Memory Safety

One of the primary benefits of Rust is its emphasis on memory safety without sacrificing performance. Rust's ownership model ensures that memory is managed correctly at compile time, reducing the risk of memory leaks and buffer overflows which are common issues in C++. This is especially crucial in robotics, where memory management errors can lead to system crashes or unpredictable behavior in real-time operations.

Concurrency

Rust's approach to concurrency is another major advantage. Concurrency errors, like race conditions, are hard to debug and can be catastrophic in robotics, leading to inconsistent states and erratic behavior. Rust's type system and ownership model prevent data races at compile time, making concurrent programming more reliable and easier to reason about.

Performance

In terms of performance, Rust is comparable to C++, which is essential in robotics where processing speed and response time are critical. Rust's zero-cost abstractions mean that high-level constructs do not add overhead at runtime. This allows developers to write high-level code without compromising on performance, an important consideration in robotics where every millisecond can count.

Improved Code Maintenance and Readability

Rust also offers improved code maintainability and readability. Its modern syntax and language features make it easier to write clear and concise code. This reduces the cognitive load on developers, making it easier to develop and maintain complex robotic systems. The compiler's strictness also ensures that many potential bugs are caught early in the development cycle, reducing the time spent on debugging.

Growing Ecosystem and Community

The Rust ecosystem is rapidly growing, with a strong focus on safety and performance. There are increasing numbers of libraries and tools being developed for Rust, including those specifically for robotics. The Rust community is known for its dedication to improving code quality and security, which aligns well with the needs of robotics development.

Overall

While the transition from C++ to Rust in a robotics context requires investment in terms of learning and codebase modification, the benefits in memory safety, concurrency handling, performance, and maintainability make it a compelling choice. The modern features of Rust, combined with its growing ecosystem and community support, position it well for developing robust, efficient, and safe robotic systems.

Progress

Though progress slowed as competition season began, our rust build system is moving out of the proof of concept stage and into something useful. We setup a cargo (rust's build system manager) target and can cargo build a vex project into an architecture-correct .elf file linked according to vexcode's standard library version and linker configurations. We then created a simple python script to convert the .elf file into the stripped binary file that the vex brain expects and call the vexcom tool provided by the vscode extension to send binaries to the brain.

Findings

Though we did not have much time before our small software team's resources were needed elsewhere, our experiments with Rust programming for VEX found many interesting things.

A surprise we came across is that for proper and safe rust environments one must provide a panic handler. This will be called whenever an error occurs or the programmer signals that the specified behavior is invalid. Though rust does many things to insure 'if it compiles

correctly it runs correctly' there is still behavior that should be signaled to be an error at run time. With the custom panic handler we are able to provide detailed error messages including line numbers and function names - a feature that is sorely missed when programming with the C/C++ API.

Though Rust does come with many benefits, we did find a blocker that is limiting more widespread adoption on the team. The C/C++ API dynamically links the C and C++ standard library after deploying such that a much smaller binary must be transferred to the brain, a life saver when wirelessly uploading. Even with aggressive minimal size optimizations, the requirement to statically link rust core library functions means even simple rust binaries would match the size of our largest C/C++ projects. The PROS ecosystem ran into a similar problem and did work with hot/cold linking in order to not deploy non-changing code each time and we are looking to explore a similar solution. However, most of our research is into undocumented areas of the VEX ecosystem and this feature is still in the early phase of development.

The work on the rust port was split between two members: one of whom ported the API of core and modified it to fit into the rust programming style and safety model and one who set up the compiler toolchain and low level system for interfacing with the vex C/C++ library. Though this was originally an organizational decision, we realized that much of core could be completely abstracted away from dealing with VEX specific components and could operate on hardware that fulfills specific interface requirements. For example, as long as we can send a voltage to a motor and read a position our drivetrain and flywheel subsystems would work no matter the actual hardware. Thanks to rust's powerful generic programming features, this flexibility can be used without sacrificing helpful compiler errors (a common C++ issue) and without sacrificing performance using runtime polymorphism.

N-Pod Odometry

Motivation

Although we have been working on the GPS odometry system, wheel odometry is still vital. It provides great small-scale, quickly updating positions as well as having near-perfect, continuous, local velocity which a GPS system can not achieve. We use odometry in two ways; either tank or differential odometry where there is one wheel on either side of the drivetrain alongside the drive wheels and 3-wheel odometry where we have 3 wheels at ninety degrees to each other. Tank odometry is limited as it can not track horizontal movement and we simply hope that we never move sideways, though it is easiest to implement in the robot so it is our most commonly used system. 3-wheel odometry solves the side-to-side problem but is much harder to implement in hardware owing to the extra wheel where other subsystems would need space.

In a plea for mercy from the hardware team, we agreed that we would take tracking wheels wherever and we could make do. Though we once again got stuck with a tank system, if our dream of more tracking wheels ever comes true we would need code to handle such a system. Also, since tank and 3 wheel odometry are special cases of an n-pod system, we could reduce code duplication.

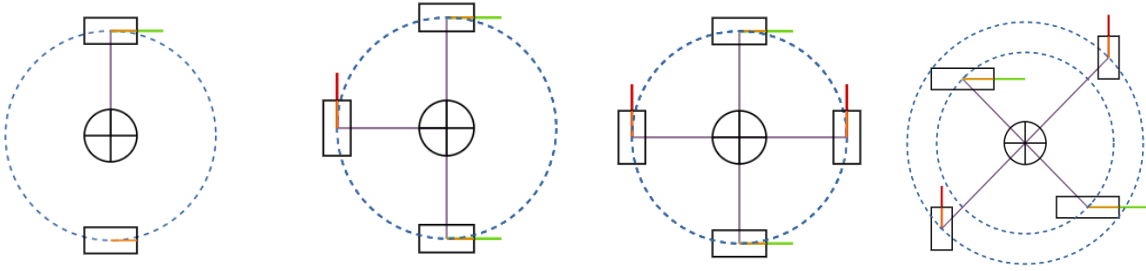


Figure 25: 2 pod, 3 pod, and arbitrary pods such a system could handle.

Syntax

After much brainstorming and many mad scientist whiteboard drawings, we believe that we have the fundamentals of a system figured out. Unfortunately, other responsibilities to the team came up so we do not yet have a functioning implementation of the system.

Imagine a robot with n number of tracking omni wheels. We could read encoder values $E_1, E_2, \dots, E_n$ from the system in radians from the initial position. As well, each encoder has a configuration $(x_1, y_1, \theta_1, r_1), \dots, (x_n, y_n, \theta_n, r_n)$ describing its position (x, y) relative to the center of rotation, an angle describing its orientation relative to the robot frame (θ) , and a radius of the wheel (r) .

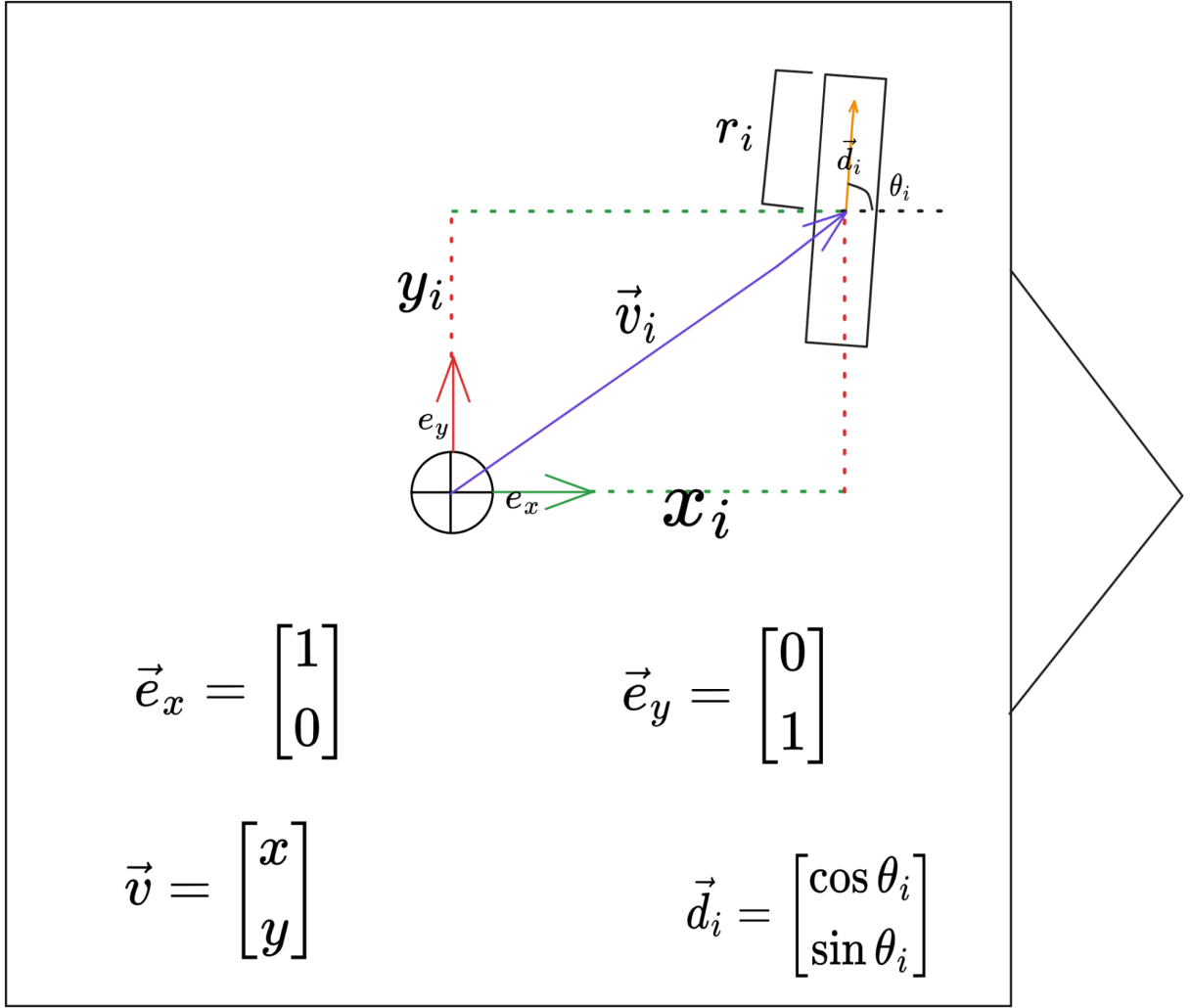


Figure 26: The configuration of a tracking wheel on the robot. (e_x, e_y) are the basis vectors of our coordinate system - the X and Y axes of the robot coordinate frame. d_i is the direction vector of the tracking wheel.

Now, if we pretend that these wheels are powered and we wish to translate and rotate the robot according to some controller input (x, y, θ) we can develop a formula for how much each wheel needs to rotate to move the robot in that direction with that rotation. Luckily, since the tracking wheels are omni wheels that roll freely in the axis against their “forward” direction, we do not need to worry about dragging a wheel so long as it is spinning the correct amount in its “forward” direction. For a desired (x, y, θ) (in the robots reference frame), for the i -th encoder, we say $E_i = xF_{xi}$. That is, for a movement in the x -axis the rotations of the i -th encoder, are the desired x movement times some scalar factor (F) for how far this specific wheel would rotate. Similarly, for a y only and θ only movement, $E_i = yF_{yi}$ and $E_i = \theta F_{\theta i}$ respectively.

Deriving Factors

$$F_x$$

F_x depends on the direction vector $\vec{d}$ of the omni-wheel. If the omni-wheel is facing along the x-axis, F_x will be higher whereas if the omni-wheel is directly perpendicular to the x-axis, it will not spin when you move only in the x-direction. Since $\vec{e}_x$ and $\vec{d}$ are unit vectors, how closely they are related is given by $\vec{e}_x \cdot \vec{d} = \cos(\text{angle between } x \text{ axis and wheel})$

F_x also depends on the radius of the wheel r . One full rotation of the wheel moves a distance of $C = 2\pi r$. If we drive in the direction of the wheel i inches, the wheel will complete $\frac{i}{2\pi r}$ revolutions. If we measure the rotations in radians, the wheel will travel $\frac{i}{r}$ radians. That is, if the encoder wheel travels E radians, we will have traveled Er inches in that direction.

So, the distance traveled in the x direction of a wheel pointing in the direction $\vec{d}$, rotating E radians is $x = Er(\vec{e}_x \cdot \vec{d})$. This gives since F_x as how many inches per radian turned,

$$F_x = \frac{x}{E} = r(\vec{e}_x \cdot \vec{d})$$

$$F_y$$

F_y is derived almost identically as F_x just instead of testing against $\vec{e}_x$ we test against $\vec{e}_y$. So,

$$F_y = \frac{y}{E} = r(\vec{e}_y \cdot \vec{d})$$

$$F_\theta$$

F_θ is a little more complicated since it is determined by the position of the wheel $\vec{v}$ as well as the orientation of the wheel $\vec{v}$

Imagine that the robot turns an angle of θ_r measured in radians. A wheel that is perfectly perpendicular to the rotation will travel an arc with distance $S = ||\vec{v}|| \theta_r$ by the arc length formula where the 'radius' of the arc is defined by the length of the vector $\vec{v}$.

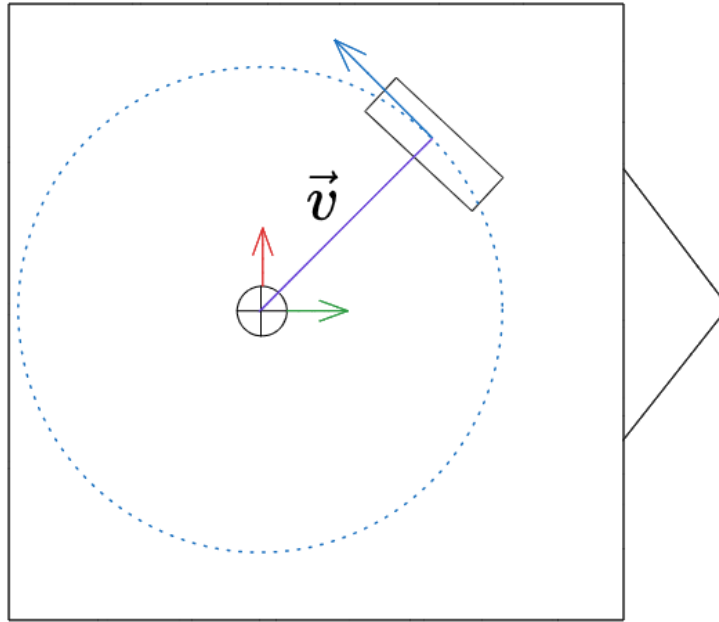
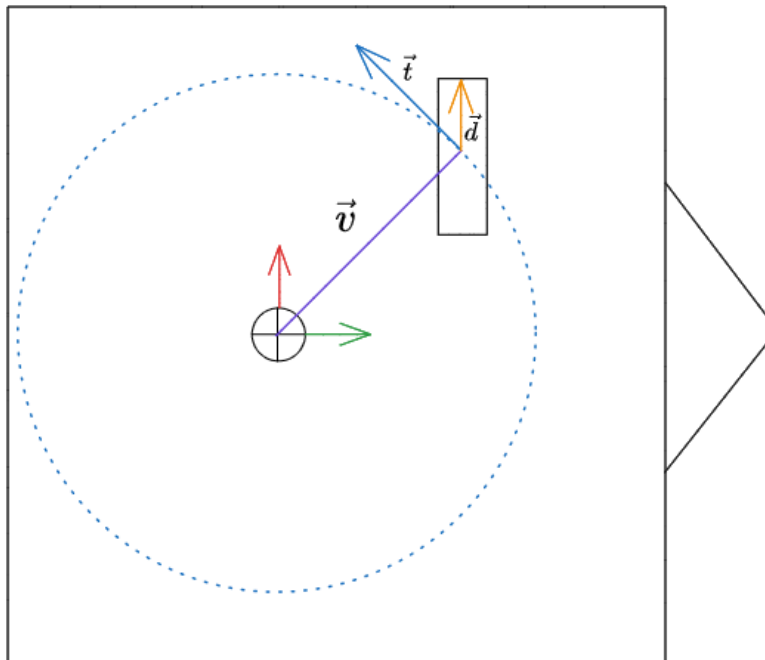


Figure 27: A conceptual perfectly perpendicular wheel

So, if we have a wheel that is always tangent to the rotation, it will travel

$$E_t r = S = ||\vec{v}||\theta_r$$

Since our wheel isn't guaranteed to be perfectly tangent to the arc, we have to use our dot product trick to get the component of its motion that is tangent to the turning circle. That is, instead of comparing to $\vec{e}_x$ or $\vec{e}_y$ we compare to the normalized vector $\vec{t}$ tangent to the turning circle.



$$E_t r = Er(\vec{t} \cdot \vec{d})$$

So

$$Er(\vec{t} \cdot \vec{d}) = S = ||\vec{v}||\theta_t$$

Since $\vec{t}$ is just a unit vector 90 degrees counterclockwise of $\vec{v}$, We can find it by multiplying $\vec{v}$ by the rotation matrix for 90 degrees and normalizing giving

$$\vec{t} = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} norm(\vec{v}) = \begin{bmatrix} -norm(\vec{v}).y \\ norm(\vec{v}).x \end{bmatrix}$$

So

$$F_\theta = \frac{\theta_r}{E} = \frac{r(\vec{t} \cdot \vec{d})}{||\vec{v}||} = \frac{r(\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} norm(\vec{v}) \cdot \vec{d})}{||\vec{v}||} = \frac{r(\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \vec{v} \cdot \vec{d})}{||\vec{v}||^2}$$

Why factors

These factors make solving this problem much simpler. For the forward case, for some wheel i , its rotation is the sum of all the motions applied to it. So $E_i = F_{xi}x + F_{yi}y + F_{\theta i}\theta$. So, for all the wheels, we plug in the commanded (x, y, θ) to each wheel's factors to get its necessary rotation. Since the factors depend only on the wheel's pose in the frame, these can be calculated once at the start of the program and are constant (unless the frame breaks apart, in which case the robot has other problems).

Now Do It Backwards

We have now solved the forward system for when we have a delta of our pose and want our wheel deltas. Now we must take our wheel deltas and solve for our pose delta. We have our formulas for each wheel's encoder motion and can consider this as a system of linear equations. At runtime, we have our wheel encoder deltas we can plug in and then we can solve the system of linear equations. This requires that we have enough data to satisfy the equations. That is, we need at least 3 separate wheels with at least some angle between them, or else the system will be not fully constrained. In the case of tank odometry, we only have two wheels but as outside observers we know we can not measure change in one dimension. So, we know one variable is zero and then have two remaining free variables and two equations to satisfy the system. For robots with greater than three encoders, we have an over-constrained system of equations but this is not an issue. Since all the encoders are modeled on a physical system, they should agree on what the solution is. Using the technique of least squares regression, we can find our (x, y, θ) to solve the over-constrained system that minimizes the error between equations. This

also gives us a way to detect errors in our drive train. If a wheel gets jammed, its encoder reading will disagree with the rest of the system, and the error value will measurably increase. If we monitor this error value we can diagnose mechanical or electrical issues from the code.

$$T = \begin{bmatrix} F_{x1} & F_{y1} & F_{\theta1} \\ \vdots & \vdots & \vdots \\ F_{xn} & F_{yn} & F_{\theta n} \end{bmatrix}$$

$$\begin{bmatrix} \frac{\text{Length}}{\text{Angle}} & \frac{\text{Length}}{\text{Angle}} & \frac{\text{Angle}}{\text{Angle}} \\ \vdots & \vdots & \vdots \end{bmatrix}$$

transfer matrix
from robot velocity
to encoder velocities
(f for factor)

$$\vec{X} = \begin{bmatrix} \frac{dx_{robot}}{dt} \\ \frac{dy_{robot}}{dt} \\ \frac{d\theta_{robot}}{dt} \end{bmatrix}$$

$$\begin{bmatrix} \frac{\text{Distance}}{\text{Time}} \\ \frac{\text{Distance}}{\text{Time}} \\ \frac{\text{Angle}}{\text{Time}} \end{bmatrix}$$

pose velocity

$$\vec{E} = \begin{bmatrix} E_1 \\ \vdots \\ E_n \end{bmatrix}$$

$$\begin{bmatrix} \frac{\text{Angle}}{\text{Time}} \\ \vdots \\ \frac{\text{Angle}}{\text{Time}} \end{bmatrix}$$

encoder wheel
velocities

$$T\vec{X} = \vec{E}$$

$$\vec{X} = T^{-1}\vec{E}$$

or in the case where the matrix is not invertible, find the best solution

The linear algebra behind the solution

V5 Debug Board

The large number of features added to core, while extremely useful, are also very difficult to debug. Without a proper real-time c++ debugger and one stream serial data for print statements, data parsing can get very messy. The improvements to the Screen subsystem have helped, but a remote solution is needed to avoid chasing after the robot to get visual data.

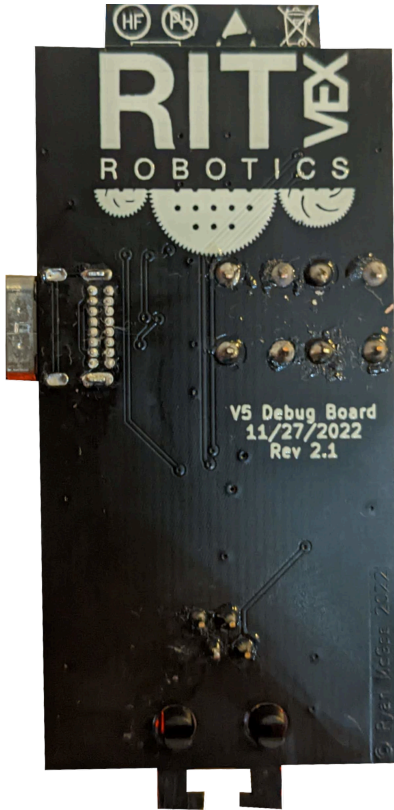


Figure 28 - Debug Board (Back)

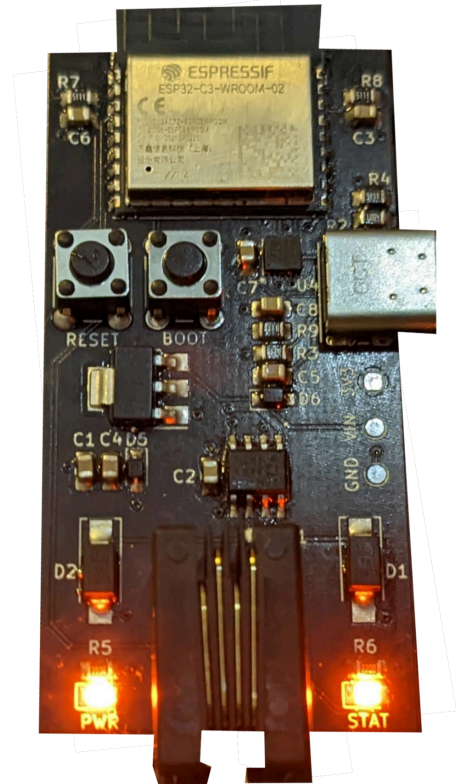


Figure 29 - Debug Board (Front)

The V5 Debug Board is a custom PCB designed by our team specifically to interface with the V5 Smart Ports, host a ROS2 node and a WiFi access point for programmers to connect to with laptops. This board is designed to ingest any kind of data and use it for graphs, real-time tuning and even displaying a 3D model of the robot on a virtual field using odometry data. This data can be viewed using either ROS' RViz or Foxglove visualization software.

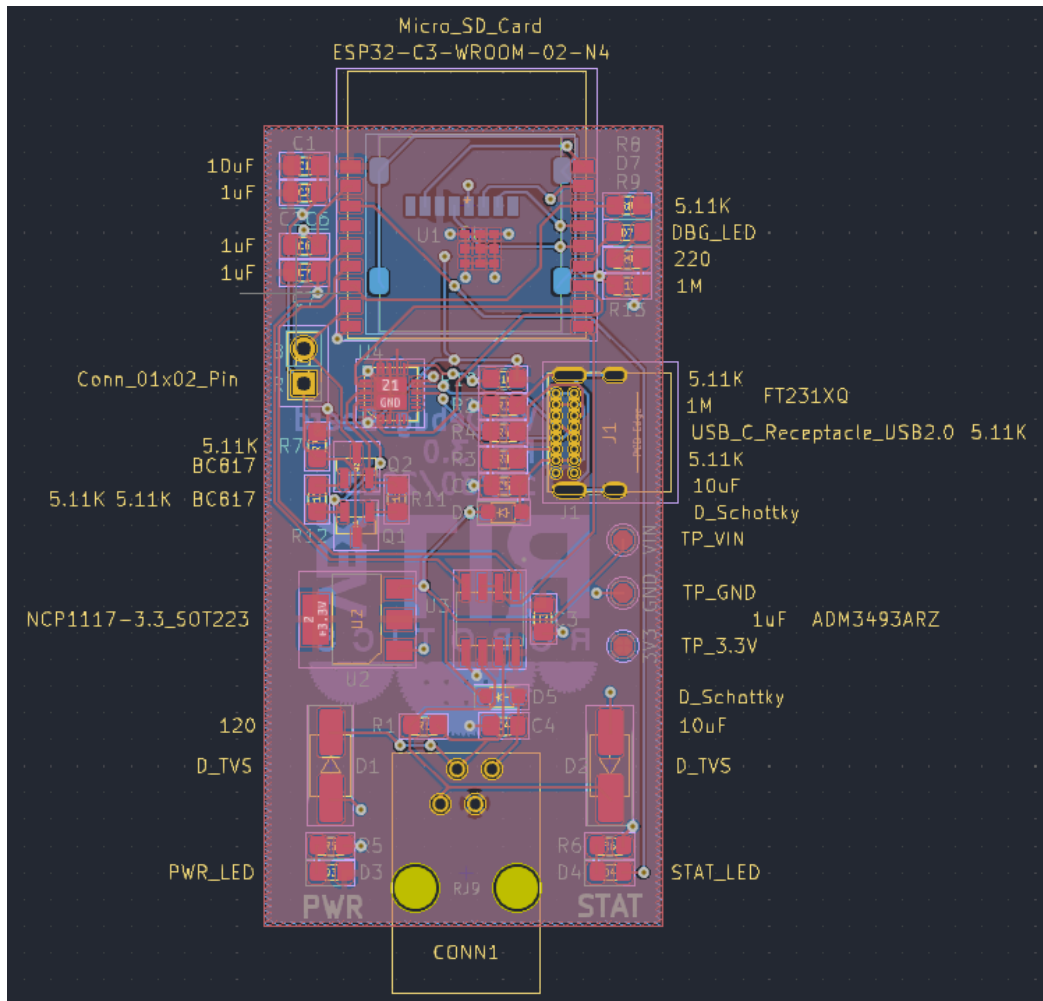


Figure 30 - Debug Board PCB Layout

Hardware design is nearing completion, with 3 revisions built and tested. Revision 3.0 is powered by an ESP32-C3-WROOM2 microcontroller, and uses an RS-485 transceiver to communicate with the Brain over a smart-port. The new addition of a Micro-SD card allows users to upload their own 3D model of the robot, and provides data logging capabilities.

As of now, the hardware design is nearly complete. Software has achieved WiFi AP broadcasting, TCP communications and work is starting on the Micro-ROS implementation. The design is fully open source under the GPL-3 license, and is hosted at github.com/superrm11/VexDebugBoard and github.com/superrm11/VexDebugBoard_PCB.

RIT VEXU Core API

Generated by Doxygen 1.17.0

1 Core	2
1.1 Getting Started	2
1.2 Features	2
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	7
3.1 Class List	7
4 Namespace Documentation	11
4.1 VDB Namespace Reference	11
4.1.1 Detailed Description	11
4.1.2 Function Documentation	11
5 Class Documentation	11
5.1 VDP::AbstractDevice Class Reference	11
5.1.1 Detailed Description	12
5.1.2 Constructor & Destructor Documentation	12
5.1.3 Member Function Documentation	12
5.2 Async Class Reference	13
5.2.1 Detailed Description	13
5.3 AutoChooser Class Reference	13
5.3.1 Detailed Description	13
5.3.2 Constructor & Destructor Documentation	14
5.3.3 Member Function Documentation	14
5.3.4 Member Data Documentation	14
5.4 BasicSolenoidSet Class Reference	15
5.4.1 Detailed Description	15
5.4.2 Constructor & Destructor Documentation	15
5.4.3 Member Function Documentation	15
5.5 BasicSpinCommand Class Reference	16
5.5.1 Detailed Description	16
5.5.2 Constructor & Destructor Documentation	16
5.5.3 Member Function Documentation	17
5.6 BasicStopCommand Class Reference	17
5.6.1 Detailed Description	17
5.6.2 Constructor & Destructor Documentation	17
5.6.3 Member Function Documentation	18
5.7 Branch Class Reference	18
5.7.1 Detailed Description	18
5.8 screen::ButtonWidget Class Reference	19
5.8.1 Detailed Description	19
5.8.2 Constructor & Destructor Documentation	19

5.8.3 Member Function Documentation	20
5.9 CommandController Class Reference	20
5.9.1 Detailed Description	21
5.9.2 Constructor & Destructor Documentation	21
5.9.3 Member Function Documentation	21
5.10 Condition Class Reference	23
5.10.1 Detailed Description	23
5.11 CRC32 Class Reference	24
5.11.1 Detailed Description	24
5.11.2 Member Function Documentation	24
5.12 CustomEncoder Class Reference	26
5.12.1 Detailed Description	26
5.12.2 Constructor & Destructor Documentation	26
5.12.3 Member Function Documentation	26
5.13 DelayCommand Class Reference	28
5.13.1 Detailed Description	28
5.13.2 Constructor & Destructor Documentation	28
5.13.3 Member Function Documentation	29
5.14 DriveForwardCommand Class Reference	29
5.14.1 Detailed Description	29
5.14.2 Constructor & Destructor Documentation	30
5.14.3 Member Function Documentation	30
5.15 DriveStopCommand Class Reference	31
5.15.1 Detailed Description	31
5.15.2 Constructor & Destructor Documentation	31
5.15.3 Member Function Documentation	32
5.16 DriveToPointCommand Class Reference	32
5.16.1 Detailed Description	32
5.16.2 Constructor & Destructor Documentation	32
5.16.3 Member Function Documentation	33
5.17 AutoChooser::entry_t Struct Reference	34
5.17.1 Detailed Description	34
5.17.2 Member Data Documentation	34
5.18 ExponentialMovingAverage Class Reference	34
5.18.1 Detailed Description	35
5.18.2 Constructor & Destructor Documentation	35
5.18.3 Member Function Documentation	35
5.19 Feedback Class Reference	36
5.19.1 Detailed Description	37
5.19.2 Member Function Documentation	37
5.20 FeedForward Class Reference	39
5.20.1 Detailed Description	39

5.20.2 Constructor & Destructor Documentation	39
5.20.3 Member Function Documentation	40
5.21 FeedForward::ff_config_t Struct Reference	40
5.21.1 Detailed Description	40
5.21.2 Member Data Documentation	41
5.22 Filter Class Reference	41
5.22.1 Detailed Description	41
5.23 Flywheel Class Reference	42
5.23.1 Detailed Description	42
5.23.2 Constructor & Destructor Documentation	42
5.23.3 Member Function Documentation	43
5.23.4 Friends And Related Symbol Documentation	45
5.24 FlywheelStopCommand Class Reference	45
5.24.1 Detailed Description	46
5.24.2 Constructor & Destructor Documentation	46
5.24.3 Member Function Documentation	46
5.25 FlywheelStopMotorsCommand Class Reference	46
5.25.1 Detailed Description	47
5.25.2 Constructor & Destructor Documentation	47
5.25.3 Member Function Documentation	47
5.26 FlywheelStopNonTasksCommand Class Reference	47
5.26.1 Detailed Description	47
5.27 FunctionCommand Class Reference	48
5.27.1 Detailed Description	48
5.28 FunctionCondition Class Reference	48
5.28.1 Detailed Description	48
5.29 screen::FunctionPage Class Reference	48
5.29.1 Detailed Description	49
5.29.2 Constructor & Destructor Documentation	49
5.29.3 Member Function Documentation	49
5.30 GenericAuto Class Reference	50
5.30.1 Detailed Description	50
5.30.2 Member Function Documentation	50
5.31 PurePursuit::hermite_point Struct Reference	52
5.31.1 Detailed Description	52
5.32 IfTimePassed Class Reference	52
5.32.1 Detailed Description	52
5.33 InOrder Class Reference	52
5.33.1 Detailed Description	53
5.34 InterpolatingMap< KEY, VALUE > Class Template Reference	53
5.34.1 Detailed Description	53
5.34.2 Member Function Documentation	53

5.35 KalmanFilter< STATES, INPUTS, OUTPUTS > Class Template Reference	54
5.35.1 Detailed Description	55
5.35.2 Constructor & Destructor Documentation	55
5.35.3 Member Function Documentation	56
5.36 Lift< T > Class Template Reference	59
5.36.1 Detailed Description	59
5.36.2 Constructor & Destructor Documentation	59
5.36.3 Member Function Documentation	60
5.37 Lift< T >::lift_cfg_t Struct Reference	63
5.37.1 Detailed Description	63
5.38 LinearPlantInversionFeedforward< STATES, INPUTS > Class Template Reference	63
5.38.1 Detailed Description	64
5.38.2 Constructor & Destructor Documentation	64
5.38.3 Member Function Documentation	65
5.39 LinearQuadraticRegulator< STATES, INPUTS > Class Template Reference	67
5.39.1 Detailed Description	67
5.39.2 Constructor & Destructor Documentation	68
5.39.3 Member Function Documentation	69
5.40 LinearSystem< STATES, INPUTS, OUTPUTS > Class Template Reference	70
5.40.1 Detailed Description	70
5.40.2 Constructor & Destructor Documentation	70
5.40.3 Member Function Documentation	71
5.41 Logger Class Reference	72
5.41.1 Detailed Description	73
5.41.2 Constructor & Destructor Documentation	73
5.41.3 Member Function Documentation	73
5.42 MotionController::m_profile_cfg_t Struct Reference	75
5.42.1 Detailed Description	75
5.43 MaybeMessage Class Reference	76
5.43.1 Detailed Description	76
5.43.2 Constructor & Destructor Documentation	76
5.43.3 Member Function Documentation	77
5.44 MecanumDrive Class Reference	77
5.44.1 Detailed Description	78
5.44.2 Constructor & Destructor Documentation	78
5.44.3 Member Function Documentation	78
5.45 MecanumDrive::mecanumdrive_config_t Struct Reference	80
5.45.1 Detailed Description	81
5.46 MotionController Class Reference	81
5.46.1 Detailed Description	82
5.46.2 Constructor & Destructor Documentation	82
5.46.3 Member Function Documentation	82

5.47 VDP::MotorDataRecord Class Reference	85
5.47.1 Detailed Description	86
5.47.2 Constructor & Destructor Documentation	86
5.47.3 Member Function Documentation	86
5.48 MovingAverage Class Reference	87
5.48.1 Detailed Description	87
5.48.2 Constructor & Destructor Documentation	87
5.48.3 Member Function Documentation	88
5.49 VDP::Number< NumT, schemaType > Class Template Reference	89
5.49.1 Detailed Description	89
5.49.2 Member Typedef Documentation	90
5.49.3 Constructor & Destructor Documentation	90
5.49.4 Member Function Documentation	90
5.50 Odometry3Wheel Class Reference	92
5.50.1 Detailed Description	94
5.50.2 Constructor & Destructor Documentation	94
5.50.3 Member Function Documentation	94
5.51 Odometry3Wheel::odometry3wheel_cfg_t Struct Reference	95
5.51.1 Detailed Description	95
5.51.2 Member Data Documentation	95
5.52 OdometryBase Class Reference	96
5.52.1 Detailed Description	97
5.52.2 Constructor & Destructor Documentation	97
5.52.3 Member Function Documentation	97
5.52.4 Member Data Documentation	100
5.53 VDP::OdometryControlRecord Class Reference	101
5.53.1 Detailed Description	102
5.53.2 Constructor & Destructor Documentation	102
5.53.3 Member Function Documentation	103
5.54 VDP::OdometryDataRecord Class Reference	103
5.54.1 Detailed Description	104
5.54.2 Constructor & Destructor Documentation	104
5.54.3 Member Function Documentation	104
5.55 screen::OdometryPage Class Reference	105
5.55.1 Detailed Description	105
5.55.2 Constructor & Destructor Documentation	105
5.55.3 Member Function Documentation	106
5.56 OdometrySerial Class Reference	106
5.56.1 Detailed Description	107
5.56.2 Constructor & Destructor Documentation	108
5.56.3 Member Function Documentation	108
5.57 OdometryTank Class Reference	111

5.57.1 Detailed Description	112
5.57.2 Constructor & Destructor Documentation	112
5.57.3 Member Function Documentation	113
5.58 OdomSetPosition Class Reference	114
5.58.1 Detailed Description	114
5.58.2 Constructor & Destructor Documentation	114
5.58.3 Member Function Documentation	115
5.59 VDP::PacketHeader Struct Reference	115
5.59.1 Detailed Description	115
5.60 VDP::PacketWriter Class Reference	115
5.60.1 Detailed Description	116
5.60.2 Constructor & Destructor Documentation	116
5.60.3 Member Function Documentation	116
5.61 screen::Page Class Reference	119
5.61.1 Detailed Description	119
5.61.2 Member Function Documentation	119
5.62 Parallel Class Reference	120
5.62.1 Detailed Description	120
5.63 VDP::Part Class Reference	121
5.63.1 Detailed Description	121
5.63.2 Constructor & Destructor Documentation	122
5.63.3 Member Function Documentation	122
5.64 PurePursuit::Path Class Reference	124
5.64.1 Detailed Description	124
5.64.2 Constructor & Destructor Documentation	124
5.64.3 Member Function Documentation	125
5.65 PID Class Reference	125
5.65.1 Detailed Description	126
5.65.2 Member Enumeration Documentation	126
5.65.3 Constructor & Destructor Documentation	126
5.65.4 Member Function Documentation	127
5.65.5 Member Data Documentation	130
5.66 PID::pid_config_t Struct Reference	130
5.66.1 Detailed Description	131
5.66.2 Member Data Documentation	131
5.67 VDP::PIDControlRecord Class Reference	131
5.67.1 Detailed Description	132
5.67.2 Constructor & Destructor Documentation	132
5.67.3 Member Function Documentation	133
5.68 VDP::PIDDataRecord Class Reference	133
5.68.1 Detailed Description	134
5.68.2 Constructor & Destructor Documentation	134

5.68.3 Member Function Documentation	134
5.69 screen::PIDPage Class Reference	135
5.69.1 Detailed Description	135
5.69.2 Constructor & Destructor Documentation	135
5.69.3 Member Function Documentation	136
5.70 Pose2d Class Reference	136
5.70.1 Detailed Description	137
5.70.2 Constructor & Destructor Documentation	137
5.70.3 Member Function Documentation	138
5.70.4 Friends And Related Symbol Documentation	142
5.71 PurePursuitCommand Class Reference	143
5.71.1 Detailed Description	143
5.71.2 Constructor & Destructor Documentation	143
5.71.3 Member Function Documentation	143
5.72 VDP::Record Class Reference	144
5.72.1 Detailed Description	144
5.72.2 Constructor & Destructor Documentation	144
5.72.3 Member Function Documentation	146
5.73 Rect Struct Reference	147
5.73.1 Detailed Description	147
5.74 VDP::RegistryListener< MutexType > Class Template Reference	147
5.74.1 Detailed Description	148
5.74.2 Constructor & Destructor Documentation	148
5.74.3 Member Function Documentation	148
5.75 ResponsePacketVisitor Class Reference	150
5.75.1 Detailed Description	150
5.75.2 Constructor & Destructor Documentation	150
5.75.3 Member Function Documentation	151
5.76 robot_specs_t Struct Reference	153
5.76.1 Detailed Description	154
5.76.2 Member Data Documentation	154
5.77 Rotation2d Class Reference	154
5.77.1 Detailed Description	155
5.77.2 Constructor & Destructor Documentation	155
5.77.3 Member Function Documentation	156
5.77.4 Friends And Related Symbol Documentation	160
5.78 ScaledSphericalSimplexSigmaPoints< STATES > Class Template Reference	161
5.78.1 Detailed Description	161
5.78.2 Constructor & Destructor Documentation	161
5.78.3 Member Function Documentation	162
5.79 screen::ScreenData Struct Reference	163
5.79.1 Detailed Description	163

5.80 Serializer Class Reference	163
5.80.1 Detailed Description	164
5.80.2 Constructor & Destructor Documentation	164
5.80.3 Member Function Documentation	164
5.81 screen::SliderWidget Class Reference	167
5.81.1 Detailed Description	168
5.81.2 Constructor & Destructor Documentation	168
5.81.3 Member Function Documentation	168
5.82 SpinRPMCommand Class Reference	169
5.82.1 Detailed Description	169
5.82.2 Constructor & Destructor Documentation	169
5.82.3 Member Function Documentation	170
5.83 PurePursuit::spline Struct Reference	170
5.83.1 Detailed Description	170
5.84 State Struct Reference	170
5.84.1 Detailed Description	170
5.85 screen::StatsPage Class Reference	171
5.85.1 Detailed Description	171
5.85.2 Constructor & Destructor Documentation	171
5.85.3 Member Function Documentation	171
5.86 VDP::String Class Reference	172
5.86.1 Detailed Description	173
5.86.2 Constructor & Destructor Documentation	173
5.86.3 Member Function Documentation	173
5.87 TakeBackHalf Class Reference	175
5.87.1 Detailed Description	176
5.87.2 Member Function Documentation	176
5.88 TankDrive Class Reference	178
5.88.1 Detailed Description	178
5.88.2 Member Enumeration Documentation	178
5.88.3 Constructor & Destructor Documentation	179
5.88.4 Member Function Documentation	179
5.89 VDP::TestRecord Class Reference	188
5.89.1 Detailed Description	189
5.89.2 Constructor & Destructor Documentation	189
5.90 VDP::TimestampedRecord Class Reference	189
5.90.1 Detailed Description	190
5.90.2 Constructor & Destructor Documentation	190
5.90.3 Member Function Documentation	190
5.91 tracking_wheel_cfg_t Struct Reference	191
5.91.1 Detailed Description	191
5.91.2 Member Data Documentation	191

5.92 Transform2d Class Reference	192
5.92.1 Detailed Description	193
5.92.2 Constructor & Destructor Documentation	193
5.92.3 Member Function Documentation	195
5.92.4 Friends And Related Symbol Documentation	197
5.93 Translation2d Class Reference	197
5.93.1 Detailed Description	198
5.93.2 Constructor & Destructor Documentation	198
5.93.3 Member Function Documentation	199
5.93.4 Friends And Related Symbol Documentation	204
5.94 TrapezoidProfile Class Reference	204
5.94.1 Detailed Description	204
5.94.2 Constructor & Destructor Documentation	204
5.94.3 Member Function Documentation	205
5.95 TurnDegreesCommand Class Reference	206
5.95.1 Detailed Description	206
5.95.2 Constructor & Destructor Documentation	206
5.95.3 Member Function Documentation	206
5.96 TurnToHeadingCommand Class Reference	207
5.96.1 Detailed Description	207
5.96.2 Constructor & Destructor Documentation	207
5.96.3 Member Function Documentation	208
5.97 Twist2d Class Reference	208
5.97.1 Detailed Description	209
5.97.2 Constructor & Destructor Documentation	209
5.97.3 Member Function Documentation	210
5.97.4 Friends And Related Symbol Documentation	211
5.98 UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS > Class Template Reference	211
5.98.1 Detailed Description	212
5.98.2 Constructor & Destructor Documentation	213
5.98.3 Member Function Documentation	214
5.99 VDP::UpcastNumbersVisitor Class Reference	219
5.99.1 Detailed Description	219
5.100 VDP::Visitor Class Reference	219
5.100.1 Detailed Description	219
5.101 WaitUntilCondition Class Reference	220
5.101.1 Detailed Description	220
5.102 WaitUntilUpToSpeedCommand Class Reference	220
5.102.1 Detailed Description	220
5.102.2 Constructor & Destructor Documentation	220
5.102.3 Member Function Documentation	220

1 Core

This is the host repository for the custom VEX libraries used by the RIT VEXU team

Automatically updated documentation is available at [here](#). There is also a downloadable [reference manual](#).

1.1 Getting Started

If you just want to start a project with Core, make a fork of the [Fork Template](#) and follow it's instructions.

To setup core for an existing project:

1. Create a new vex project (using the VSCode extension or other methods)
2. Initialize a git repository for the project
3. Execute `git subtree add --prefix=core https://github.com/RIT-VEX-U/Core.↵
git main`
4. Update the vex Makefile (or any other build system) to know about the core files (`core/src` for source files, `core/include` for headers) (See [here](#) for an example)
5. Enable [Eigen](#) (Latest supported version is 3.4.0):
 - `mkdir vendor`
 - `git submodule add https://gitlab.com/libeigen/eigen.git vendor/eigen`
 - `cd vendor/eigen`
 - `git checkout 3.4.0`
 - Add the following to the makefile to give Core access to the library: `INC += -Ivendor/eigen` (See [here](#) for an example)

If you only wish to use a single version of Core, you can simply clone `core/` into your project and add the core source and header files to your makefile.

1.2 Features

Here is the current feature list this repo provides:

Subsystems (See [Wiki/Subsystems](#)):

- Tank drivetrain (user control / autonomous)
- Mecanum drivetrain (user control / autonomous)
- Odometry
 - Tank (Differential)
 - [N-Pod](#)

- [Flywheel](#)
- [Lift](#)
- Custom encoders

Utilities (See [Wiki/Utilites](#)):

- [PID](#) controller
- [FeedForward](#) controller
- Trapezoidal motion profile controller
- Pure Pursuit
- Generic auto program builder
- Auto program UI selector
- Mathematical classes (Vector2D, Moving Average)

2 Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

VDP::AbstractDevice	11
Async	13
BasicSolenoidSet	15
BasicSpinCommand	16
BasicStopCommand	17
Branch	18
screen::ButtonWidget	19
CommandController	20
Condition	23
FunctionCondition	48
IfTimePassed	52
CRC32	24
CustomEncoder	26
DelayCommand	28
DriveForwardCommand	29

DriveStopCommand	31
DriveToPointCommand	32
AutoChooser::entry_t	34
Feedback	36
MotionController	81
PID	125
TakeBackHalf	175
FeedForward	39
FeedForward::ff_config_t	40
Filter	41
ExponentialMovingAverage	34
MovingAverage	87
Flywheel	42
FlywheelStopCommand	45
FlywheelStopMotorsCommand	46
FlywheelStopNonTasksCommand	47
FunctionCommand	48
GenericAuto	50
PurePursuit::hermite_point	52
InOrder	52
InterpolatingMap< KEY, VALUE >	53
KalmanFilter< STATES, INPUTS, OUTPUTS >	54
Lift< T >	59
Lift< T >lift_cfg_t	63
LinearPlantInversionFeedforward< STATES, INPUTS >	63
LinearQuadraticRegulator< STATES, INPUTS >	67
LinearSystem< STATES, INPUTS, OUTPUTS >	70
Logger	72
MotionController::m_profile_cfg_t	75
MaybeMessage	76
MecanumDrive	77
MecanumDrive::mecanumdrive_config_t	80

Odometry3Wheel::odometry3wheel_cfg_t	95
OdometryBase	96
Odometry3Wheel	92
OdometrySerial	106
OdometryTank	111
OdomSetPosition	114
VDP::PacketHeader	115
VDP::PacketWriter	115
screen::Page	119
AutoChooser	13
screen::FunctionPage	48
screen::OdometryPage	105
screen::PIDPage	135
screen::StatsPage	171
Parallel	120
VDP::Part	121
VDP::Number< float, Type::Float >	89
VDP::Number< double, Type::Double >	89
VDP::Number< uint8_t, Type::UInt8 >	89
VDP::Number< uint16_t, Type::UInt16 >	89
VDP::Number< uint32_t, Type::UInt32 >	89
VDP::Number< uint64_t, Type::UInt64 >	89
VDP::Number< int8_t, Type::Int8 >	89
VDP::Number< int16_t, Type::Int16 >	89
VDP::Number< int32_t, Type::Int32 >	89
VDP::Number< int64_t, Type::Int64 >	89
VDP::Number< NumT, schemaType >	89
VDP::Record	144
VDP::MotorDataRecord	85
VDP::OdometryControlRecord	101
VDP::OdometryDataRecord	103
VDP::PIDControlRecord	131

VDP::PIDDataRecord	133
VDP::TestRecord	188
VDP::TimestampedRecord	189
VDP::String	172
PurePursuit::Path	124
PID::pid_config_t	130
Pose2d	136
PurePursuitCommand	143
Rect	147
VDP::RegistryListener< MutexType >	147
robot_specs_t	153
Rotation2d	154
ScaledSphericalSimplexSigmaPoints< STATES >	161
screen::ScreenData	163
Serializer	163
screen::SliderWidget	167
SpinRPMCommand	169
PurePursuit::spline	170
State	170
TankDrive	178
tracking_wheel_cfg_t	191
Transform2d	192
Translation2d	197
TrapezoidProfile	204
TurnDegreesCommand	206
TurnToHeadingCommand	207
Twist2d	208
UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >	211
VDP::Visitor	219
ResponsePacketVisitor	150
VDP::UpcastNumbersVisitor	219
WaitUntilCondition	220

WaitUntilUpToSpeedCommand	220
---------------------------	-----

3 Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

VDP::AbstractDevice	11
Async	
Async runs a command asynchronously will simply let it go and never look back THIS HAS A VERY NICHE USE CASE. THINK ABOUT IF YOU REALLY NEED IT	13
AutoChooser	13
BasicSolenoidSet	15
BasicSpinCommand	16
BasicStopCommand	17
Branch	
Branch chooses from multiple options at runtime. the function decider returns an index into the choices vector If you wish to make no choice and skip this section, return NO_CHOICE; any choice that is out of bounds set to NO_CHOICE	18
screen::ButtonWidget	
Widget that does something when you tap it. The function is only called once when you first tap it	19
CommandController	20
Condition	23
CRC32	
A class for calculating the CRC32 checksum from arbitrary data	24
CustomEncoder	26
DelayCommand	28
DriveForwardCommand	29
DriveStopCommand	31
DriveToPointCommand	32
AutoChooser::entry_t	34
ExponentialMovingAverage	34
Feedback	36
FeedForward	39
FeedForward::ff_config_t	40
Filter	41

Flywheel	42
FlywheelStopCommand	45
FlywheelStopMotorsCommand	46
FlywheelStopNonTasksCommand	47
FunctionCommand	48
FunctionCondition	
FunctionCondition is a quick and dirty Condition to wrap some expression that should be evaluated at runtime	48
screen::FunctionPage	
Simple page that stores no internal data. the draw and update functions use only global data rather than storing anything	48
GenericAuto	50
PurePursuit::hermite_point	52
IfTimePassed	
IfTimePassed tests based on time since the command controller was constructed. Returns true if elapsed time > time_s	52
InOrder	
InOrder runs its commands sequentially then continues. How to handle timeout in this case. Automatically set it to sum of commands timeouts?	52
InterpolatingMap< KEY, VALUE >	53
KalmanFilter< STATES, INPUTS, OUTPUTS >	54
Lift< T >	59
Lift< T >::lift_cfg_t	63
LinearPlantInversionFeedforward< STATES, INPUTS >	63
LinearQuadraticRegulator< STATES, INPUTS >	67
LinearSystem< STATES, INPUTS, OUTPUTS >	70
Logger	
Class to simplify writing to files	72
MotionController::m_profile_cfg_t	75
MaybeMessage	
State Machine :)))))) A fun fun way of controlling stateful subsystems - used in the 2023-2024 Over Under game for our overly complex intake-cata subsystem (see there for an example) The statemachine runs in a background thread and a user thread can interact with it through current_state and send_message	76
MecanumDrive	77
MecanumDrive::mecanumdrive_config_t	80
MotionController	81
VDP::MotorDataRecord	85

MovingAverage	87
VDP::Number< NumT, schemaType >	89
Odometry3Wheel	92
Odometry3Wheel::odometry3wheel_cfg_t	95
OdometryBase	96
VDP::OdometryControlRecord	101
VDP::OdometryDataRecord	103
screen::OdometryPage	
Page that shows odometry position and rotation and a map (if an sd card with the file is on)	105
OdometrySerial	106
OdometryTank	111
OdomSetPosition	114
VDP::PacketHeader	115
VDP::PacketWriter	115
screen::Page	
Page describes one part of the screen slideshow	119
Parallel	
Parallel runs multiple commands in parallel and waits for all to finish before continuing. if none finish before this command's timeout, it will call on_timeout on all children continue	120
VDP::Part	121
PurePursuit::Path	124
PID	125
PID::pid_config_t	130
VDP::PIDControlRecord	131
VDP::PIDDataRecord	133
screen::PIDPage	
PIDPage provides a way to tune a pid controller on the screen	135
Pose2d	136
PurePursuitCommand	143
VDP::Record	144
Rect	147
VDP::RegistryListener< MutexType >	147
ResponsePacketVisitor	150
robot_specs_t	153

Rotation2d	154
ScaledSphericalSimplexSigmaPoints< STATES >	161
screen::ScreenData	
Holds the data that will be passed to the screen thread you probably shouldnt have to use it	163
Serializer	
Serializes Arbitrary data to a file on the SD Card	163
screen::SliderWidget	
Widget that updates a double value. Updates by reference so watch out for race conditions cuz the screen stuff lives on another thread	167
SpinRPMCommand	169
PurePursuit::spline	170
State	170
screen::StatsPage	
Draws motor stats and battery stats to the screen	171
VDP::String	172
TakeBackHalf	
A velocity controller	175
TankDrive	178
VDP::TestRecord	188
VDP::TimestampedRecord	189
tracking_wheel_cfg_t	191
Transform2d	192
Translation2d	197
TrapezoidProfile	204
TurnDegreesCommand	206
TurnToHeadingCommand	207
Twist2d	208
UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >	211
VDP::UpcastNumbersVisitor	219
VDP::Visitor	219
WaitUntilCondition	
Waits until the condition is true	220
WaitUntilUpToSpeedCommand	220

4 Namespace Documentation

4.1 VDB Namespace Reference

Functions

- uint32_t [time_ms](#) ()
- void [delay_ms](#) (uint32_t ms)

4.1.1 Detailed Description

Defines a COBS Serial Device to transmit [VDB](#) data through

4.1.2 Function Documentation

delay_ms()

```
void VDB::delay_ms (
    uint32_t ms)
```

delay for ms time

Parameters

<i>ms</i>	the ms to delay for
-----------	---------------------

time_ms()

```
uint32_t VDB::time_ms ()
```

Returns

the time in ms of the bot since startup

5 Class Documentation

5.1 VDP::AbstractDevice Class Reference

```
#include <protocol.hpp>
```

Public Member Functions

- virtual bool [send_packet](#) (const VDP::Packet &packet)=0
- virtual void [register_receive_callback](#) (std::function< void(const VDP::Packet &packet)> callback)=0
- virtual [~AbstractDevice](#) ()

5.1.1 Detailed Description

defines a generic device to trasmit packets through

5.1.2 Constructor & Destructor Documentation

~AbstractDevice()

```
VDP::AbstractDevice::~~AbstractDevice () [virtual]
```

deleter for the device, used to delete it when it is no longer needed

5.1.3 Member Function Documentation

register_receive_callback()

```
virtual void VDP::AbstractDevice::register_receive_callback (
    std::function< void(const VDP::Packet &packet)> callback) [pure virtual]
```

a callback to function that runs when a new packet is available

Parameters

<i>the</i>	function for the callback to call me when my ex-wife
------------	--

send_packet()

```
virtual bool VDP::AbstractDevice::send_packet (
    const VDP::Packet & packet) [pure virtual]
```

Sends a packet over some transmission medium It is not specified how the packet reaches the partner The transmission medium and wire format are left to the user

Parameters

<i>packet</i>	the packet to send through the device
---------------	---------------------------------------

Returns

whether the packet was sent sucessfully or not

The documentation for this class was generated from the following files:

- protocol.hpp
- protocol.cpp

5.2 Async Class Reference

[Async](#) runs a command asynchronously will simply let it go and never look back THIS HAS A VERY NICHE USE CASE. THINK ABOUT IF YOU REALLY NEED IT.

```
#include <auto_command.h>
```

5.2.1 Detailed Description

[Async](#) runs a command asynchronously will simply let it go and never look back THIS HAS A VERY NICHE USE CASE. THINK ABOUT IF YOU REALLY NEED IT.

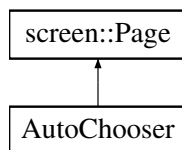
The documentation for this class was generated from the following files:

- [auto_command.h](#)
- [auto_command.cpp](#)

5.3 AutoChooser Class Reference

```
#include <auto_chooser.h>
```

Inheritance diagram for AutoChooser:



Classes

- struct [entry_t](#)

Public Member Functions

- [AutoChooser](#) (std::vector< std::string > paths, size_t def=0)
- size_t [get_choice](#) ()

Protected Attributes

- size_t [choice](#)
- std::vector< [entry_t](#) > [list](#)

5.3.1 Detailed Description

Autochooser is a utility to make selecting robot autonomous programs easier source: RIT VexU Wiki During a season, we usually code between 4 and 6 autonomous programs. Most teams will change their entire robot program as a way of choosing autonomi but this may cause issues if you have an emergency patch to upload during a competition. This class was built as a way of using the robot screen to list autonomous programs, and the touchscreen to select them.

5.3.2 Constructor & Destructor Documentation

AutoChooser()

```
AutoChooser::AutoChooser (
    std::vector< std::string > paths,
    size_t def = 0)
```

Initialize the auto-chooser. This class places a choice menu on the brain screen, so the driver can choose which autonomous to run.

Parameters

<i>brain</i>	the brain on which to draw the selection boxes
--------------	--

5.3.3 Member Function Documentation

get_choice()

```
size_t AutoChooser::get_choice ()
```

Get the currently selected auto choice

Returns

the identifier to the auto path

Return the selected autonomous

5.3.4 Member Data Documentation

choice

```
size_t AutoChooser::choice [protected]
```

the current choice of auto

list

```
std::vector<entry_t> AutoChooser::list [protected]
```

< a list of all possible auto choices

The documentation for this class was generated from the following files:

- auto_chooser.h
- auto_chooser.cpp

5.4 BasicSolenoidSet Class Reference

```
#include <basic_command.h>
```

Public Member Functions

- [BasicSolenoidSet](#) (vex::pneumatics &solenoid, bool setting)
Construct a new [BasicSolenoidSet](#) Command.
- bool [run](#) () override
Runs the [BasicSolenoidSet](#) Overrides run command from AutoCommand.

5.4.1 Detailed Description

AutoCommand wrapper class for [BasicSolenoidSet](#) Using the Vex hardware functions

5.4.2 Constructor & Destructor Documentation

BasicSolenoidSet()

```
BasicSolenoidSet::BasicSolenoidSet (
    vex::pneumatics & solenoid,
    bool setting)
```

Construct a new [BasicSolenoidSet](#) Command.

Parameters

<i>solenoid</i>	Solenoid being set
<i>setting</i>	Setting of the solenoid in boolean (true,false)

5.4.3 Member Function Documentation

run()

```
bool BasicSolenoidSet::run () [override]
```

Runs the [BasicSolenoidSet](#) Overrides run command from AutoCommand.

Returns

True Command runs once

The documentation for this class was generated from the following files:

- basic_command.h
- basic_command.cpp

5.5 BasicSpinCommand Class Reference

```
#include <basic_command.h>
```

Public Member Functions

- [BasicSpinCommand](#) (vex::motor &motor, vex::directionType dir, BasicSpinCommand::type setting, double power)
Construct a new [BasicSpinCommand](#).
- bool [run](#) () override
Runs the [BasicSpinCommand](#) Overrides run from Auto Command.

5.5.1 Detailed Description

AutoCommand wrapper class for [BasicSpinCommand](#) using the vex hardware functions

5.5.2 Constructor & Destructor Documentation

BasicSpinCommand()

```
BasicSpinCommand::BasicSpinCommand (
    vex::motor & motor,
    vex::directionType dir,
    BasicSpinCommand::type setting,
    double power)
```

Construct a new [BasicSpinCommand](#).

a BasicMotorSpin Command

Parameters

<i>motor</i>	Motor to spin
<i>direc</i>	Direction of motor spin
<i>setting</i>	Power setting in volts,percentage,velocity
<i>power</i>	Value of desired power
<i>motor</i>	Motor port to spin
<i>dir</i>	Direction for spinning
<i>setting</i>	Power setting in volts,percentage,velocity
<i>power</i>	Value of desired power

5.5.3 Member Function Documentation

run()

```
bool BasicSpinCommand::run () [override]
```

Runs the [BasicSpinCommand](#) Overrides run from Auto Command.

Run the [BasicSpinCommand](#) Overrides run from Auto Command.

Returns

True [Async](#) running command
True Command runs once

The documentation for this class was generated from the following files:

- [basic_command.h](#)
- [basic_command.cpp](#)

5.6 BasicStopCommand Class Reference

```
#include <basic_command.h>
```

Public Member Functions

- [BasicStopCommand](#) (vex::motor &motor, vex::brakeType setting)
Construct a new BasicMotorStop Command.
- bool [run](#) () override
Runs the BasicMotorStop Command Overrides run command from AutoCommand.

5.6.1 Detailed Description

AutoCommand wrapper class for [BasicStopCommand](#) Using the Vex hardware functions

5.6.2 Constructor & Destructor Documentation

BasicStopCommand()

```
BasicStopCommand::BasicStopCommand (
    vex::motor & motor,
    vex::brakeType setting)
```

Construct a new BasicMotorStop Command.

Construct a BasicMotorStop Command.

Parameters

<i>motor</i>	The motor to stop
<i>setting</i>	The brake setting for the motor
<i>motor</i>	Motor to stop
<i>setting</i>	Braketype setting brake,coast,hold

5.6.3 Member Function Documentation**run()**

```
bool BasicStopCommand::run () [override]
```

Runs the BasicMotorStop Command Overrides run command from AutoCommand.

Runs the BasicMotorStop command Overrides run command from AutoCommand.

Returns

True Command runs once

The documentation for this class was generated from the following files:

- basic_command.h
- basic_command.cpp

5.7 Branch Class Reference

[Branch](#) chooses from multiple options at runtime. the function decider returns an index into the choices vector If you wish to make no choice and skip this section, return NO_CHOICE; any choice that is out of bounds set to NO_CHOICE.

```
#include <auto_command.h>
```

5.7.1 Detailed Description

[Branch](#) chooses from multiple options at runtime. the function decider returns an index into the choices vector If you wish to make no choice and skip this section, return NO_CHOICE; any choice that is out of bounds set to NO_CHOICE.

The documentation for this class was generated from the following files:

- auto_command.h
- auto_command.cpp

5.8 screen::ButtonWidget Class Reference

Widget that does something when you tap it. The function is only called once when you first tap it.

```
#include <screen.h>
```

Public Member Functions

- [ButtonWidget](#) (std::function< void(void)> onpress, [Rect](#) rect, std::string name)
Create a Button widget.
- [ButtonWidget](#) (void(*onpress)(), [Rect](#) rect, std::string name)
Create a Button widget.
- bool [update](#) (bool was_pressed, int x, int y)
responds to user input
- void [draw](#) (vex::brain::lcd &, bool first_draw, unsigned int frame_number)
draws the button to the screen

5.8.1 Detailed Description

Widget that does something when you tap it. The function is only called once when you first tap it.

5.8.2 Constructor & Destructor Documentation

ButtonWidget() [1/2]

```
screen::ButtonWidget::ButtonWidget (
    std::function< void(void)> onpress,
    Rect rect,
    std::string name) [inline]
```

Create a Button widget.

Parameters

<i>onpress</i>	the function to be called when the button is tapped
<i>rect</i>	the area the button should take up on the screen
<i>name</i>	the label put on the button

ButtonWidget() [2/2]

```
screen::ButtonWidget::ButtonWidget (
    void(* onpress )(),
    Rect rect,
    std::string name) [inline]
```

Create a Button widget.

Parameters

<i>onpress</i>	the function to be called when the button is tapped
<i>rect</i>	the area the button should take up on the screen
<i>name</i>	the label put on the button

5.8.3 Member Function Documentation**update()**

```
bool screen::ButtonWidget::update (
    bool was_pressed,
    int x,
    int y)
```

responds to user input

Parameters

<i>was_pressed</i>	if the screen is pressed
<i>x</i>	x position if the screen was pressed
<i>y</i>	y position if the screen was pressed

Returns

true if the button was pressed

The documentation for this class was generated from the following files:

- screen.h
- screen.cpp

5.9 CommandController Class Reference

```
#include <command_controller.h>
```

Public Member Functions

- [CommandController](#) (std::initializer_list< AutoCommand * > cmds)
Create a [CommandController](#) with commands pre added. More can be added with [CommandController::add\(\)](#).
- void [add](#) (std::vector< AutoCommand * > cmds)
- void [add](#) (AutoCommand *cmd, double timeout_seconds=10.0)
- void [add](#) (std::vector< AutoCommand * > cmds, double timeout_sec)
- void [add_delay](#) (int ms)
- void [add_cancel_func](#) (std::function< bool(void)> true_if_cancel)
add_cancel_func specifies that when this func evaluates to true, to cancel the command controller
- void [run](#) ()
- bool [last_command_timed_out](#) ()

5.9.1 Detailed Description

File: [command_controller.h](#) Desc: A [CommandController](#) manages the AutoCommands that make up an autonomous route. The AutoCommands are kept in a queue and get executed and removed from the queue in FIFO order.

5.9.2 Constructor & Destructor Documentation

CommandController()

```
CommandController::CommandController (
    std::initializer_list< AutoCommand * > cmds) [inline]
```

Create a [CommandController](#) with commands pre added. More can be added with [CommandController::add\(\)](#).

Parameters

<i>cmds</i>	
-------------	--

5.9.3 Member Function Documentation

add() [1/3]

```
void CommandController::add (
    AutoCommand * cmd,
    double timeout_seconds = 10.0)
```

File: [command_controller.cpp](#) Desc: A [CommandController](#) manages the AutoCommands that make up an autonomous route. The AutoCommands are kept in a queue and get executed and removed from the queue in FIFO order. Adds a command to the queue

Parameters

<i>cmd</i>	the AutoCommand we want to add to our list
<i>timeout_seconds</i>	the number of seconds we will let the command run for. If it exceeds this, we cancel it and run on_timeout

add() [2/3]

```
void CommandController::add (
    std::vector< AutoCommand * > cmds)
```

Adds a command to the queue

Parameters

<i>cmd</i>	the AutoCommand we want to add to our list
<i>timeout_seconds</i>	the number of seconds we will let the command run for. If it exceeds this, we cancel it and run on_timeout. if it is <= 0 no time out will be applied

Add multiple commands to the queue. No timeout here.

Parameters

<i>cmds</i>	the AutoCommands we want to add to our list
-------------	---

add() [3/3]

```
void CommandController::add (
    std::vector< AutoCommand * > cmds,
    double timeout_sec)
```

Add multiple commands to the queue. No timeout here.

Parameters

<i>cmds</i>	the AutoCommands we want to add to our list Add multiple commands to the queue. No timeout here.
<i>cmds</i>	the AutoCommands we want to add to our list
<i>timeout_sec</i>	timeout in seconds to apply to all commands if they are still the default

Add multiple commands to the queue. No timeout here.

Parameters

<i>cmds</i>	the AutoCommands we want to add to our list
<i>timeout</i>	timeout in seconds to apply to all commands if they are still the default

add_cancel_func()

```
void CommandController::add_cancel_func (
    std::function< bool(void)> true_if_cancel)
```

add_cancel_func specifies that when this func evaluates to true, to cancel the command controller

Parameters

<i>true_if_cancel</i>	a function that returns true when we want to cancel the command controller
-----------------------	--

add_delay()

```
void CommandController::add_delay (
    int ms)
```

Parameters

<i>ms</i>	- number of milliseconds to wait before continuing execution of autonomous
-----------	--

last_command_timed_out()

```
bool CommandController::last_command_timed_out ()
```

last_command_timed_out tells how the last command ended Use this if you want to make decisions based on the end of the last command

Returns

true if the last command timed out. false if it finished regularly

run()

```
void CommandController::run ()
```

Begin execution of the queue Execute and remove commands in FIFO order

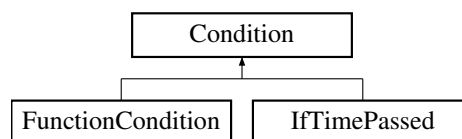
The documentation for this class was generated from the following files:

- command_controller.h
- command_controller.cpp

5.10 Condition Class Reference

```
#include <auto_command.h>
```

Inheritance diagram for Condition:

**5.10.1 Detailed Description**

File: [auto_command.h](#) Desc: Interface for module-specific commands A [Condition](#) is a function that returns true or false is_even is a predicate that would return true if a number is even For our purposes, a [Condition](#) is a choice to be made at runtime drive_sys.reached_point(10, 30) is a predicate time.has_elapsed(10, vex::seconds) is a predicate extend this class for different choices you wish to make

The documentation for this class was generated from the following files:

- auto_command.h
- auto_command.cpp

5.11 CRC32 Class Reference

A class for calculating the [CRC32](#) checksum from arbitrary data.

```
#include <crc32.hpp>
```

Public Member Functions

- **CRC32** ()
Initialize an empty [CRC32](#) checksum.
- void **reset** ()
Reset the checksum calculation.
- void **update** (const uint8_t &data)
Update the current checksum calculation with the given data.
- template<typename Type>
void **update** (const Type &data)
Update the current checksum calculation with the given data.
- template<typename Type>
void **update** (const Type *data, std::size_t size)
Update the current checksum calculation with the given data.
- uint32_t **finalize** () const

Static Public Member Functions

- template<typename Type>
static uint32_t **calculate** (const Type *data, std::size_t size)
Calculate the checksum of an arbitrary data array.

5.11.1 Detailed Description

A class for calculating the [CRC32](#) checksum from arbitrary data.

See also

<http://forum.arduino.cc/index.php?topic=91179.0>

5.11.2 Member Function Documentation

calculate()

```
template<typename Type>
uint32_t CRC32::calculate (
    const Type * data,
    std::size_t size) [inline], [static]
```

Calculate the checksum of an arbitrary data array.

Parameters

<i>Type</i>	The data type to read.
<i>data</i>	A pointer to the data to add to the checksum.
<i>size</i>	The size of the data to add to the checksum.

Returns

the calculated checksum.

finalize()

```
uint32_t CRC32::finalize () const
```

Returns

the calculated checksum.

update() [1/3]

```
template<typename Type>
void CRC32::update (
    const Type & data) [inline]
```

Update the current checksum calculation with the given data.

Parameters

<i>Type</i>	The data type to read.
<i>data</i>	The data to add to the checksum.

update() [2/3]

```
template<typename Type>
void CRC32::update (
    const Type * data,
    std::size_t size) [inline]
```

Update the current checksum calculation with the given data.

Parameters

<i>Type</i>	The data type to read.
<i>data</i>	The array to add to the checksum.
<i>size</i>	Size of the array to add.

Parameters

<i>data</i>	The data to add to the checksum.
-------------	----------------------------------

The documentation for this class was generated from the following files:

- crc32.hpp
- crc32.cpp

5.12 CustomEncoder Class Reference

```
#include <custom_encoder.h>
```

Public Member Functions

- [CustomEncoder](#) (vex::triport::port &port, double ticks_per_rev)
- void [setRotation](#) (double val, vex::rotationUnits units)
- void [setPosition](#) (double val, vex::rotationUnits units)
- double [rotation](#) (vex::rotationUnits units)
- double [position](#) (vex::rotationUnits units)
- double [velocity](#) (vex::velocityUnits units)

5.12.1 Detailed Description

A wrapper class for the vex encoder that allows the use of 3rd party encoders with different tick-per-revolution values.

5.12.2 Constructor & Destructor Documentation**CustomEncoder()**

```
CustomEncoder::CustomEncoder (
    vex::triport::port & port,
    double ticks_per_rev)
```

Construct an encoder with a custom number of ticks

Parameters

<i>port</i>	the triport port on the brain the encoder is plugged into
<i>ticks_per_rev</i>	the number of ticks the encoder will report for one revolution

5.12.3 Member Function Documentation**position()**

```
double CustomEncoder::position (
    vex::rotationUnits units)
```

Parameters

<i>units</i>	the unit we want the return value to be in
--------------	--

Returns

the position of the encoder in the units specified

rotation()

```
double CustomEncoder::rotation (
    vex::rotationUnits units)
```

get the rotation that the encoder is at

Parameters

<i>units</i>	the unit we want the return value to be in
--------------	--

Returns

the rotation of the encoder in the units specified

setPosition()

```
void CustomEncoder::setPosition (
    double val,
    vex::rotationUnits units)
```

sets the stored position of the encoder. Any further movements will be from this value

Parameters

<i>val</i>	the numerical value of the position we are setting to
<i>units</i>	the unit of val

setRotation()

```
void CustomEncoder::setRotation (
    double val,
    vex::rotationUnits units)
```

sets the stored rotation of the encoder. Any further movements will be from this value

Parameters

<i>val</i>	the numerical value of the angle we are setting to
<i>units</i>	the unit of val

velocity()

```
double CustomEncoder::velocity (
    vex::velocityUnits units)
```

get the velocity that the encoder is moving at

Parameters

<i>units</i>	the unit we want the return value to be in
--------------	--

Returns

the velocity of the encoder in the units specified

The documentation for this class was generated from the following files:

- custom_encoder.h
- custom_encoder.cpp

5.13 DelayCommand Class Reference

```
#include <delay_command.h>
```

Public Member Functions

- [DelayCommand](#) (int ms)
- bool [run](#) () override

5.13.1 Detailed Description

File: [delay_command.h](#) Desc: A [DelayCommand](#) will make the robot wait the set amount of milliseconds before continuing execution of the autonomous route

5.13.2 Constructor & Destructor Documentation

DelayCommand()

```
DelayCommand::DelayCommand (
    int ms) [inline]
```

Construct a delay command

Parameters

<i>ms</i>	the number of milliseconds to delay for
-----------	---

5.13.3 Member Function Documentation**run()**

```
bool DelayCommand::run () [inline], [override]
```

Delays for the amount of milliseconds stored in the command Overrides run from AutoCommand

Returns

true when complete

The documentation for this class was generated from the following file:

- delay_command.h

5.14 DriveForwardCommand Class Reference

```
#include <drive_commands.h>
```

Public Member Functions

- [DriveForwardCommand](#) ([TankDrive](#) &drive_sys, [Feedback](#) &feedback, double inches, directionType dir, double max_speed=1, double end_speed=0)
- bool [run](#) () override
- void [on_timeout](#) () override

5.14.1 Detailed Description

AutoCommand wrapper class for the drive_forward function in the [TankDrive](#) class

5.14.2 Constructor & Destructor Documentation

DriveForwardCommand()

```
DriveForwardCommand::DriveForwardCommand (
    TankDrive & drive_sys,
    Feedback & feedback,
    double inches,
    directionType dir,
    double max_speed = 1,
    double end_speed = 0)
```

File: [drive_commands.h](#) Desc: Holds all the AutoCommand subclasses that wrap (currently) [TankDrive](#) functions

Currently includes:

- [drive_forward](#)
- [turn_degrees](#)
- [drive_to_point](#)
- [turn_to_heading](#)
- [stop](#)

Also holds AutoCommand subclasses that wrap [OdometryBase](#) functions

Currently includes:

- [set_position](#) Construct a DriveForward Command

Parameters

<i>drive_sys</i>	the drive system we are commanding
<i>feedback</i>	the feedback controller we are using to execute the drive
<i>inches</i>	how far forward to drive
<i>dir</i>	the direction to drive
<i>max_speed</i>	0 -> 1 percentage of the drive systems speed to drive at

5.14.3 Member Function Documentation

on_timeout()

```
void DriveForwardCommand::on_timeout () [override]
```

Cleans up drive system if we time out before finishing

reset the drive system if we timeout

run()

```
bool DriveForwardCommand::run () [override]
```

Run drive_forward Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- drive_commands.h
- drive_commands.cpp

5.15 DriveStopCommand Class Reference

```
#include <drive_commands.h>
```

Public Member Functions

- [DriveStopCommand](#) ([TankDrive](#) &drive_sys)
- bool [run](#) () override

5.15.1 Detailed Description

AutoCommand wrapper class for the stop() function in the [TankDrive](#) class

5.15.2 Constructor & Destructor Documentation**DriveStopCommand()**

```
DriveStopCommand::DriveStopCommand (
    TankDrive & drive_sys)
```

Construct a DriveStop Command

Parameters

<i>drive_sys</i>	the drive system we are commanding
------------------	------------------------------------

5.15.3 Member Function Documentation

run()

```
bool DriveStopCommand::run () [override]
```

Stop the drive system Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

Stop the drive train Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- drive_commands.h
- drive_commands.cpp

5.16 DriveToPointCommand Class Reference

```
#include <drive_commands.h>
```

Public Member Functions

- [DriveToPointCommand](#) ([TankDrive](#) &drive_sys, [Feedback](#) &feedback, double x, double y, directionType dir, double max_speed=1, double end_speed=0)
- [DriveToPointCommand](#) ([TankDrive](#) &drive_sys, [Feedback](#) &feedback, [Translation2d](#) translation, directionType dir, double max_speed=1, double end_speed=0)
- bool [run](#) () override

5.16.1 Detailed Description

AutoCommand wrapper class for the drive_to_point function in the [TankDrive](#) class

5.16.2 Constructor & Destructor Documentation

DriveToPointCommand() [1/2]

```
DriveToPointCommand::DriveToPointCommand (  
    TankDrive & drive_sys,  
    Feedback & feedback,  
    double x,  
    double y,  
    directionType dir,  
    double max_speed = 1,  
    double end_speed = 0)
```

Parameters

<i>drive_sys</i>	the drive system we are commanding
<i>feedback</i>	the feedback controller we are using to execute the drive
<i>x</i>	where to drive in the x dimension
<i>y</i>	where to drive in the y dimension
<i>dir</i>	the direction to drive
<i>max_speed</i>	0 -> 1 percentage of the drive systems speed to drive at

DriveToPointCommand() [2/2]

```
DriveToPointCommand::DriveToPointCommand (
    TankDrive & drive_sys,
    Feedback & feedback,
    Translation2d translation,
    directionType dir,
    double max_speed = 1,
    double end_speed = 0)
```

Construct a DriveForward Command

Parameters

<i>drive_sys</i>	the drive system we are commanding
<i>feedback</i>	the feedback controller we are using to execute the drive
<i>translation</i>	the point to drive to
<i>dir</i>	the direction to drive
<i>max_speed</i>	0 -> 1 percentage of the drive systems speed to drive at

5.16.3 Member Function Documentation**run()**

```
bool DriveToPointCommand::run () [override]
```

Run drive_to_point Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- drive_commands.h
- drive_commands.cpp

5.17 AutoChooser::entry_t Struct Reference

```
#include <auto_chooser.h>
```

Public Attributes

- `std::string` [name](#)

5.17.1 Detailed Description

[entry_t](#) is a datatype used to store information that the chooser knows about an auto selection button

5.17.2 Member Data Documentation

name

```
std::string AutoChooser::entry_t::name
```

name of the auto represented by the block

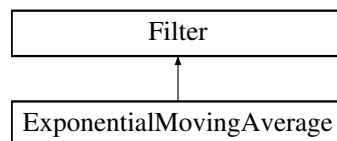
The documentation for this struct was generated from the following file:

- `auto_chooser.h`

5.18 ExponentialMovingAverage Class Reference

```
#include <moving_average.h>
```

Inheritance diagram for ExponentialMovingAverage:



Public Member Functions

- [ExponentialMovingAverage](#) (int buffer_size)
- [ExponentialMovingAverage](#) (int buffer_size, double starting_value)
- void [add_entry](#) (double n) override
- double [get_value](#) () const override
- int [get_size](#) ()

5.18.1 Detailed Description

ExponentialMovingAverage

An exponential moving average is a way of smoothing out noisy data. For many sensor readings, the noise is roughly symmetric around the actual value. This means that if you collect enough samples those that are too high are cancelled out by the samples that are too low leaving the real value.

A simple moving average lags significantly with time as it has to counteract old samples. An exponential moving average keeps more up to date by weighting newer readings higher than older readings so it is more up to date while also still smoothed.

The [ExponentialMovingAverage](#) class provides an simple interface to do this smoothing from our noisy sensor values.

5.18.2 Constructor & Destructor Documentation

ExponentialMovingAverage() [1/2]

```
ExponentialMovingAverage::ExponentialMovingAverage (
    int buffer_size)
```

Create a moving average calculator with 0 as the default value

Parameters

<i>buffer_size</i>	The size of the buffer. The number of samples that constitute a valid reading
--------------------	---

ExponentialMovingAverage() [2/2]

```
ExponentialMovingAverage::ExponentialMovingAverage (
    int buffer_size,
    double starting_value)
```

Create a moving average calculator with a specified default value

Parameters

<i>buffer_size</i>	The size of the buffer. The number of samples that constitute a valid reading
<i>starting_value</i>	The value that the average will be before any data is added

5.18.3 Member Function Documentation

add_entry()

```
void ExponentialMovingAverage::add_entry (
    double n) [override], [virtual]
```

Add a reading to the buffer Before: [1 1 2 2 3 3] => 2¹⁶ After: [2 1 2 2 3 3] => 2.16¹⁶

Parameters

n	the sample that will be added to the moving average.
-----	--

Implements [Filter](#).

get_size()

```
int ExponentialMovingAverage::get_size ()
```

How many samples the average is made from

Returns

the number of samples used to calculate this average

get_value()

```
double ExponentialMovingAverage::get_value () const [override], [virtual]
```

Returns the average based off of all the samples collected so far

Returns

the calculated average. `sum(samples)/numsamples`

How many samples the average is made from

Returns

the number of samples used to calculate this average

Implements [Filter](#).

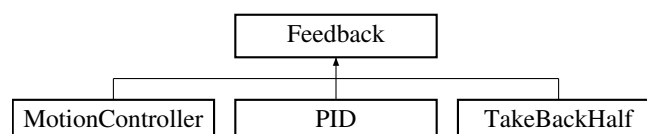
The documentation for this class was generated from the following files:

- `moving_average.h`
- `moving_average.cpp`

5.19 Feedback Class Reference

```
#include <feedback_base.h>
```

Inheritance diagram for Feedback:



Public Member Functions

- virtual void [init](#) (double start_pt, double set_pt)=0
- virtual double [update](#) (double val)=0
- virtual double [get](#) ()=0
- virtual void [set_limits](#) (double lower, double upper)=0
- virtual bool [is_on_target](#) ()=0

5.19.1 Detailed Description

Interface so that subsystems can easily switch between feedback loops

Author

Ryan McGee

Date

9/25/2022

5.19.2 Member Function Documentation

get()

```
virtual double Feedback::get () [pure virtual]
```

Returns

the last saved result from the feedback controller

Implemented in [MotionController](#), [PID](#), and [TakeBackHalf](#).

init()

```
virtual void Feedback::init (  
    double start_pt,  
    double set_pt) [pure virtual]
```

Initialize the feedback controller for a movement

Parameters

<i>start_pt</i>	the current sensor value
<i>set_pt</i>	where the sensor value should be
<i>start_vel</i>	Movement starting velocity

<i>end_vel</i>	Movement ending velocity
----------------	--------------------------

Implemented in [MotionController](#), [PID](#), and [TakeBackHalf](#).

is_on_target()

```
virtual bool Feedback::is_on_target () [pure virtual]
```

Returns

true if the feedback controller has reached it's setpoint

Implemented in [MotionController](#), [PID](#), and [TakeBackHalf](#).

set_limits()

```
virtual void Feedback::set_limits (  
    double lower,  
    double upper) [pure virtual]
```

Clamp the upper and lower limits of the output. If both are 0, no limits should be applied.

Parameters

<i>lower</i>	Upper limit
<i>upper</i>	Lower limit

Implemented in [MotionController](#), [PID](#), and [TakeBackHalf](#).

update()

```
virtual double Feedback::update (  
    double val) [pure virtual]
```

Iterate the feedback loop once with an updated sensor value

Parameters

<i>val</i>	value from the sensor
------------	-----------------------

Returns

feedback loop result

Implemented in [MotionController](#), [PID](#), and [TakeBackHalf](#).

The documentation for this class was generated from the following file:

- `feedback_base.h`

5.20 FeedForward Class Reference

```
#include <feedforward.h>
```

Classes

- struct [ff_config_t](#)

Public Member Functions

- [FeedForward](#) ([ff_config_t](#) &cfg)
- double [calculate](#) (double v, double a, double pid_ref=0.0)
Perform the feedforward calculation.

5.20.1 Detailed Description

[FeedForward](#)

Stores the feedforward constants, and allows for quick computation. Feedforward should be used in systems that require smooth precise movements and have high inertia, such as drivetrains and lifts.

This is best used alongside a [PID](#) loop, with the form: `output = pid.get() + feedforward.calculate(v, a);`

In this case, the feedforward does the majority of the heavy lifting, and the pid loop only corrects for inconsistencies

For information about tuning feedforward, I recommend looking at this post: <https://www.chiefdelphi.com/t/paper-frc-drivetrain-characterization/160915> (yes I know it's for FRC but trust me, it's useful)

Author

Ryan McGee

Date

6/13/2022

5.20.2 Constructor & Destructor Documentation

FeedForward()

```
FeedForward::FeedForward (  
    ff\_config\_t & cfg) [inline]
```

Creates a [FeedForward](#) object.

Parameters

<i>cfg</i>	Configuration Struct for tuning
------------	---------------------------------

5.20.3 Member Function Documentation**calculate()**

```
double FeedForward::calculate (
    double v,
    double a,
    double pid_ref = 0.0) [inline]
```

Perform the feedforward calculation.

This calculation is the equation: $F = kG + kS*\text{sgn}(v) + kV*v + kA*a$

Parameters

<i>v</i>	Requested velocity of system
<i>a</i>	Requested acceleration of system

Returns

A feedforward that should closely represent the system if tuned correctly

The documentation for this class was generated from the following file:

- feedforward.h

5.21 FeedForward::ff_config_t Struct Reference

```
#include <feedforward.h>
```

Public Attributes

- double [kS](#)
- double [kV](#)
- double [kA](#)
- double [kG](#)

5.21.1 Detailed Description

[ff_config_t](#) holds the parameters to make the theoretical model of a real world system equation is of the form kS if the system is not stopped, 0 otherwise

- $kV * \text{desired velocity}$
- $kA * \text{desired acceleration}$
- kG

5.21.2 Member Data Documentation

kA

```
double FeedForward::ff_config_t::kA
```

kA - Acceleration coefficient: the power required to change the mechanism's speed. Multiplied by the requested acceleration.

kG

```
double FeedForward::ff_config_t::kG
```

kG - Gravity coefficient: only needed for lifts. The power required to overcome gravity and stay at steady state.

kS

```
double FeedForward::ff_config_t::kS
```

Coefficient to overcome static friction: the point at which the motor *starts* to move.

kV

```
double FeedForward::ff_config_t::kV
```

Veclocity coefficient: the power required to keep the mechanism in motion. Multiplied by the requested velocity.

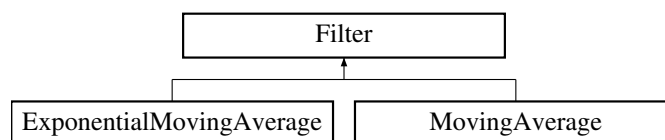
The documentation for this struct was generated from the following file:

- feedforward.h

5.22 Filter Class Reference

```
#include <moving_average.h>
```

Inheritance diagram for Filter:



5.22.1 Detailed Description

Interface for filters Use `add_entry` to supply data and `get_value` to retrieve the filtered value

The documentation for this class was generated from the following file:

- moving_average.h

5.23 Flywheel Class Reference

```
#include <flywheel.h>
```

Public Member Functions

- [Flywheel](#) (vex::motor_group &motors, [Feedback](#) &feedback, [FeedForward](#) &helper, const double ratio, [Filter](#) &filt)
- double [get_target](#) () const
- double [getRPM](#) () const
- vex::motor_group & [get_motors](#) () const
- void [spin_manual](#) (double speed, directionType dir=fwd)
- void [spin_rpm](#) (double rpm)
- void [stop](#) ()
- bool [is_on_target](#) ()
 - check if the feedback controller thinks the flywheel is on target*
- AutoCommand * [SpinRpmCmd](#) (int rpm)
 - Creates a page displaying info about the flywheel.*
- AutoCommand * [WaitUntilUpToSpeedCmd](#) ()
 - Creates a new auto command that will hold until the flywheel has its target as defined by its feedback controller.*

Friends

- int [spinRPMTask](#) (void *wheelPointer)

5.23.1 Detailed Description

a [Flywheel](#) class that handles all control of a high inertia spinning disk. It gives multiple options for what control system to use in order to control wheel velocity and functions alerting the user when the flywheel is up to speed. [Flywheel](#) is a set and forget class. Once you create it you can call `spin_rpm` or `stop` on it at any time and it will take all necessary steps to accomplish this.

5.23.2 Constructor & Destructor Documentation

Flywheel()

```
Flywheel::Flywheel (
    vex::motor_group & motors,
    Feedback & feedback,
    FeedForward & helper,
    const double ratio,
    Filter & filt)
```

Create the [Flywheel](#) object using [PID](#) + feedforward for control.

Parameters

<i>motors</i>	pointer to the motors on the fly wheel
---------------	--

<i>feedback</i>	a feedback controller
<i>helper</i>	a feedforward config (only kV is used) to help the feedback controller along
<i>ratio</i>	ratio of the gears from the motor to the flywheel just multiplies the velocity
<i>filter</i>	the filter to use to smooth noisy motor readings

5.23.3 Member Function Documentation

get_motors()

```
motor_group & Flywheel::get_motors () const
```

Returns the motors

Returns

the motors used to run the flywheel

get_target()

```
double Flywheel::get_target () const
```

Return the target_rpm that the flywheel is currently trying to achieve

Returns

target_rpm the target rpm

Return the current value that the target_rpm should be set to

getRPM()

```
double Flywheel::getRPM () const
```

return the velocity of the flywheel

is_on_target()

```
bool Flywheel::is_on_target () [inline]
```

check if the feedback controller thinks the flywheel is on target

Returns

true if on target

Parameters

<i>speed</i>	- speed (between -1 and 1) to set the motor
<i>dir</i>	- direction that the motor moves in; defaults to forward

Spin motors using voltage; defaults forward at 12 volts FOR USE BY OPCONTROL AND AUTONOMOUS - this only applies if the RPM thread is not running

Parameters

<i>speed</i>	- speed (between -1 and 1) to set the motor
<i>dir</i>	- direction that the motor moves in; defaults to forward

spin_rpm()

```
void Flywheel::spin_rpm (  
    double input_rpm)
```

starts or sets the target_rpm thread at new value what control scheme is dependent on control_style

Parameters

<i>rpm</i>	- the target_rpm we want to spin at
------------	-------------------------------------

starts or sets the RPM thread at new value what control scheme is dependent on control_style

Parameters

<i>input_rpm</i>	- set the current RPM
------------------	-----------------------

SpinRpmCmd()

```
AutoCommand * Flywheel::SpinRpmCmd (  
    int rpm) [inline]
```

Creates a page displaying info about the flywheel.

Returns

the page should be used for ``screen::start_screen(screen, {fw.Page()});`` [screen::Page](#) *Page() const;

```
/**
```

Creates a new auto command to spin the flywheel at the desired velocity

Parameters

<i>rpm</i>	the rpm to spin at
------------	--------------------

Returns

an auto command to add to a command controller

stop()

```
void Flywheel::stop ()
```

Stops the motors. If manually spinning, this will do nothing just call `spin_manual(0.0)` to send 0 volts

stop the RPM thread and the wheel

WaitUntilUpToSpeedCmd()

```
AutoCommand * Flywheel::WaitUntilUpToSpeedCmd () [inline]
```

Creates a new auto command that will hold until the flywheel has its target as defined by its feedback controller.

Returns

an auto command to add to a command controller

5.23.4 Friends And Related Symbol Documentation**spinRPMTask**

```
int spinRPMTask (
    void * wheelPointer) [friend]
```

Runs a thread that keeps track of updating flywheel RPM and controlling it accordingly

The documentation for this class was generated from the following files:

- flywheel.h
- flywheel.cpp

5.24 FlywheelStopCommand Class Reference

```
#include <flywheel_commands.h>
```

Public Member Functions

- [FlywheelStopCommand](#) ([Flywheel](#) &flywheel)
- bool [run](#) () override

5.24.1 Detailed Description

AutoCommand wrapper class for the stop function in the [Flywheel](#) class

5.24.2 Constructor & Destructor Documentation

FlywheelStopCommand()

```
FlywheelStopCommand::FlywheelStopCommand (  
    Flywheel & flywheel)
```

Construct a [FlywheelStopCommand](#)

Parameters

<i>flywheel</i>	the flywheel system we are commanding
-----------------	---------------------------------------

5.24.3 Member Function Documentation

run()

```
bool FlywheelStopCommand::run () [override]
```

Run stop Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- flywheel_commands.h
- flywheel_commands.cpp

5.25 FlywheelStopMotorsCommand Class Reference

```
#include <flywheel_commands.h>
```

Public Member Functions

- [FlywheelStopMotorsCommand](#) ([Flywheel](#) &flywheel)
- bool [run](#) () override

5.25.1 Detailed Description

AutoCommand wrapper class for the stopMotors function in the [Flywheel](#) class

5.25.2 Constructor & Destructor Documentation

FlywheelStopMotorsCommand()

```
FlywheelStopMotorsCommand::FlywheelStopMotorsCommand (
    Flywheel & flywheel)
```

Construct a FlywheelStopMotors Command

Parameters

<i>flywheel</i>	the flywheel system we are commanding
-----------------	---------------------------------------

5.25.3 Member Function Documentation

run()

```
bool FlywheelStopMotorsCommand::run () [override]
```

Run stop Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- flywheel_commands.h
- flywheel_commands.cpp

5.26 FlywheelStopNonTasksCommand Class Reference

```
#include <flywheel_commands.h>
```

5.26.1 Detailed Description

AutoCommand wrapper class for the stopNonTasks function in the [Flywheel](#) class

The documentation for this class was generated from the following files:

- flywheel_commands.h
- flywheel_commands.cpp

5.27 FunctionCommand Class Reference

```
#include <auto_command.h>
```

5.27.1 Detailed Description

[FunctionCommand](#) is fun and good way to do simple things Printing, launching nukes, and other quick and dirty one time things

The documentation for this class was generated from the following file:

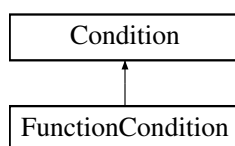
- [auto_command.h](#)

5.28 FunctionCondition Class Reference

[FunctionCondition](#) is a quick and dirty [Condition](#) to wrap some expression that should be evaluated at runtime.

```
#include <auto_command.h>
```

Inheritance diagram for FunctionCondition:



5.28.1 Detailed Description

[FunctionCondition](#) is a quick and dirty [Condition](#) to wrap some expression that should be evaluated at runtime.

The documentation for this class was generated from the following files:

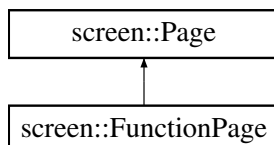
- [auto_command.h](#)
- [auto_command.cpp](#)

5.29 screen::FunctionPage Class Reference

Simple page that stores no internal data. the draw and update functions use only global data rather than storing anything.

```
#include <screen.h>
```

Inheritance diagram for screen::FunctionPage:



Public Member Functions

- [FunctionPage](#) (update_func_t update_f, draw_func_t draw_t)
Creates a function page.
- void [update](#) (bool was_pressed, int x, int y) override
update uses the supplied update function to update this page
- void [draw](#) (vex::brain::lcd &, bool first_draw, unsigned int frame_number) override
draw uses the supplied draw function to draw to the screen

5.29.1 Detailed Description

Simple page that stores no internal data. the draw and update functions use only global data rather than storing anything.

5.29.2 Constructor & Destructor Documentation

FunctionPage()

```
screen::FunctionPage::FunctionPage (
    update_func_t update_f,
    draw_func_t draw_f)
```

Creates a function page.

[FunctionPage](#).

Parameters

<i>update↔ _f</i>	the function called every tick to respond to user input or do data collection
<i>draw_t</i>	the function called to draw to the screen
<i>update↔ _f</i>	drawing function
<i>draw_f</i>	drawing function

5.29.3 Member Function Documentation

draw()

```
void screen::FunctionPage::draw (
    vex::brain::lcd & screen,
    bool first_draw,
    unsigned int frame_number) [override], [virtual]
```

draw uses the supplied draw function to draw to the screen

See also

[Page::draw](#)

Reimplemented from [screen::Page](#).

update()

```
void screen::FunctionPage::update (
    bool was_pressed,
    int x,
    int y) [override], [virtual]
```

update uses the supplied update function to update this page

See also

[Page::update](#)

Reimplemented from [screen::Page](#).

The documentation for this class was generated from the following files:

- screen.h
- screen.cpp

5.30 GenericAuto Class Reference

```
#include <generic_auto.h>
```

Public Member Functions

- bool [run](#) (bool blocking)
- void [add](#) (state_ptr new_state)
- void [add_async](#) (state_ptr async_state)
- void [add_delay](#) (int ms)

5.30.1 Detailed Description

[GenericAuto](#) provides a pleasant interface for organizing an auto path steps of the path can be added with [add\(\)](#) and when ready, calling [run\(\)](#) will begin executing the path

5.30.2 Member Function Documentation

add()

```
void GenericAuto::add (
    state_ptr new_state)
```

Add a new state to the autonomous via function point of type "bool (ptr*)()"

Parameters

<i>new_state</i>	the function to run
------------------	---------------------

add_async()

```
void GenericAuto::add_async (  
    state_ptr async_state)
```

Add a new state to the autonomous via function point of type "bool (ptr*)()" that will run asynchronously

Parameters

<i>async_state</i>	the function to run
--------------------	---------------------

add_delay()

```
void GenericAuto::add_delay (  
    int ms)
```

add_delay adds a period where the auto system will simply wait for the specified time

Parameters

<i>ms</i>	how long to wait in milliseconds
-----------	----------------------------------

run()

```
bool GenericAuto::run (  
    bool blocking)
```

The method that runs the autonomous. If 'blocking' is true, then this method will run through every state until it finished.

If blocking is false, then assuming every state is also non-blocking, the method will run through the current state in the list and return immediately.

Parameters

<i>blocking</i>	Whether or not to block the thread until all states have run
-----------------	--

Returns

true after all states have finished.

The documentation for this class was generated from the following files:

- generic_auto.h
- generic_auto.cpp

5.31 PurePursuit::hermite_point Struct Reference

```
#include <pure_pursuit.h>
```

5.31.1 Detailed Description

a position along the hermite path contains a position and orientation information that the robot would be at at this point

The documentation for this struct was generated from the following file:

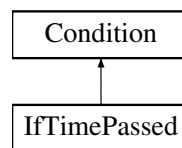
- pure_pursuit.h

5.32 IfTimePassed Class Reference

[IfTimePassed](#) tests based on time since the command controller was constructed. Returns true if elapsed time > time_s.

```
#include <auto_command.h>
```

Inheritance diagram for IfTimePassed:



5.32.1 Detailed Description

[IfTimePassed](#) tests based on time since the command controller was constructed. Returns true if elapsed time > time_s.

The documentation for this class was generated from the following files:

- auto_command.h
- auto_command.cpp

5.33 InOrder Class Reference

[InOrder](#) runs its commands sequentially then continues. How to handle timeout in this case. Automatically set it to sum of commands timeouts?

```
#include <auto_command.h>
```

5.33.1 Detailed Description

`InOrder` runs its commands sequentially then continues. How to handle timeout in this case. Automatically set it to sum of commands timeouts?

`InOrder` runs its commands sequentially then continues. How to handle timeout in this case. Automatically set it to sum of commands timeouts?

The documentation for this class was generated from the following files:

- `auto_command.h`
- `auto_command.cpp`

5.34 InterpolatingMap< KEY, VALUE > Class Template Reference

```
#include <interpolating_map.h>
```

Public Member Functions

- void `insert` (const KEY &key, const VALUE &value)
- VALUE `operator[]` (const KEY &key)
- void `clear` ()

5.34.1 Detailed Description

```
template<typename KEY, typename VALUE>
class InterpolatingMap< KEY, VALUE >
```

This class implements a map of key-value pairs.

If there is not a pair with the given key in the map, the value will be a linear interpolation of the preceding and following values.

Template Parameters

<i>KEY</i>	The type of the key.
<i>VALUE</i>	The type of the value.

5.34.2 Member Function Documentation

`clear()`

```
template<typename KEY, typename VALUE>
void InterpolatingMap< KEY, VALUE >::clear () [inline]
```

Clears the contents of the map.

Parameters

<i>key</i>	The key.
<i>value</i>	The value.

operator[]()

```
template<typename KEY, typename VALUE>
VALUE InterpolatingMap< KEY, VALUE >::operator[] (
    const KEY & key) [inline]
```

Obtains the value at the given key.

If the key does not exactly match a pair in the map, it will interpolate between the preceding and following pairs.

Parameters

<i>key</i>	The key.
------------	----------

Returns

The value.

The documentation for this class was generated from the following file:

- interpolating_map.h

5.35 KalmanFilter< STATES, INPUTS, OUTPUTS > Class Template Reference

```
#include <kalman_filter.h>
```

Public Member Functions

- [KalmanFilter](#) ([LinearSystem](#)< STATES, INPUTS, OUTPUTS > &plant, const StateVector &state_stddevs, const OutputVector &measurement_stddevs)
- [KalmanFilter](#) (const StateMatrix &A, const InputMatrix &B, const EMat< OUTPUTS, STATES > &C, const EMat< OUTPUTS, INPUTS > &D, const StateVector &state_stddevs, const OutputVector &measurement_stddevs)
- StateMatrix [P](#) () const
- void [set_P](#) (const StateMatrix &P)
- const StateVector & [xhat](#) () const
- double [xhat](#) (int i) const
- void [set_xhat](#) (const StateVector &xhat)
- void [set_xhat](#) (int i, double value)
- void [reset](#) ()
- void [predict](#) (const InputVector &u, const double &dt)
- void [correct](#) (const OutputVector &y, const InputVector &u)
- void [correct](#) (const OutputVector &y, const InputVector &u, const EMat< OUTPUTS, OUTPUTS > &R)
- template<int ROWS>
void [correct](#) (const EVec< ROWS > &y, const InputVector &u, const EMat< ROWS, STATES > &C, const EMat< ROWS, INPUTS > &D, const EMat< ROWS, ROWS > &R)

5.35.1 Detailed Description

```
template<int STATES, int INPUTS, int OUTPUTS>
class KalmanFilter< STATES, INPUTS, OUTPUTS >
```

Kalman filters combine predictions from a model and measurements to estimate a system's true state.

Each call of predict moves the state forward in time according to the matrix A, and the covariance has white noise Q added.

Each call of correct applies a measurement which moves the state more toward the true state, and it reduces the state covariance.

To read more about Kalman filters read: <https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python>

Template Parameters

<i>STATES</i>	Dimension of the state vector.
<i>INPUTS</i>	Dimension of the control input vector.
<i>OUTPUTS</i>	Dimension of the measurement vector.

5.35.2 Constructor & Destructor Documentation

KalmanFilter() [1/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
KalmanFilter< STATES, INPUTS, OUTPUTS >::KalmanFilter (
    LinearSystem< STATES, INPUTS, OUTPUTS > & plant,
    const StateVector & state_stddevs,
    const OutputVector & measurement_stddevs) [inline]
```

Constructs a Kalman filter.

Parameters

<i>plant</i>	The linear system the filter tracks.
<i>state_stddevs</i>	The standard deviations of the states.
<i>measurement_stddevs</i>	The standard deviations of the measurements.

KalmanFilter() [2/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
KalmanFilter< STATES, INPUTS, OUTPUTS >::KalmanFilter (
    const StateMatrix & A,
    const InputMatrix & B,
    const EMat< OUTPUTS, STATES > & C,
    const EMat< OUTPUTS, INPUTS > & D,
    const StateVector & state_stddevs,
    const OutputVector & measurement_stddevs) [inline]
```

Generated by Doxygen

Constructs a Kalman filter.

Parameters

A	The state matrix.
B	The input matrix.
C	The measurement matrix.
D	The feedthrough matrix.
<i>state_stddevs</i>	The standard deviations of the states.
<i>measurement_stddevs</i>	The standard deviations of the measurements.

5.35.3 Member Function Documentation**correct()** [1/3]

```
template<int STATES, int INPUTS, int OUTPUTS>
template<int ROWS>
void KalmanFilter< STATES, INPUTS, OUTPUTS >::correct (
    const EVec< ROWS > & y,
    const InputVector & u,
    const EMat< ROWS, STATES > & C,
    const EMat< ROWS, INPUTS > & D,
    const EMat< ROWS, ROWS > & R) [inline]
```

Correct the state estimate using the measurements in y , custom measurement and feedthrough matrices, and custom measurement measurement noise. This is useful for when a different set of measurements are being applied than what the plant defines.

Parameters

y	The vector of measurements.
u	The control input used in the last predict step.
C	The measurement matrix to use for this step.
D	The feedthrough matrix to use for this step.
R	The measurement noise matrix to use for this step.

correct() [2/3]

```
template<int STATES, int INPUTS, int OUTPUTS>
void KalmanFilter< STATES, INPUTS, OUTPUTS >::correct (
    const OutputVector & y,
    const InputVector & u) [inline]
```

Correct the state estimate using the measurements in y .

Parameters

y	The vector of measurements.
u	The control input used in the last predict step.

correct() [3/3]

```
template<int STATES, int INPUTS, int OUTPUTS>
void KalmanFilter< STATES, INPUTS, OUTPUTS >::correct (
    const OutputVector & y,
    const InputVector & u,
    const EMat< OUTPUTS, OUTPUTS > & R) [inline]
```

Correct the state estimate using the measurements in y , and custom measurement noise matrix. This is useful for when the noise in the measurements vary.

Parameters

y	The vector of measurements.
u	The control input used in the last predict step.
R	The measurement noise matrix to use for this step.

P()

```
template<int STATES, int INPUTS, int OUTPUTS>
StateMatrix KalmanFilter< STATES, INPUTS, OUTPUTS >::P () const [inline]
```

Returns the covariance matrix P .

predict()

```
template<int STATES, int INPUTS, int OUTPUTS>
void KalmanFilter< STATES, INPUTS, OUTPUTS >::predict (
    const InputVector & u,
    const double & dt) [inline]
```

Projects the state into the future by dt seconds with control input u .

Parameters

u	The control input.
dt	The timestep in seconds.

reset()

```
template<int STATES, int INPUTS, int OUTPUTS>
void KalmanFilter< STATES, INPUTS, OUTPUTS >::reset () [inline]
```

Resets the filter.

set_P()

```
template<int STATES, int INPUTS, int OUTPUTS>
void KalmanFilter< STATES, INPUTS, OUTPUTS >::set_P (
    const StateMatrix & P) [inline]
```

Parameters

P	The covariance matrix P .
-----	-----------------------------

set_xhat() [1/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
void KalmanFilter< STATES, INPUTS, OUTPUTS >::set_xhat (
    const StateVector & xhat) [inline]
```

Set the current state estimate x-hat.

set_xhat() [2/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
void KalmanFilter< STATES, INPUTS, OUTPUTS >::set_xhat (
    int i,
    double value) [inline]
```

Set one element of the current state estimate x-hat.

Parameters

i	Row of x-hat.
-----	---------------

xhat() [1/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
const StateVector & KalmanFilter< STATES, INPUTS, OUTPUTS >::xhat () const [inline]
```

Returns the current state estimate x-hat.

xhat() [2/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
double KalmanFilter< STATES, INPUTS, OUTPUTS >::xhat (
    int i) const [inline]
```

Returns one element of the current state estimate x-hat.

Parameters

i	Row of x-hat.
-----	---------------

The documentation for this class was generated from the following file:

- kalman_filter.h

5.36 Lift< T > Class Template Reference

```
#include <lift.h>
```

Classes

- struct [lift_cfg_t](#)

Public Member Functions

- [Lift](#) (motor_group &lift_motors, [lift_cfg_t](#) &lift_cfg, map< T, double > &setpoint_map, limit *homing_switch=NULL)
- void [control_continuous](#) (bool up_ctrl, bool down_ctrl)
- void [control_manual](#) (bool up_btn, bool down_btn, int volt_up, int volt_down)
- void [control_setpoints](#) (bool up_step, bool down_step, vector< T > pos_list)
- bool [set_position](#) (T pos)
- bool [set_setpoint](#) (double val)
- double [get_setpoint](#) ()
- void [hold](#) ()
- void [home](#) ()
- bool [get_async](#) ()
- void [set_async](#) (bool val)
- void [set_sensor_function](#) (double(*fn_ptr)(void))
- void [set_sensor_reset](#) (void(*fn_ptr)(void))

5.36.1 Detailed Description

```
template<typename T>
```

```
class Lift< T >
```

LIFT A general class for lifts (e.g. 4bar, dr4bar, linear, etc) Uses a [PID](#) to hold the lift at a certain height under load, and to move the lift to different heights

Author

Ryan McGee

5.36.2 Constructor & Destructor Documentation

Lift()

```
template<typename T>
Lift< T >::Lift (
    motor_group & lift_motors,
    lift_cfg_t & lift_cfg,
    map< T, double > & setpoint_map,
    limit * homing_switch = NULL) [inline]
```

Construct the [Lift](#) object and begin the background task that controls the lift.

Generated by Doxygen

Usage example: /code{.cpp} enum Positions {UP, MID, DOWN}; map<Positions, double> setpt_map { {DOWN, 0.0}, {MID, 0.5}, {UP, 1.0} }; [Lift<Positions>](#) my_lift(motors, lift_cfg, setpt_map); /endcode

Parameters

<i>lift_motors</i>	A set of motors, all set that positive rotation correlates with the lift going up
<i>lift_cfg</i>	Lift characterization information; PID tunings and movement speeds
<i>setpoint_map</i>	A map of enum type T, in which each enum entry corresponds to a different lift height

5.36.3 Member Function Documentation**control_continuous()**

```
template<typename T>
void Lift< T >::control_continuous (
    bool up_ctrl,
    bool down_ctrl) [inline]
```

Control the lift with an "up" button and a "down" button. Use [PID](#) to hold the lift when letting go.

Parameters

<i>up_ctrl</i>	Button controlling the "UP" motion
<i>down_ctrl</i>	Button controlling the "DOWN" motion

control_manual()

```
template<typename T>
void Lift< T >::control_manual (
    bool up_btn,
    bool down_btn,
    int volt_up,
    int volt_down) [inline]
```

Control the lift with manual controls (no holding voltage)

Parameters

<i>up_btn</i>	Raise the lift when true
<i>down_btn</i>	Lower the lift when true
<i>volt_up</i>	Motor voltage when raising the lift
<i>volt_down</i>	Motor voltage when lowering the lift

control_setpoints()

```
template<typename T>
void Lift< T >::control_setpoints (
    bool up_step,
    bool down_step,
    vector< T > pos_list) [inline]
```

Control the lift in "steps". When the "up" button is pressed, the lift will go to the next position as defined by `pos_list`. Order matters!

Parameters

<i>up_step</i>	A button that increments the position of the lift.
<i>down_step</i>	A button that decrements the position of the lift.
<i>pos_list</i>	A list of positions for the lift to go through. The higher the index, the higher the lift should be (generally).

get_async()

```
template<typename T>
bool Lift< T >::get_async () [inline]
```

Returns

whether or not the background thread is running the lift

get_setpoint()

```
template<typename T>
double Lift< T >::get_setpoint () [inline]
```

Returns

The current setpoint for the lift

hold()

```
template<typename T>
void Lift< T >::hold () [inline]
```

Target the class's setpoint. Calculate the [PID](#) output and set the lift motors accordingly.

home()

```
template<typename T>
void Lift< T >::home () [inline]
```

A blocking function that automatically homes the lift based on a sensor or hard stop, and sets the position to 0. A watchdog times out after 3 seconds, to avoid damage.

set_async()

```
template<typename T>
void Lift< T >::set_async (
    bool val) [inline]
```

Enables or disables the background task. Note that running the control functions, or set_position functions will immediately re-enable the task for autonomous use.

Parameters

<i>val</i>	Whether or not the background thread should run the lift
------------	--

set_position()

```
template<typename T>
bool Lift< T >::set_position (
    T pos) [inline]
```

Enable the background task, and send the lift to a position, specified by the setpoint map from the constructor.

Parameters

<i>pos</i>	A lift position enum type
------------	---------------------------

Returns

True if the pid has reached the setpoint

set_sensor_function()

```
template<typename T>
void Lift< T >::set_sensor_function (
    double(* fn_ptr ) (void)) [inline]
```

Creates a custom hook for any other type of sensor to be used on the lift. Example: `/code{.cpp} my_lift.set_↵ sensor_function( [](){return my_sensor.position();} );/endcode`

Parameters

<i>fn_ptr</i>	Pointer to custom sensor function
---------------	-----------------------------------

set_sensor_reset()

```
template<typename T>
void Lift< T >::set_sensor_reset (
    void(* fn_ptr ) (void)) [inline]
```

Creates a custom hook to reset the sensor used in [set_sensor_function\(\)](#). Example: `/code{.cpp} my_lift.set_↵ sensor_reset( my_sensor.resetPosition );/endcode`

set_setpoint()

```
template<typename T>
bool Lift< T >::set_setpoint (
    double val) [inline]
```

Manually set a setpoint value for the lift [PID](#) to go to.

Parameters

<i>val</i>	Lift setpoint, in motor revolutions or sensor units defined by <code>get_sensor</code> . Cannot be outside the softstops.
------------	---

Returns

True if the pid has reached the setpoint

The documentation for this class was generated from the following file:

- `lift.h`

5.37 Lift< T >::lift_cfg_t Struct Reference

```
#include <lift.h>
```

5.37.1 Detailed Description

```
template<typename T>
struct Lift< T >::lift_cfg_t
```

[lift_cfg_t](#) holds the physical parameter specifications of a lify system. includes:

- maximum speeds for the system
- softstops to stop the lift from hitting the hard stops too hard

The documentation for this struct was generated from the following file:

- `lift.h`

5.38 LinearPlantInversionFeedforward< STATES, INPUTS > Class Template Reference

```
#include <linear_plant_inversion_feedforward.h>
```

Public Member Functions

- `template<int OUTPUTS>`
[LinearPlantInversionFeedforward](#) ([LinearSystem](#)< STATES, INPUTS, OUTPUTS > &plant, const double &dt)
- [LinearPlantInversionFeedforward](#) (const EMat< STATES, STATES > &A, const EMat< STATES, INPUTS > &B, const double &dt)
- EVec< INPUTS > [calculate](#) (const EVec< STATES > &r, const EVec< STATES > &next_r)
- EVec< INPUTS > [calculate](#) (const EVec< STATES > &next_r)
- EVec< INPUTS > [calculate](#) (const EVec< STATES > &r, const EVec< STATES > &next_r, const double &dt)
- EVec< INPUTS > [calculate](#) (const EVec< STATES > &next_r, const double &dt)
- void [reset](#) (const EVec< STATES > &initial_state)
- void [reset](#) ()
- void [set_r](#) (const EVec< STATES > &r)

5.38.1 Detailed Description

```
template<int STATES, int INPUTS>
class LinearPlantInversionFeedforward< STATES, INPUTS >
```

This class computes a feedforward control input by inverting the discrete plant dynamics. A continuous linear system is provided, it is then discretized on some timestep, then the feedforward control input is computed to satisfy:

$$B_d * u_{ff} = next_state - A_d * current_state$$

5.38.2 Constructor & Destructor Documentation

LinearPlantInversionFeedforward() [1/2]

```
template<int STATES, int INPUTS>
template<int OUTPUTS>
LinearPlantInversionFeedforward< STATES, INPUTS >::LinearPlantInversionFeedforward (
    LinearSystem< STATES, INPUTS, OUTPUTS > & plant,
    const double & dt) [inline]
```

Constructs a feedforward given a plant and the nominal timestep.

Template Parameters

<i>OUTPUTS</i>	The number of outputs of the plant.
----------------	-------------------------------------

Parameters

<i>plant</i>	The linear system.
<i>dt</i>	The nominal timestep in seconds.

LinearPlantInversionFeedforward() [2/2]

```
template<int STATES, int INPUTS>
LinearPlantInversionFeedforward< STATES, INPUTS >::LinearPlantInversionFeedforward (
    const EMat< STATES, STATES > & A,
    const EMat< STATES, INPUTS > & B,
    const double & dt) [inline]
```

Constructs a feedforward given the state and input matrices from a plant.

Parameters

<i>A</i>	The state matrix of the linear system.
<i>B</i>	The input matrix of the linear system.
<i>dt</i>	The nominal timestep in seconds.

5.38.3 Member Function Documentation

calculate() [1/4]

```
template<int STATES, int INPUTS>
EVec< INPUTS > LinearPlantInversionFeedforward< STATES, INPUTS >::calculate (
    const EVec< STATES > & next_r) [inline]
```

Computes the feedforward control input given only the next reference state. This assumes that the previous reference is already set.

Parameters

<i>next_ _r</i>	The next reference state.
---------------------	---------------------------

calculate() [2/4]

```
template<int STATES, int INPUTS>
EVec< INPUTS > LinearPlantInversionFeedforward< STATES, INPUTS >::calculate (
    const EVec< STATES > & next_r,
    const double & dt) [inline]
```

Computes the feedforward control input given only the next reference state. This assumes that the previous reference is already set.

This is slower because it discretizes A and B on each run, requiring computing a matrix exponential. Don't use this unless you have to.

Parameters

<i>next_ _r</i>	The next reference state.
<i>dt</i>	The timestep for this run.

calculate() [3/4]

```
template<int STATES, int INPUTS>
EVec< INPUTS > LinearPlantInversionFeedforward< STATES, INPUTS >::calculate (
    const EVec< STATES > & r,
    const EVec< STATES > & next_r) [inline]
```

Computes the feedforward control input given the current reference state and the next reference state. This also sets the current reference state to the next reference state for you.

Parameters

<i>r</i>	The current reference state.
----------	------------------------------

<i>next</i> _{$\leftarrow$} <i>_r</i>	The next reference state.
---	---------------------------

calculate() [4/4]

```
template<int STATES, int INPUTS>
EVec< INPUTS > LinearPlantInversionFeedforward< STATES, INPUTS >::calculate (
    const EVec< STATES > & r,
    const EVec< STATES > & next_r,
    const double & dt) [inline]
```

Computes the feedforward control input given the current reference state and the next reference state. This also sets the current reference state to the next reference state for you. Use this function if your timestep is not the same between each run.

This is slower because it discretizes A and B on each run, requiring computing a matrix exponential. Don't use this unless you have to.

Parameters

<i>r</i>	The current reference state.
<i>next</i> _{$\leftarrow$} <i>_r</i>	The next reference state.
<i>dt</i>	The timestep for this run.

reset() [1/2]

```
template<int STATES, int INPUTS>
void LinearPlantInversionFeedforward< STATES, INPUTS >::reset () [inline]
```

Resets the reference to all zeros, and the feedforward to zero.

reset() [2/2]

```
template<int STATES, int INPUTS>
void LinearPlantInversionFeedforward< STATES, INPUTS >::reset (
    const EVec< STATES > & initial_state) [inline]
```

Resets the reference to the given state, and the feedforward to zero.

Parameters

<i>initial_state</i>	The state to set the current reference to.
----------------------	--

set_r()

```
template<int STATES, int INPUTS>
void LinearPlantInversionFeedforward< STATES, INPUTS >::set_r (
    const EVec< STATES > & r) [inline]
```

Sets the current reference to a given state.

Parameters

<i>r</i>	The state to set the current reference to.
----------	--

The documentation for this class was generated from the following file:

- linear_plant_inversion_feedforward.h

5.39 LinearQuadraticRegulator< STATES, INPUTS > Class Template Reference

```
#include <linear_quadratic_regulator.h>
```

Public Member Functions

- template<int OUTPUTS>
LinearQuadraticRegulator (LinearSystem< STATES, INPUTS, OUTPUTS > &plant, const VectorX &Qtolerances, const VectorU &Rtolerances, const double &dt)
- LinearQuadraticRegulator (const MatrixA &A, const MatrixB &B, const VectorX &Qtolerances, const VectorU &Rtolerances, const double &dt)
- LinearQuadraticRegulator (const MatrixA &A, const MatrixB &B, const EMat< STATES, STATES > &Q, const EMat< INPUTS, INPUTS > &R, const double &dt)
- VectorU calculate (const VectorX &x, const VectorX &r)
- template<int OUTPUTS>
void latency_compensate (LinearSystem< STATES, INPUTS, OUTPUTS > &plant, const double &dt, const double &input_delay)

5.39.1 Detailed Description

```
template<int STATES, int INPUTS>
class LinearQuadraticRegulator< STATES, INPUTS >
```

Class implements an LQR controller. This finds the optimal gain matrix K where:

$$u = K(r - x)$$

K is optimized to minimize a cost function:

$$\infty$$

$$J = \sum x_k^T Q x_k + u_k^T R u_k \quad k=0$$

Where Q and R are the state and control cost matrices.

Template Parameters

<i>STATES</i>	The number of states in the system.
<i>INPUTS</i>	The number of inputs to the system.

5.39.2 Constructor & Destructor Documentation

LinearQuadraticRegulator() [1/3]

```
template<int STATES, int INPUTS>
template<int OUTPUTS>
LinearQuadraticRegulator< STATES, INPUTS >::LinearQuadraticRegulator (
    LinearSystem< STATES, INPUTS, OUTPUTS > & plant,
    const VectorX & Qtolerances,
    const VectorU & Rtolerances,
    const double & dt) [inline]
```

Constructs an LQR given a plant, a vector of tolerances for the states and inputs, and the timestep in seconds.

Template Parameters

<i>OUTPUTS</i>	The number of outputs of the plant.
----------------	-------------------------------------

Parameters

<i>plant</i>	The linear system to control.
<i>Qtolerances</i>	A vector of tolerances for each state.
<i>Rtolerances</i>	A vector of tolerances for each input.

LinearQuadraticRegulator() [2/3]

```
template<int STATES, int INPUTS>
LinearQuadraticRegulator< STATES, INPUTS >::LinearQuadraticRegulator (
    const MatrixA & A,
    const MatrixB & B,
    const VectorX & Qtolerances,
    const VectorU & Rtolerances,
    const double & dt) [inline]
```

Constructs an LQR given state and input matrices, a vector of tolerances for the states and inputs, and the timestep in seconds.

Parameters

<i>A</i>	The state matrix of the linear system.
<i>B</i>	The input matrix of the linear system.
<i>Qtolerances</i>	A vector of tolerances for each state.
<i>Rtolerances</i>	A vector of tolerances for each input.

LinearQuadraticRegulator() [3/3]

```
template<int STATES, int INPUTS>
LinearQuadraticRegulator< STATES, INPUTS >::LinearQuadraticRegulator (
    const MatrixA & A,
    const MatrixB & B,
    const EMat< STATES, STATES > & Q,
    const EMat< INPUTS, INPUTS > & R,
    const double & dt) [inline]
```

Constructs an LQR given state and input matrices, the cost matrices of states and inputs, and the timestep in seconds.

Parameters

<i>A</i>	The state matrix of the linear system.
<i>B</i>	The input matrix of the linear system.
<i>Q</i>	The cost matrix of the states.
<i>R</i>	The cost matrix of the inputs.

5.39.3 Member Function Documentation**calculate()**

```
template<int STATES, int INPUTS>
VectorU LinearQuadraticRegulator< STATES, INPUTS >::calculate (
    const VectorX & x,
    const VectorX & r) [inline]
```

Computes the control input *u* as:

$$u = K(r - x)$$

Parameters

<i>x</i>	The current state.
<i>r</i>	The reference state.

latency_compensate()

```
template<int STATES, int INPUTS>
template<int OUTPUTS>
void LinearQuadraticRegulator< STATES, INPUTS >::latency_compensate (
    LinearSystem< STATES, INPUTS, OUTPUTS > & plant,
    const double & dt,
    const double & input_delay) [inline]
```

Recomputes *K* to work for a time delayed state.

$$K_{\text{delay}} = K(A - BK)^{(\text{delay} / dt)}$$

Template Parameters

<i>OUTPUTS</i>	The number of outputs of the plant
----------------	------------------------------------

Parameters

<i>plant</i>	The linear system.
<i>dt</i>	The timestep in seconds.
<i>input_delay</i>	The time delay of the system.

The documentation for this class was generated from the following file:

- `linear_quadratic_regulator.h`

5.40 LinearSystem< STATES, INPUTS, OUTPUTS > Class Template Reference

```
#include <linear_system.h>
```

Public Member Functions

- [LinearSystem](#) (const MatrixA &A, const MatrixB &B, const MatrixC &C, const MatrixD &D)
- MatrixA [A](#) ()
- MatrixB [B](#) ()
- const std::tuple< std::tuple< MatrixA, MatrixB > > & [discAB](#) (const double &dt)
- MatrixC [C](#) ()
- MatrixD [D](#) ()
- VectorX [compute_X](#) (const VectorX &x, const VectorU &u, double dt)
- VectorY [compute_Y](#) (const VectorX &x, const VectorU &u)

5.40.1 Detailed Description

```
template<int STATES, int INPUTS, int OUTPUTS>
class LinearSystem< STATES, INPUTS, OUTPUTS >
```

This class represents a state-space model of a linear system.

It contains the following continuous matrices: A, System matrix B, Input matrix C, Output matrix D, Feedthrough matrix

5.40.2 Constructor & Destructor Documentation

LinearSystem()

```
template<int STATES, int INPUTS, int OUTPUTS>
LinearSystem< STATES, INPUTS, OUTPUTS >::LinearSystem (
    const MatrixA & A,
    const MatrixB & B,
    const MatrixC & C,
    const MatrixD & D) [inline]
```

Parameters

<i>A</i>	The continuous system matrix
<i>B</i>	The continuous input matrix
<i>C</i>	The output matrix
<i>D</i>	The feedthrough matrix

5.40.3 Member Function Documentation**A()**

```
template<int STATES, int INPUTS, int OUTPUTS>
MatrixA LinearSystem< STATES, INPUTS, OUTPUTS >::A () [inline]
```

Returns the continuous system matrix A.

B()

```
template<int STATES, int INPUTS, int OUTPUTS>
MatrixB LinearSystem< STATES, INPUTS, OUTPUTS >::B () [inline]
```

Returns the continuous input matrix B.

C()

```
template<int STATES, int INPUTS, int OUTPUTS>
MatrixC LinearSystem< STATES, INPUTS, OUTPUTS >::C () [inline]
```

Returns the output matrix C.

compute_X()

```
template<int STATES, int INPUTS, int OUTPUTS>
VectorX LinearSystem< STATES, INPUTS, OUTPUTS >::compute_X (
    const VectorX & x,
    const VectorU & u,
    double dt) [inline]
```

Computes the new state vector given the previous state vector, an input vector, and the timestep in seconds.

Parameters

<i>x</i>	The current state vector.
<i>u</i>	The input vector.
<i>dt</i>	The timestep in seconds.

Returns

The new state vector.

compute_Y()

```
template<int STATES, int INPUTS, int OUTPUTS>
VectorY LinearSystem< STATES, INPUTS, OUTPUTS >::compute_Y (
    const VectorX & x,
    const VectorU & u) [inline]
```

Computes the output vector given a state and an input.

Parameters

x	The state vector.
u	The input vector.

Returns

The output vector.

D()

```
template<int STATES, int INPUTS, int OUTPUTS>
MatrixD LinearSystem< STATES, INPUTS, OUTPUTS >::D () [inline]
```

Returns the feedthrough matrix D.

discAB()

```
template<int STATES, int INPUTS, int OUTPUTS>
const std::tuple< std::tuple< MatrixA, MatrixB > > & LinearSystem< STATES, INPUTS, OUTPUTS
>::discAB (
    const double & dt) [inline]
```

Returns a tuple of A and B after being discretized.

The documentation for this class was generated from the following file:

- linear_system.h

5.41 Logger Class Reference

Class to simplify writing to files.

```
#include <logger.h>
```

Public Member Functions

- `Logger` (const std::string &filename)
Create a logger that will save to a file.
- `Logger` (const `Logger` &l)=delete
copying not allowed
- `Logger & operator=` (const `Logger` &l)=delete
copying not allowed
- void `Log` (const std::string &s)
Write a string to the log.
- void `Log` (LogLevel level, const std::string &s)
Write a string to the log with a loglevel.
- void `Logln` (const std::string &s)
Write a string and newline to the log.
- void `Logln` (LogLevel level, const std::string &s)
Write a string and a newline to the log with a loglevel.
- void `Logf` (const char *fmt,...)
Write a formatted string to the log.
- void `Logf` (LogLevel level, const char *fmt,...)
Write a formatted string to the log with a loglevel.

Static Public Attributes

- static constexpr int `MAX_FORMAT_LEN` = 512
maximum size for a string to be before it's written

5.41.1 Detailed Description

Class to simplify writing to files.

5.41.2 Constructor & Destructor Documentation

Logger()

```
Logger::Logger (
    const std::string & filename) [explicit]
```

Create a logger that will save to a file.

Parameters

<i>filename</i>	the file to save to
-----------------	---------------------

5.41.3 Member Function Documentation

Log() [1/2]

```
Generated by Doxygen
void Logger::Log (
    const std::string & s)
```

Write a string to the log.

Parameters

<i>s</i>	the string to write
----------	---------------------

Log() [2/2]

```
void Logger::Log (  
    LogLevel level,  
    const std::string & s)
```

Write a string to the log with a loglevel.

Parameters

<i>level</i>	the level to write. DEBUG, NOTICE, WARNING, ERROR, CRITICAL, TIME
<i>s</i>	the string to write

Logf() [1/2]

```
void Logger::Logf (  
    const char * fmt,  
    ...)
```

Write a formatted string to the log.

Parameters

<i>fmt</i>	the format string (like printf)
...	the args

Logf() [2/2]

```
void Logger::Logf (  
    LogLevel level,  
    const char * fmt,  
    ...)
```

Write a formatted string to the log with a loglevel.

Parameters

<i>level</i>	the level to write. DEBUG, NOTICE, WARNING, ERROR, CRITICAL, TIME
<i>fmt</i>	the format string (like printf)
...	the args

Logln() [1/2]

```
void Logger::Logln (  
    const std::string & s)
```

Parameters

s	the string to write
---	---------------------

Logln() [2/2]

```
void Logger::Logln (
    LogLevel level,
    const std::string & s)
```

Write a string and a newline to the log with a loglevel.

Parameters

level	the level to write. DEBUG, NOTICE, WARNING, ERROR, CRITICAL, TIME
s	the string to write

The documentation for this class was generated from the following files:

- logger.h
- logger.cpp

5.42 MotionController::m_profile_cfg_t Struct Reference

```
#include <motion_controller.h>
```

Public Attributes

- double **max_v**
the maximum velocity the robot can drive
- double **accel**
the most acceleration the robot can do
- [PID::pid_config_t](#) **pid_cfg**
configuration parameters for the internal [PID](#) controller
- [FeedForward::ff_config_t](#) **ff_cfg**
configuration parameters for the internal

5.42.1 Detailed Description

m_profile_config holds all data the motion controller uses to plan paths When motion profile is given a target to drive to, max_v and accel are used to make the trapezoid profile instructing the controller how to drive pid_cfg, ff_cfg are used to find the motor outputs necessary to execute this path

The documentation for this struct was generated from the following file:

- motion_controller.h

5.43 MaybeMessage Class Reference

[State Machine](#) :)))))) A fun fun way of controlling stateful subsystems - used in the 2023-2024 Over Under game for our overly complex intake-cata subsystem (see there for an example) The statemachine runs in a background thread and a user thread can interact with it through `current_state` and `send_message`.

```
#include <state_machine.h>
```

Public Member Functions

- **MaybeMessage** ()
Empty message - when theres no message.
- **MaybeMessage** (Message msg)
Create a maybemessage with a message.
- bool **has_message** ()
check if the message is here
- Message **message** ()
Get the message stored. The return value is invalid unless has_message returned true.

5.43.1 Detailed Description

[State Machine](#) :)))))) A fun fun way of controlling stateful subsystems - used in the 2023-2024 Over Under game for our overly complex intake-cata subsystem (see there for an example) The statemachine runs in a background thread and a user thread can interact with it through `current_state` and `send_message`.

Designwise: the System class should hold onto any motors, feedback controllers, etc that are persistent in the system States themselves should hold any data that *only* that state needs. For example if a state should be exited after a certain amount of time, it should hold a timer rather than the System holding that timer. (see Junder from 2024 for an example of this design)

Template Parameters

<i>System</i>	The system that this is the base class of <code>class Thing : public StateMachine<Thing></code> @tparam IDType The ID enum that recognizes states. Hint hint, use an <code>enum class</code>
<i>Message</i>	the message enum that a state or an outside can send and that states respond to
<i>delay_ms</i>	the delay to wait between each state processing to allow other threads to work
<i>do_log</i>	true if you want print statements describing incoming messages and current states. If true, it is expected that IDType and Message have a function called <code>to_string</code> that takes them as its only parameter and returns a <code>std::string</code> <code>*/ template <typename System, typename IDType, typename Message, int32_t delay_ms, bool do_log = false> class StateMachine { static_assert(std::is_enum<Message>::value, "Message should be an enum (it's easier that way)"); static_assert(std::is_enum<IDType>::value, "IDType should be an enum (it's easier that way)");</code>

```
public: /**
```

```
MaybeMessage a message of Message type or nothing MaybeMessage m = {}; // empty MaybeMessage m = Message::EnumField1
```

5.43.2 Constructor & Destructor Documentation

MaybeMessage()

Parameters

<i>msg</i>	the message to hold on to
------------	---------------------------

5.43.3 Member Function Documentation**has_message()**

```
bool MaybeMessage::has_message () [inline]
```

check if the message is here

Returns

true if there is a message

message()

```
Message MaybeMessage::message () [inline]
```

Get the message stored. The return value is invalid unless has_message returned true.

Returns

The message if it exists. Undefined otherwise

The documentation for this class was generated from the following file:

- state_machine.h

5.44 MecanumDrive Class Reference

```
#include <mecanum_drive.h>
```

Classes

- struct [mecanumdrive_config_t](#)

Public Member Functions

- [MecanumDrive](#) (vex::motor &left_front, vex::motor &right_front, vex::motor &left_rear, vex::motor &right_rear, vex::rotation *lateral_wheel=NULL, vex::inertial *imu=NULL, [mecanumdrive_config_t](#) *config=NULL)
- void [drive_raw](#) (double direction_deg, double magnitude, double rotation)
- void [drive](#) (double left_y, double left_x, double right_x, int power=2)
- bool [auto_drive](#) (double inches, double direction, double speed, bool gyro_correction=true)
- bool [auto_turn](#) (double degrees, double speed, bool ignore_imu=false)

5.44.1 Detailed Description

A class representing the Mecanum drivetrain. Contains 4 motors, a possible IMU (intertial), and a possible undriven perpendicular wheel.

5.44.2 Constructor & Destructor Documentation

MecanumDrive()

```
MecanumDrive::MecanumDrive (
    vex::motor & left_front,
    vex::motor & right_front,
    vex::motor & left_rear,
    vex::motor & right_rear,
    vex::rotation * lateral_wheel = NULL,
    vex::inertial * imu = NULL,
    mecanumdrive_config_t * config = NULL)
```

Create the Mecanum drivetrain object

5.44.3 Member Function Documentation

auto_drive()

```
bool MecanumDrive::auto_drive (
    double inches,
    double direction,
    double speed,
    bool gyro_correction = true)
```

Drive the robot in a straight line automatically. If the inertial was declared in the constructor, use it to correct while driving. If the lateral wheel was declared in the constructor, use it for more accurate positioning while strafing.

Parameters

<i>inches</i>	How far the robot should drive, in inches
<i>direction</i>	What direction the robot should travel in, in degrees. 0 is forward, +/-180 is reverse, clockwise is positive.
<i>speed</i>	The maximum speed the robot should travel, in percent: -1.0->+1.0
<i>gyro_correction</i>	=true Whether or not to use the gyro to help correct while driving. Will always be false if no gyro was declared in the constructor.

Drive the robot in a straight line automatically. If the inertial was declared in the constructor, use it to correct while driving. If the lateral wheel was declared in the constructor, use it for more accurate positioning while strafing.

Parameters

<i>inches</i>	How far the robot should drive, in inches
---------------	---

<i>direction</i>	What direction the robot should travel in, in degrees. 0 is forward, +/-180 is reverse, clockwise is positive.
<i>speed</i>	The maximum speed the robot should travel, in percent: -1.0->+1.0
<i>gyro_correction</i>	= true Whether or not to use the gyro to help correct while driving. Will always be false if no gyro was declared in the constructor.

Returns

Whether or not the maneuver is complete.

auto_turn()

```
bool MecanumDrive::auto_turn (
    double degrees,
    double speed,
    bool ignore_imu = false)
```

Autonomously turn the robot X degrees over it's center point. Uses a closed loop for control.

Parameters

<i>degrees</i>	How many degrees to rotate the robot. Clockwise postive.
<i>speed</i>	What percentage to run the motors at: 0.0 -> 1.0
<i>ignore_imu</i>	=false Whether or not to use the Inertial for determining angle. Will instead use circumference formula + robot's wheelbase + encoders to determine.

Returns

whether or not the robot has finished the maneuver

Autonomously turn the robot X degrees over it's center point. Uses a closed loop for control.

Parameters

<i>degrees</i>	How many degrees to rotate the robot. Clockwise postive.
<i>speed</i>	What percentage to run the motors at: 0.0 -> 1.0
<i>ignore_imu</i>	= false Whether or not to use the Inertial for determining angle. Will instead use circumference formula + robot's wheelbase + encoders to determine.

Returns

whether or not the robot has finished the maneuver

drive()

```
void MecanumDrive::drive (
    double left_y,
    double left_x,
    double right_x,
    int power = 2)
```

Parameters

<i>left_y</i>	left joystick, Y axis (forward / backwards)
<i>left_x</i>	left joystick, X axis (strafe left / right)
<i>right↔ _x</i>	right joystick, X axis (rotation left / right)
<i>power</i>	=2 how much of a "curve" there should be on drive controls; better for low speed maneuvers. Leave blank for a default curve of 2 (higher means more fidelity)

Drive the robot with a mecanum-style / arcade drive. Inputs are in percent (-100.0 -> 100.0) straight from the controller. Controls are mixed, so the robot can drive forward / strafe / rotate all at the same time.

Parameters

<i>left_y</i>	left joystick, Y axis (forward / backwards)
<i>left_x</i>	left joystick, X axis (strafe left / right)
<i>right↔ _x</i>	right joystick, X axis (rotation left / right)
<i>power</i>	= 2 how much of a "curve" there should be on drive controls; better for low speed maneuvers. Leave blank for a default curve of 2 (higher means more fidelity)

drive_raw()

```
void MecanumDrive::drive_raw (
    double direction_deg,
    double magnitude,
    double rotation)
```

Drive the robot using vectors. This handles all the math required for mecanum control.

Parameters

<i>direction_deg</i>	the direction to drive the robot, in degrees. 0 is forward, 180 is back, clockwise is positive, counterclockwise is negative.
<i>magnitude</i>	How fast the robot should drive, in percent: 0.0->1.0
<i>rotation</i>	How fast the robot should rotate, in percent: -1.0->+1.0

The documentation for this class was generated from the following files:

- mecanum_drive.h
- mecanum_drive.cpp

5.45 MecanumDrive::mecanumdrive_config_t Struct Reference

```
#include <mecanum_drive.h>
```

5.45.1 Detailed Description

Configure the Mecanum drive [PID](#) tunings and robot configurations

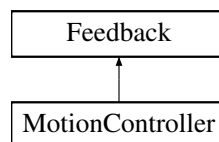
The documentation for this struct was generated from the following file:

- `mecanum_drive.h`

5.46 MotionController Class Reference

```
#include <motion_controller.h>
```

Inheritance diagram for MotionController:



Classes

- struct [m_profile_cfg_t](#)

Public Member Functions

- [MotionController](#) ([m_profile_cfg_t](#) &config)
Construct a new Motion Controller object.
- void [init](#) (double start_pt, double end_pt) override
Initialize the motion profile for a new movement This will also reset the [PID](#) and profile timers.
- double [update](#) (double sensor_val) override
Update the motion profile with a new sensor value.
- double [get](#) () override
- void [set_limits](#) (double lower, double upper) override
- bool [is_on_target](#) () override
- motion_t [get_motion](#) () const

Static Public Member Functions

- static [FeedForward::ff_config_t](#) [tune_feedforward](#) ([TankDrive](#) &drive, [OdometryTank](#) &odometry, double pct=0.6, double duration=2)

5.46.1 Detailed Description

Motion Controller class

This class defines a top-level motion profile, which can act as an intermediate between a subsystem class and the motors themselves

This takes the constants kS, kV, kA, kP, kI, kD, max_v and acceleration and wraps around a feedforward, [PID](#) and trapezoid profile. It does so with the following formula:

```
out = feedforward.calculate(motion_profile.get(time_s)) + pid.get(motion_profile.get(time_s))
```

For [PID](#) and Feedforward specific formulae, see [pid.h](#), [feedforward.h](#), and [trapezoid_profile.h](#)

Author

Ryan McGee

Date

7/13/2022

5.46.2 Constructor & Destructor Documentation

MotionController()

```
MotionController::MotionController (  
    m_profile_cfg_t & config)
```

Construct a new Motion Controller object.

Parameters

<i>config</i>	The definition of how the robot is able to move max_v Maximum velocity the movement is capable of accel Acceleration / deceleration of the movement pid_cfg Definitions of kP, kI, and kD ff_cfg Definitions of kS, kV, and kA
---------------	--

5.46.3 Member Function Documentation

get()

```
double MotionController::get () [override], [virtual]
```

Returns

the last saved result from the feedback controller

Implements [Feedback](#).

get_motion()

```
motion_t MotionController::get_motion () const
```

Returns

The current position, velocity and acceleration setpoints

init()

```
void MotionController::init (
    double start_pt,
    double end_pt) [override], [virtual]
```

Initialize the motion profile for a new movement This will also reset the [PID](#) and profile timers.

Parameters

<i>start_pt</i>	Movement starting position
<i>end_pt</i>	Movement ending position

Implements [Feedback](#).

is_on_target()

```
bool MotionController::is_on_target () [override], [virtual]
```

Returns

Whether or not the movement has finished, and the [PID](#) confirms it is on target

Implements [Feedback](#).

set_limits()

```
void MotionController::set_limits (
    double lower,
    double upper) [override], [virtual]
```

Clamp the upper and lower limits of the output. If both are 0, no limits should be applied. if limits are applied, the controller will not target any value below lower or above upper

Parameters

<i>lower</i>	upper limit
--------------	-------------

<i>upper</i>	lower limit
--------------	-------------

Clamp the upper and lower limits of the output. If both are 0, no limits should be applied.

Parameters

<i>lower</i>	Upper limit
<i>upper</i>	Lower limit

Implements [Feedback](#).

tune_feedforward()

```
FeedForward::ff_config_t MotionController::tune_feedforward (
    TankDrive & drive,
    OdometryTank & odometry,
    double pct = 0.6,
    double duration = 2) [static]
```

This method attempts to characterize the robot's drivetrain and automatically tune the feedforward. It does this by first calculating the kS (voltage to overcome static friction) by slowly increasing the voltage until it moves.

Next is kV (voltage to sustain a certain velocity), where the robot will record it's steady-state velocity at 'pct' speed.

Finally, kA (voltage needed to accelerate by a certain rate), where the robot will record the entire movement's velocity and acceleration, record a plot of $[X=(pct-kV*V-kS), Y=(Acceleration)]$ along the movement, and since $kA*Accel = pct-kV*V-kS$, the reciprocal of the linear regression is the kA value.

Parameters

<i>drive</i>	The tankdrive to operate on
<i>odometry</i>	The robot's odometry subsystem
<i>pct</i>	Maximum velocity in percent (0->1.0)
<i>duration</i>	Amount of time the robot should be moving for the test

Returns

A tuned feedforward object

update()

```
double MotionController::update (
    double sensor_val) [override], [virtual]
```

Update the motion profile with a new sensor value.

Parameters

<i>sensor_val</i>	Value from the sensor
-------------------	-----------------------

Returns

the motor input generated from the motion profile

Implements [Feedback](#).

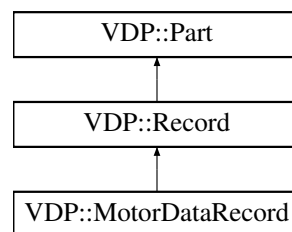
The documentation for this class was generated from the following files:

- motion_controller.h
- motion_controller.cpp

5.47 VDP::MotorDataRecord Class Reference

```
#include <builtins.hpp>
```

Inheritance diagram for VDP::MotorDataRecord:

**Public Member Functions**

- [MotorDataRecord](#) (std::string name, vex::motor &mot)
- void [fetch](#) () override

Public Member Functions inherited from [VDP::Record](#)

- [Record](#) (std::string name)
- [Record](#) (std::string name, const std::vector< [Part](#) * > &parts)
- [Record](#) (std::string name, std::vector< PartPtr > parts)
- [Record](#) (std::string name, PacketReader &reader)
- void [set_fields](#) (std::vector< PartPtr > fields)
- void [fetch](#) () override
- void [read_data_from_message](#) (PacketReader &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Additional Inherited Members

Protected Member Functions inherited from [VDP::Record](#)

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.47.1 Detailed Description

Defines a record that holds motor values

5.47.2 Constructor & Destructor Documentation

MotorDataRecord()

```
VDP::MotorDataRecord::MotorDataRecord (
    std::string name,
    vex::motor & motor)
```

Creates a record that contains a Float of the motor position Float of the motor velocity Float of the motor temperature
Float of the motor voltage Float of the motor current

Parameters

<i>name</i>	the name of the record to create
<i>mot</i>	the motor to get data from

5.47.3 Member Function Documentation

fetch()

```
void VDP::MotorDataRecord::fetch () [override], [virtual]
```

sets the data that the Motor Parts hold

Implements [VDP::Part](#).

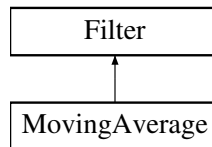
The documentation for this class was generated from the following files:

- builtins.hpp
- builtins.cpp

5.48 MovingAverage Class Reference

```
#include <moving_average.h>
```

Inheritance diagram for MovingAverage:



Public Member Functions

- [MovingAverage](#) (int buffer_size)
- [MovingAverage](#) (int buffer_size, double starting_value)
- void [add_entry](#) (double n) override
- double [get_value](#) () const override
- int [get_size](#) () const

5.48.1 Detailed Description

[MovingAverage](#)

A moving average is a way of smoothing out noisy data. For many sensor readings, the noise is roughly symmetric around the actual value. This means that if you collect enough samples those that are too high are cancelled out by the samples that are too low leaving the real value.

The [MovingAverage](#) class provides a simple interface to do this smoothing from our noisy sensor values.

WARNING: because we need a lot of samples to get the actual value, the value given by the [MovingAverage](#) will 'lag' behind the actual value that the sensor is reading. Using a [MovingAverage](#) is thus a tradeoff between accuracy and lag time (more samples) vs. less accuracy and faster updating (less samples).

5.48.2 Constructor & Destructor Documentation

[MovingAverage\(\)](#) [1/2]

```
MovingAverage::MovingAverage (
    int buffer_size)
```

Create a moving average calculator with 0 as the default value

Parameters

<i>buffer_size</i>	The size of the buffer. The number of samples that constitute a valid reading
--------------------	---

[MovingAverage\(\)](#) [2/2]

Generated by Doxygen

```
MovingAverage::MovingAverage (
    int buffer_size,
    double starting_value)
```

Parameters

<i>buffer_size</i>	The size of the buffer. The number of samples that constitute a valid reading
<i>starting_value</i>	The value that the average will be before any data is added

5.48.3 Member Function Documentation**add_entry()**

```
void MovingAverage::add_entry (  
    double n) [override], [virtual]
```

Add a reading to the buffer Before: [1 1 2 2 3 3] => 2 ^ After: [2 1 2 2 3 3] => 2.16 ^

Parameters

<i>n</i>	the sample that will be added to the moving average.
----------	--

Implements [Filter](#).

get_size()

```
int MovingAverage::get_size () const
```

How many samples the average is made from

Returns

the number of samples used to calculate this average

get_value()

```
double MovingAverage::get_value () const [override], [virtual]
```

Returns the average based off of all the samples collected so far

Returns

the calculated average. $\text{sum}(\text{samples})/\text{numsamples}$

How many samples the average is made from

Returns

the number of samples used to calculate this average

Implements [Filter](#).

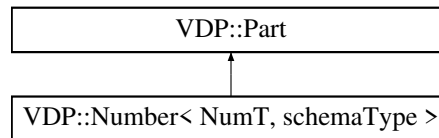
The documentation for this class was generated from the following files:

- moving_average.h
- moving_average.cpp

5.49 VDP::Number< NumT, schemaType > Class Template Reference

```
#include <types.hpp>
```

Inheritance diagram for VDP::Number< NumT, schemaType >:



Public Types

- using [FetchFunc](#) = std::function<NumberType()>

Public Member Functions

- [Number](#) (std::string field_name, [FetchFunc](#) fetcher=[]) { return(NumberType) 0;}
- void [fetch](#) () override
- void [set_value](#) (NumberType val)
- NumberType [get_value](#) ()
- void [pprint](#) (std::stringstream &ss, size_t indent) const override
- void [pprint_data](#) (std::stringstream &ss, size_t indent) const override
- void [read_data_from_message](#) (PacketReader &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Protected Member Functions

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.49.1 Detailed Description

```
template<typename NumT, Type schemaType>
class VDP::Number< NumT, schemaType >
```

A number conveyed as a part

5.49.2 Member Typedef Documentation

FetchFunc

```
template<typename NumT, Type schemaType>
using VDP::Number< NumT, schemaType >::FetchFunc = std::function<NumberType()>
```

Function to run when fetching this number

5.49.3 Constructor & Destructor Documentation

Number()

```
template<typename NumT, Type schemaType>
VDP::Number< NumT, schemaType >::Number (
    std::string field_name,
    FetchFunc fetcher = []() { return (NumberType)0; }) [inline], [explicit]
```

creates a number with a name and fetcher

Parameters

<i>field</i>	name for the number part
<i>fetcher</i>	the function to run when fetching this number

5.49.4 Member Function Documentation

fetch()

```
template<typename NumT, Type schemaType>
void VDP::Number< NumT, schemaType >::fetch () [inline], [override], [virtual]
```

sets the value of the number stored to the value returned by its fetcher

Implements [VDP::Part](#).

get_value()

```
template<typename NumT, Type schemaType>
NumberType VDP::Number< NumT, schemaType >::get_value () [inline]
```

Returns

the currently stored number value

pprint()

```
template<typename NumT, Type schemaType>
void VDP::Number< NumT, schemaType >::pprint (
    std::stringstream & ss,
    size_t indent) const [inline], [override], [virtual]
```

Parameters

<i>ss</i>	the stream of strings to print to
<i>indent</i>	the amount of indents to use

Implements [VDP::Part](#).

pprint_data()

```
template<typename NumT, Type schemaType>
void VDP::Number< NumT, schemaType >::pprint_data (
    std::stringstream & ss,
    size_t indent) const [inline], [override], [virtual]
```

prints the data the number holds with the format "[indent]name: value"

Parameters

<i>ss</i>	the stream of strings to print to
<i>indent</i>	the amount of indents to use

Implements [VDP::Part](#).

read_data_from_message()

```
template<typename NumT, Type schemaType>
void VDP::Number< NumT, schemaType >::read_data_from_message (
    PacketReader & reader) [inline], [override], [virtual]
```

sets the value of the number stored to the value read by a PacketReader

Parameters

<i>reader</i>	the packet reader to get the number from
---------------	--

Implements [VDP::Part](#).

set_value()

```
template<typename NumT, Type schemaType>
void VDP::Number< NumT, schemaType >::set_value (
    NumberType val) [inline]
```

sets the value of the number stored

Parameters

<i>val</i>	the value to store
------------	--------------------

write_message()

```
template<typename NumT, Type schemaType>
void VDP::Number< NumT, schemaType >::write_message (
    PacketWriter & sofar) const [inline], [override], [protected], [virtual]
```

writes the number's data to a packet

Parameters

<i>sofar</i>	the packet writer to write with
--------------	---------------------------------

Implements [VDP::Part](#).

write_schema()

```
template<typename NumT, Type schemaType>
void VDP::Number< NumT, schemaType >::write_schema (
    PacketWriter & sofar) const [inline], [override], [protected], [virtual]
```

writes the number's schematic to a packet

Parameters

<i>sofar</i>	the packet writer to write with
--------------	---------------------------------

Implements [VDP::Part](#).

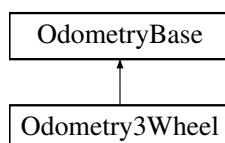
The documentation for this class was generated from the following file:

- types.hpp

5.50 Odometry3Wheel Class Reference

```
#include <odometry_3wheel.h>
```

Inheritance diagram for Odometry3Wheel:



Classes

- struct [odometry3wheel_cfg_t](#)

Public Member Functions

- [Odometry3Wheel](#) ([CustomEncoder](#) &lside_fwd, [CustomEncoder](#) &rside_fwd, [CustomEncoder](#) &off_axis, [odometry3wheel_cfg_t](#) &cfg, bool is_async=true)
- [Pose2d update](#) () override
- void [tune](#) (vex::controller &con, [TankDrive](#) &drive)

Public Member Functions inherited from [OdometryBase](#)

- [OdometryBase](#) (bool is_async)
- virtual [Pose2d get_position](#) (void)
- virtual void [set_position](#) (const [Pose2d](#) &newpos=zero_pos)
- void [end_async](#) ()
- virtual double [get_speed](#) ()
- virtual double [get_accel](#) ()
- double [get_angular_speed_deg](#) ()
- double [get_angular_accel_deg](#) ()

Additional Inherited Members

Static Public Member Functions inherited from [OdometryBase](#)

- static int [background_task](#) (void *ptr)
- static double [smallest_angle](#) (double start_deg, double end_deg)

Public Attributes inherited from [OdometryBase](#)

- bool **end_task** = false
end_task is true if we instruct the odometry thread to shut down
- vex::task * [handle](#)
- vex::mutex [mut](#)
- [Pose2d](#) [current_pos](#)
- double [speed](#)
- double [accel](#)
- double [ang_speed_deg](#)
- double [ang_accel_deg](#)

5.50.1 Detailed Description

Odometry3Wheel

This class handles the code for a standard 3-pod odometry setup, where there are 3 "pods" made up of undriven (dead) wheels connected to encoders in the following configuration:

+Y ----- ^ ||||| ||| O ||||| ||| == || ----- | +-----> + X

Where O is the center of rotation. The robot will monitor the changes in rotation of these wheels and calculate the robot's X, Y and rotation on the field.

This is a "set and forget" class, meaning once the object is created, the robot will immediately begin tracking it's movement in the background.

Author

Ryan McGee

Date

Oct 31 2022

5.50.2 Constructor & Destructor Documentation

Odometry3Wheel()

```
Odometry3Wheel::Odometry3Wheel (
    CustomEncoder & lside_fwd,
    CustomEncoder & rside_fwd,
    CustomEncoder & off_axis,
    odometry3wheel_cfg_t & cfg,
    bool is_async = true)
```

Construct a new Odometry 3 Wheel object

Parameters

<i>lside_fwd</i>	left-side encoder reference
<i>rside_fwd</i>	right-side encoder reference
<i>off_axis</i>	off-axis (perpendicular) encoder reference
<i>cfg</i>	robot odometry configuration
<i>is_async</i>	true to constantly run in the background

5.50.3 Member Function Documentation

tune()

```
void Odometry3Wheel::tune (
    vex::controller & con,
    TankDrive & drive)
```

Generated by Doxygen

A guided tuning process to automatically find tuning parameters. This method is blocking, and returns when tuning

Parameters

<i>con</i>	Controller reference, for screen and button control
<i>drive</i>	Drivetrain reference for robot control

A guided tuning process to automatically find tuning parameters. This method is blocking, and returns when tuning has finished. Follow the instructions on the controller to complete the tuning process

It is assumed the gear ratio and encoder PPR have been set correctly

update()

`Pose2d` Odometry3Wheel::update () [override], [virtual]

Update the current position of the robot once, using the current state of the encoders and the previous known location

Returns

the robot's updated position

Implements [OdometryBase](#).

The documentation for this class was generated from the following files:

- odometry_3wheel.h
- odometry_3wheel.cpp

5.51 Odometry3Wheel::odometry3wheel_cfg_t Struct Reference

```
#include <odometry_3wheel.h>
```

Public Attributes

- double [wheelbase_dist](#)
- double [off_axis_center_dist](#)
- double [wheel_diam](#)

5.51.1 Detailed Description

[odometry3wheel_cfg_t](#) holds all the specifications for how to calculate position with 3 encoders See the core wiki for what exactly each of these parameters measures

5.51.2 Member Data Documentation**off_axis_center_dist**

```
double Odometry3Wheel::odometry3wheel_cfg_t::off_axis_center_dist
```

distance from the center of the robot to the center off axis wheel

wheel_diam

```
double Odometry3Wheel::odometry3wheel_cfg_t::wheel_diam
```

the diameter of the tracking wheel

wheelbase_dist

```
double Odometry3Wheel::odometry3wheel_cfg_t::wheelbase_dist
```

distance from the center of the left wheel to the center of the right wheel

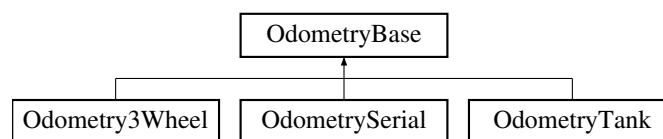
The documentation for this struct was generated from the following file:

- `odometry_3wheel.h`

5.52 OdometryBase Class Reference

```
#include <odometry_base.h>
```

Inheritance diagram for OdometryBase:

**Public Member Functions**

- [OdometryBase](#) (bool is_async)
- virtual [Pose2d get_position](#) (void)
- virtual void [set_position](#) (const [Pose2d](#) &newpos=zero_pos)
- virtual [Pose2d update](#) ()=0
- void [end_async](#) ()
- virtual double [get_speed](#) ()
- virtual double [get_accel](#) ()
- double [get_angular_speed_deg](#) ()
- double [get_angular_accel_deg](#) ()

Static Public Member Functions

- static int [background_task](#) (void *ptr)
- static double [smallest_angle](#) (double start_deg, double end_deg)

Public Attributes

- bool **end_task** = false
end_task is true if we instruct the odometry thread to shut down
- vex::task * **handle**
- vex::mutex **mut**
- Pose2d **current_pos**
- double **speed**
- double **accel**
- double **ang_speed_deg**
- double **ang_accel_deg**

5.52.1 Detailed Description

OdometryBase

This base class contains all the shared code between different implementations of odometry. It handles the asynchronous management, position input/output and basic math functions, and holds positional types specific to field orientation.

All future odometry implementations should extend this file and redefine [update\(\)](#) function.

Author

Ryan McGee

Date

Aug 11 2021

5.52.2 Constructor & Destructor Documentation

OdometryBase()

```
OdometryBase::OdometryBase (
    bool is_async)
```

Construct a new Odometry Base object

Parameters

<i>is_async</i>	True to run constantly in the background, false to call update() manually
-----------------	---

5.52.3 Member Function Documentation

background_task()

```
int OdometryBase::background_task (
    void * ptr) [static]
```

Generated by Doxygen

Function that runs in the background task. This function pointer is passed to the vex::task constructor.

Parameters

<i>ptr</i>	Pointer to OdometryBase object
------------	--

Returns

Required integer return code. Unused.

end_async()

```
void OdometryBase::end_async ()
```

End the background task. Cannot be restarted. If the user wants to end the thread but keep the data up to date, they must run the [update\(\)](#) function manually from then on.

get_accel()

```
double OdometryBase::get_accel () [virtual]
```

Get the current acceleration

Returns

the acceleration rate of the robot (inch/s²)

get_angular_accel_deg()

```
double OdometryBase::get_angular_accel_deg ()
```

Get the current angular acceleration in degrees

Returns

the angular acceleration at which we are turning (deg/s²)

get_angular_speed_deg()

```
double OdometryBase::get_angular_speed_deg ()
```

Get the current angular speed in degrees

Returns

the angular velocity at which we are turning (deg/s)

get_position()

```
Pose2d OdometryBase::get_position (
    void ) [virtual]
```

Gets the current position and rotation

Returns

the position that the odometry believes the robot is at

Gets the current position and rotation

Reimplemented in [OdometrySerial](#).

get_speed()

```
double OdometryBase::get_speed () [virtual]
```

Get the current speed

Returns

the speed at which the robot is moving and grooving (inch/s)

set_position()

```
void OdometryBase::set_position (
    const Pose2d & newpos = zero_pos) [virtual]
```

Sets the current position of the robot

Parameters

<i>newpos</i>	the new position that the odometry will believe it is at
---------------	--

Sets the current position of the robot

Reimplemented in [OdometrySerial](#), and [OdometryTank](#).

smallest_angle()

```
double OdometryBase::smallest_angle (
    double start_deg,
    double end_deg) [static]
```

Get the smallest difference in angle between a start heading and end heading. Returns the difference between -180 degrees and +180 degrees, representing the robot turning left or right, respectively.

Parameters

<i>start_deg</i>	initial angle (degrees)
<i>end_deg</i>	final angle (degrees)

Returns

the smallest angle from the initial to the final angle. This takes into account the wrapping of rotations around 360 degrees

Get the smallest difference in angle between a start heading and end heading. Returns the difference between -180 degrees and +180 degrees, representing the robot turning left or right, respectively.

update()

```
virtual Pose2d OdometryBase::update () [pure virtual]
```

Update the current position on the field based on the sensors

Returns

the location that the robot is at after the odometry does its calculations

Implemented in [Odometry3Wheel](#), [OdometrySerial](#), and [OdometryTank](#).

5.52.4 Member Data Documentation**accel**

```
double OdometryBase::accel
```

the rate at which we are accelerating (inch/s²)

ang_accel_deg

```
double OdometryBase::ang_accel_deg
```

the rate at which we are accelerating our turn (deg/s²)

ang_speed_deg

```
double OdometryBase::ang_speed_deg
```

the speed at which we are turning (deg/s)

current_pos

`Pose2d` `OdometryBase::current_pos`

Current position of the robot in terms of x,y,rotation

handle

`vex::task*` `OdometryBase::handle`

handle to the vex task that is running the odometry code

mut

`vex::mutex` `OdometryBase::mut`

Mutex to control multithreading

speed

`double` `OdometryBase::speed`

the speed at which we are travelling (inch/s)

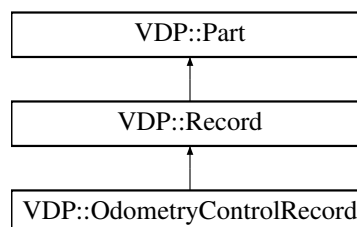
The documentation for this class was generated from the following files:

- `odometry_base.h`
- `odometry_base.cpp`

5.53 VDP::OdometryControlRecord Class Reference

```
#include <builtins.hpp>
```

Inheritance diagram for VDP::OdometryControlRecord:

**Public Member Functions**

- `OdometryControlRecord` (`std::string` name, `OdometryBase` &odom)
- void `response` () override

Public Member Functions inherited from [VDP::Record](#)

- [Record](#) (std::string name)
- [Record](#) (std::string name, const std::vector< [Part](#) * > &parts)
- [Record](#) (std::string name, std::vector< [PartPtr](#) > parts)
- [Record](#) (std::string name, [PacketReader](#) &reader)
- void [set_fields](#) (std::vector< [PartPtr](#) > fields)
- void [fetch](#) () override
- void [read_data_from_message](#) ([PacketReader](#) &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Additional Inherited Members

Protected Member Functions inherited from [VDP::Record](#)

- void [write_schema](#) ([PacketWriter](#) &sofar) const override
- void [write_message](#) ([PacketWriter](#) &sofar) const override

5.53.1 Detailed Description

Defines a record sets odometry values from the board

5.53.2 Constructor & Destructor Documentation

OdometryControlRecord()

```
VDP::OdometryControlRecord::OdometryControlRecord (
    std::string name,
    OdometryBase & odom)
```

Creates a record for taking odometry data from the debug board

Parameters

<i>name</i>	the name of the record to create
<i>odom</i>	the odometry to get data from

5.53.3 Member Function Documentation

response()

```
void VDP::OdometryControlRecord::response () [override], [virtual]
```

sets the odom position to the values from the board

sets the odometry position to the values from the debug board

Reimplemented from [VDP::Part](#).

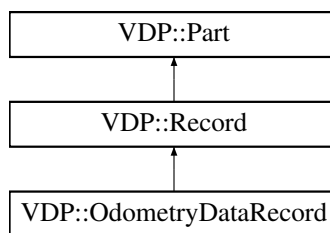
The documentation for this class was generated from the following files:

- builtins.hpp
- builtins.cpp

5.54 VDP::OdometryDataRecord Class Reference

```
#include <builtins.hpp>
```

Inheritance diagram for VDP::OdometryDataRecord:



Public Member Functions

- [OdometryDataRecord](#) (std::string name, [OdometryBase](#) &odom)
- void [fetch](#) () override

Public Member Functions inherited from [VDP::Record](#)

- [Record](#) (std::string name)
- [Record](#) (std::string name, const std::vector< [Part](#) * > &parts)
- [Record](#) (std::string name, std::vector< PartPtr > parts)
- [Record](#) (std::string name, PacketReader &reader)
- void [set_fields](#) (std::vector< PartPtr > fields)
- void [fetch](#) () override
- void [read_data_from_message](#) (PacketReader &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Additional Inherited Members

Protected Member Functions inherited from [VDP::Record](#)

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.54.1 Detailed Description

Defines a record that holds odometry values to be sent to the board

5.54.2 Constructor & Destructor Documentation

OdometryDataRecord()

```
VDP::OdometryDataRecord::OdometryDataRecord (
    std::string name,
    OdometryBase & odom)
```

Creates a record that contains a Float of the odometry X postion Float of the odometry Y postion Float of the odometry Rotation

Parameters

<i>name</i>	the name of the record to create
<i>odom</i>	the odometry to get data from

5.54.3 Member Function Documentation

fetch()

```
void VDP::OdometryDataRecord::fetch() [override], [virtual]
```

sets the data that the Odometry Parts hold

Implements [VDP::Part](#).

The documentation for this class was generated from the following files:

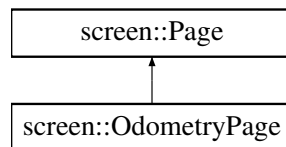
- builtins.hpp
- builtins.cpp

5.55 screen::OdometryPage Class Reference

a page that shows odometry position and rotation and a map (if an sd card with the file is on)

```
#include <screen.h>
```

Inheritance diagram for screen::OdometryPage:



Public Member Functions

- [OdometryPage](#) ([OdometryBase](#) &odom, double robot_width, double robot_height, bool do_trail)
Create an odometry trail. Make sure odometry is initilized before now.
- void [update](#) (bool was_pressed, int x, int y) override
- void [draw](#) (vex::brain::lcd &, bool first_draw, unsigned int frame_number) override

5.55.1 Detailed Description

a page that shows odometry position and rotation and a map (if an sd card with the file is on)

5.55.2 Constructor & Destructor Documentation

OdometryPage()

```
screen::OdometryPage::OdometryPage (
    OdometryBase & odom,
    double robot_width,
    double robot_height,
    bool do_trail)
```

Create an odometry trail. Make sure odometry is initilized before now.

Parameters

<i>odom</i>	the odometry system to monitor
<i>robot_width</i>	the width (side to side) of the robot in inches. Used for visualization
<i>robot_height</i>	the robot_height (front to back) of the robot in inches. Used for visualization
<i>do_trail</i>	whether or not to calculate and draw the trail. Drawing and storing takes a very <i>slight</i> extra amount of processing power

5.55.3 Member Function Documentation

draw()

```
void screen::OdometryPage::draw (
    vex::brain::lcd & scr,
    bool first_draw,
    unsigned int frame_number) [override], [virtual]
```

See also

[Page::draw](#)

Reimplemented from [screen::Page](#).

update()

```
void screen::OdometryPage::update (
    bool was_pressed,
    int x,
    int y) [override], [virtual]
```

See also

[Page::update](#)

Reimplemented from [screen::Page](#).

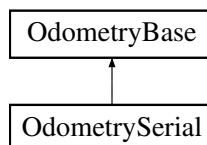
The documentation for this class was generated from the following files:

- screen.h
- screen.cpp

5.56 OdometrySerial Class Reference

```
#include <odometry_serial.h>
```

Inheritance diagram for OdometrySerial:



Public Member Functions

- [OdometrySerial](#) (bool is_async, bool calc_vel_acc_on_brain, [Pose2d](#) initial_pose, [Pose2d](#) sensor_offset, int32_t port, int32_t baudrate)
- void [send_config](#) (const [Pose2d](#) &initial_pose, const [Pose2d](#) &sensor_offset, const bool &calc_vel_acc_on_brain)
- [Pose2d](#) [update](#) () override
- void [set_position](#) (const [Pose2d](#) &new_pose) override
- int [receive_cobs_packet](#) (uint32_t port, uint8_t *buffer, size_t buffer_size)
- [Pose2d](#) [get_position](#) (void) override
- [Pose2d](#) [get_pose2d](#) (void)
- size_t [cobs_decode](#) (const uint8_t *buffer, size_t length, void *data)
- size_t [cobs_encode](#) (const void *data, size_t length, uint8_t *buffer)

Public Member Functions inherited from [OdometryBase](#)

- [OdometryBase](#) (bool is_async)
- void [end_async](#) ()
- double [get_angular_speed_deg](#) ()
- double [get_angular_accel_deg](#) ()

Additional Inherited Members**Static Public Member Functions inherited from [OdometryBase](#)**

- static int [background_task](#) (void *ptr)
- static double [smallest_angle](#) (double start_deg, double end_deg)

Public Attributes inherited from [OdometryBase](#)

- bool **end_task** = false
end_task is true if we instruct the odometry thread to shut down
- vex::task * [handle](#)
- vex::mutex [mut](#)
- [Pose2d](#) [current_pos](#)
- double [speed](#)
- double [accel](#)
- double [ang_speed_deg](#)
- double [ang_accel_deg](#)

5.56.1 Detailed Description[OdometrySerial](#)

This class handles the code for an odometry setup where calculations are done on an external coprocessor. Data is sent to the brain via smart port, using a generic serial (UART) connection.

This is a "set and forget" class, meaning once the object is created, the robot will immediately begin tracking it's movement in the background.

<https://rit.enterprise.slack.com/files/U04112Y5RB6/F080M01KPA5/predictperpindiculars2.pdf> 2024-2025 Notebook: Entries/Software Entries/Localization/N-Pod Odometry

Author

Jack Cammarata

Date

Jan 16 2025

5.56.2 Constructor & Destructor Documentation

OdometrySerial()

```
OdometrySerial::OdometrySerial (
    bool is_async,
    bool calc_vel_acc_on_brain,
    Pose2d initial_pose,
    Pose2d sensor_offset,
    int32_t port,
    int32_t baudrate)
```

Construct a new Odometry Serial Object

OdometrySerial

This class handles the code for an odometry setup where calculations are done on an external coprocessor. Data is sent to the brain via smart port, using a generic serial (UART) connection.

This is a "set and forget" class, meaning once the object is created, the robot will immediately begin tracking it's movement in the background.

<https://rit.enterprise.slack.com/files/U04112Y5RB6/F080M01KPA5/predictperpendiculars2.pdf> 2024-2025 Notebook: Entries/Software Entries/Localization/N-Pod Odometry

Author

Jack Cammarata

Date

Jan 16 2025 Construct a new Odometry Serial Object

5.56.3 Member Function Documentation

cobs_decode()

```
size_t OdometrySerial::cobs_decode (
    const uint8_t * buffer,
    size_t length,
    void * data)
```

COBS decode data from buffer

Parameters

<i>buffer</i>	Pointer to encoded input bytes
<i>length</i>	Number of bytes to decode
<i>data</i>	Pointer to decoded output data

Returns

Number of bytes successfully decoded

Note

Stops decoding if delimiter byte is found

cobs_encode()

```
size_t OdometrySerial::cobs_encode (
    const void * data,
    size_t length,
    uint8_t * buffer)
```

COBS encode data to buffer

Parameters

<i>data</i>	Pointer to input data to encode
<i>length</i>	Number of bytes to encode
<i>buffer</i>	Pointer to encoded output buffer

Returns

Encoded buffer length in bytes

Note

Does not output delimiter byte

get_pose2d()

```
Pose2d OdometrySerial::get_pose2d (
    void )
```

Gets the current position and rotation

Returns

the position that the odometry believes the robot is at

get_position()

```
Pose2d OdometrySerial::get_position (
    void ) [override], [virtual]
```

Gets the current position and rotation

Returns

the position that the odometry believes the robot is at

Reimplemented from [OdometryBase](#).

receive_cobs_packet()

Generated by Doxygen

```
int OdometrySerial::receive_cobs_packet (
    uint32_t port,
```

Parameters

<i>port</i>	the port number the serial is plugged into, counts from 0 instead of 1
<i>buffer</i>	pointer to a <code>uint8_t[]</code> where we put the data
<i>buffer_size</i>	length in bytes of the buffer

Returns

0 success

send_config()

```
void OdometrySerial::send_config (
    const Pose2d & initial_pose,
    const Pose2d & sensor_offset,
    const bool & calc_vel_acc_on_brain)
```

Send

set_position()

```
void OdometrySerial::set_position (
    const Pose2d & new_pose) [override], [virtual]
```

Resets the position and rotational data to the input.

Resets the position and rotational data to the input.

Parameters

<i>new_pose</i>	the pose to set the odometry to
-----------------	---------------------------------

Reimplemented from [OdometryBase](#).

update()

```
Pose2d OdometrySerial::update () [override], [virtual]
```

Update the current position of the robot once by reading a single packet from the serial port

Returns

the robot's updated position

Update the current position of the robot once by reading a single packet from the serial port, then updating all over values, velocity, accel

Returns

the robot's updated position

Implements [OdometryBase](#).

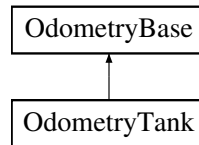
The documentation for this class was generated from the following files:

- `odometry_serial.h`
- `odometry_serial.cpp`

5.57 OdometryTank Class Reference

```
#include <odometry_tank.h>
```

Inheritance diagram for OdometryTank:



Public Member Functions

- [OdometryTank](#) (vex::motor_group &left_side, vex::motor_group &right_side, [robot_specs_t](#) &config, vex::inertial *imu=NULL, bool is_async=true)
- [OdometryTank](#) ([CustomEncoder](#) &left_custom_enc, [CustomEncoder](#) &right_custom_enc, [robot_specs_t](#) &config, vex::inertial *imu=NULL, bool is_async=true)
- [OdometryTank](#) (vex::encoder &left_vex_enc, vex::encoder &right_vex_enc, [robot_specs_t](#) &config, vex::inertial *imu=NULL, bool is_async=true)
- [Pose2d update](#) () override
- void [set_position](#) (const [Pose2d](#) &newpos=zero_pos) override

Public Member Functions inherited from [OdometryBase](#)

- [OdometryBase](#) (bool is_async)
- virtual [Pose2d get_position](#) (void)
- void [end_async](#) ()
- virtual double [get_speed](#) ()
- virtual double [get_accel](#) ()
- double [get_angular_speed_deg](#) ()
- double [get_angular_accel_deg](#) ()

Additional Inherited Members

Static Public Member Functions inherited from [OdometryBase](#)

- static int [background_task](#) (void *ptr)
- static double [smallest_angle](#) (double start_deg, double end_deg)

Public Attributes inherited from [OdometryBase](#)

- bool [end_task](#) = false
end_task is true if we instruct the odometry thread to shut down
- vex::task * [handle](#)
- vex::mutex [mut](#)
- [Pose2d current_pos](#)
- double [speed](#)
- double [accel](#)
- double [ang_speed_deg](#)
- double [ang_accel_deg](#)

5.57.1 Detailed Description

[OdometryTank](#) defines an odometry system for a tank drivetrain. This requires encoders in the same orientation as the drive wheels. Odometry is a "start and forget" subsystem, which means once it's created and configured, it will constantly run in the background and track the robot's X, Y and rotation coordinates.

5.57.2 Constructor & Destructor Documentation

OdometryTank() [1/3]

```
OdometryTank::OdometryTank (
    vex::motor_group & left_side,
    vex::motor_group & right_side,
    robot_specs_t & config,
    vex::inertial * imu = NULL,
    bool is_async = true)
```

Initialize the Odometry module, calculating position from the drive motors.

Parameters

<i>left_side</i>	The left motors
<i>right_side</i>	The right motors
<i>config</i>	the specifications that supply the odometry with descriptions of the robot. See robot_specs_t for what is contained
<i>imu</i>	The robot's inertial sensor. If not included, rotation is calculated from the encoders.
<i>is_async</i>	If true, position will be updated in the background continuously. If false, the programmer will have to manually call update() .

OdometryTank() [2/3]

```
OdometryTank::OdometryTank (
    CustomEncoder & left_custom_enc,
    CustomEncoder & right_custom_enc,
    robot_specs_t & config,
    vex::inertial * imu = NULL,
    bool is_async = true)
```

Initialize the Odometry module, calculating position from the drive motors.

Parameters

<i>left_custom_enc</i>	The left custom encoder
<i>right_custom_enc</i>	The right custom encoder
<i>config</i>	the specifications that supply the odometry with descriptions of the robot. See robot_specs_t for what is contained
<i>imu</i>	The robot's inertial sensor. If not included, rotation is calculated from the encoders.

<i>is_async</i>	If true, position will be updated in the background continuously. If false, the programmer will have to manually call update() .
-----------------	--

OdometryTank() [3/3]

```
OdometryTank::OdometryTank (
    vex::encoder & left_vex_enc,
    vex::encoder & right_vex_enc,
    robot_specs_t & config,
    vex::inertial * imu = NULL,
    bool is_async = true)
```

Initialize the Odometry module, calculating position from the drive motors.

Parameters

<i>left_vex_enc</i>	The left vex encoder
<i>right_vex_enc</i>	The right vex encoder
<i>config</i>	the specifications that supply the odometry with descriptions of the robot. See robot_specs_t for what is contained
<i>imu</i>	The robot's inertial sensor. If not included, rotation is calculated from the encoders.
<i>is_async</i>	If true, position will be updated in the background continuously. If false, the programmer will have to manually call update() .

5.57.3 Member Function Documentation**set_position()**

```
void OdometryTank::set_position (
    const Pose2d & newpos = zero_pos) [override], [virtual]
```

set_position tells the odometry to place itself at a position

Parameters

<i>newpos</i>	the position the odometry will take
---------------	-------------------------------------

Resets the position and rotational data to the input.

Reimplemented from [OdometryBase](#).

update()

```
Pose2d OdometryTank::update () [override], [virtual]
```

Update the current position on the field based on the sensors

Returns

the position that odometry has calculated itself to be at

Update, store and return the current position of the robot. Only use if not initializing with a separate thread.

Implements [OdometryBase](#).

The documentation for this class was generated from the following files:

- odometry_tank.h
- odometry_tank.cpp

5.58 OdomSetPosition Class Reference

```
#include <drive_commands.h>
```

Public Member Functions

- [OdomSetPosition](#) ([OdometryBase](#) &odom, const [Pose2d](#) &newpos=[OdometryBase::zero_pos](#))
- bool [run](#) () override

5.58.1 Detailed Description

AutoCommand wrapper class for the set_position function in the Odometry class

5.58.2 Constructor & Destructor Documentation**OdomSetPosition()**

```
OdomSetPosition::OdomSetPosition (
    OdometryBase & odom,
    const Pose2d & newpos = OdometryBase::zero\_pos)
```

constructs a new [OdomSetPosition](#) command

Parameters

<i>odom</i>	the odometry system we are setting
<i>newpos</i>	the position we are telling the odometry to take. defaults to (0, 0), angle = 90

Parameters

<i>odom</i>	the odometry system we are setting
<i>newpos</i>	the now position to set the odometry to

5.58.3 Member Function Documentation**run()**

```
bool OdomSetPosition::run () [override]
```

Run set_position Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- drive_commands.h
- drive_commands.cpp

5.59 VDP::PacketHeader Struct Reference

```
#include <protocol.hpp>
```

5.59.1 Detailed Description

struct to define the header of a packet, defines wheether a packet is Broadcoast or data and whether a packet is send or recieve

The documentation for this struct was generated from the following file:

- protocol.hpp

5.60 VDP::PacketWriter Class Reference

```
#include <protocol.hpp>
```

Public Member Functions

- [PacketWriter](#) (Packet &scratch_space)
- void [clear](#) ()
- size_t [size](#) ()
- void [write_byte](#) (uint8_t b)
- void [write_type](#) (Type t)
- void [write_string](#) (const std::string &str)
- void [write_channel_acknowledge](#) (const Channel &chan)
- void [write_channel_broadcast](#) (const Channel &chan)
- void [write_response](#) (std::deque< Channel > &channels)
- void [write_data_message](#) (const Channel &part)
- void [write_request](#) ()
- const Packet & [get_packet](#) () const
- template<typename Number>
void [write_number](#) (const Number &num)

5.60.1 Detailed Description

Defines a [PacketWriter](#), it writes packets

5.60.2 Constructor & Destructor Documentation

PacketWriter()

```
VDP::PacketWriter::PacketWriter (
    VDP::Packet & scratch) [explicit]
```

creates a packet writer

Parameters

<i>scratch_space</i>	the packet for the writer to write to
----------------------	---------------------------------------

5.60.3 Member Function Documentation

clear()

```
void VDP::PacketWriter::clear ()
```

clears the packet the writer is writing to

get_packet()

```
const Packet & VDP::PacketWriter::get_packet () const
```

Returns

the packet the writer is writing to

size()

```
size_t VDP::PacketWriter::size ()
```

Returns

the size of the packet

write_byte()

```
void VDP::PacketWriter::write_byte (
    uint8_t b)
```

writes a byte to the end of the packet

Parameters

<i>b</i>	the byte to write
----------	-------------------

write_channel_acknowledge()

```
void VDP::PacketWriter::write_channel_acknowledge (
    const Channel & chan)
```

writes a broadcast acknowledgement of a channel to the packet

Parameters

<i>chan</i>	the channel to write the acknowledgement for
-------------	--

write_channel_broadcast()

```
void VDP::PacketWriter::write_channel_broadcast (
    const Channel & chan)
```

writes a broadcast of a channel schematic to the packet

Parameters

<i>chan</i>	the channel to write the schematic from
-------------	---

write_data_message()

```
void VDP::PacketWriter::write_data_message (
    const Channel & chan)
```

writes the data from a channel to the packet

Parameters

<i>chan</i>	the Channel to write the data from
-------------	------------------------------------

write_number()

```
template<typename Number>
void VDP::PacketWriter::write_number (
    const Number & num) [inline]
```

writes a number to the end of the packet

write_request()

```
void VDP::PacketWriter::write_request ()
```

writes a request for a channel schematic to the packets

Parameters

<i>chan</i>	the Channel to write the data from
-------------	------------------------------------

writes a request for a channel schematic to the packet

Parameters

<i>chan</i>	the channel to request
-------------	------------------------

write_response()

```
void VDP::PacketWriter::write_response (
    std::deque< Channel > & response_queue)
```

writes a response packet to the packets

Parameters

<i>chan</i>	the Channel to write the data from
-------------	------------------------------------

writes a response packet to the brain

Parameters

<i>response_queue</i>	the queue of channels to respond with
-----------------------	---------------------------------------

write_string()

```
void VDP::PacketWriter::write_string (
    const std::string & str)
```

Parameters

<i>str</i>	the string to write to the packet
------------	-----------------------------------

write_type()

```
void VDP::PacketWriter::write_type (
    Type t)
```

writes a VDP type to the packet in the form of a byte

Parameters

<i>t</i>	the VDP type to write to the packet
----------	-------------------------------------

The documentation for this class was generated from the following files:

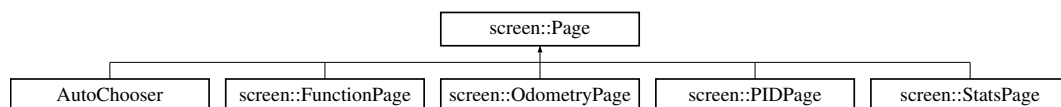
- protocol.hpp
- protocol.cpp

5.61 screen::Page Class Reference

[Page](#) describes one part of the screen slideshow.

```
#include <screen.h>
```

Inheritance diagram for screen::Page:

**Public Member Functions**

- virtual void [update](#) (bool was_pressed, int x, int y)
collect data, respond to screen input, do fast things (runs at 50hz even if you're not focused on this [Page](#) (only drawn page gets touch updates))
- virtual void [draw](#) (vex::brain::lcd &screen, bool first_draw, unsigned int frame_number)
draw stored data to the screen (runs at 10 hz and only runs if this page is in front)

5.61.1 Detailed Description

[Page](#) describes one part of the screen slideshow.

5.61.2 Member Function Documentation

Generated by Doxygen

draw()

Parameters

<i>first_draw</i>	true if we just switched to this page
<i>frame_number</i>	frame of drawing we are on (basically an animation tick)

Reimplemented in [screen::FunctionPage](#), [screen::OdometryPage](#), [screen::PIDPage](#), and [screen::StatsPage](#).

update()

```
virtual void screen::Page::update (
    bool was_pressed,
    int x,
    int y) [virtual]
```

collect data, respond to screen input, do fast things (runs at 50hz even if you're not focused on this [Page](#) (only drawn page gets touch updates))

Parameters

<i>was_pressed</i>	true if the screen has been pressed
<i>x</i>	x position of screen press (if the screen was pressed)
<i>y</i>	y position of screen press (if the screen was pressed)

Reimplemented in [screen::FunctionPage](#), [screen::OdometryPage](#), [screen::PIDPage](#), and [screen::StatsPage](#).

The documentation for this class was generated from the following file:

- screen.h

5.62 Parallel Class Reference

[Parallel](#) runs multiple commands in parallel and waits for all to finish before continuing. if none finish before this command's timeout, it will call `on_timeout` on all children continue.

```
#include <auto_command.h>
```

5.62.1 Detailed Description

[Parallel](#) runs multiple commands in parallel and waits for all to finish before continuing. if none finish before this command's timeout, it will call `on_timeout` on all children continue.

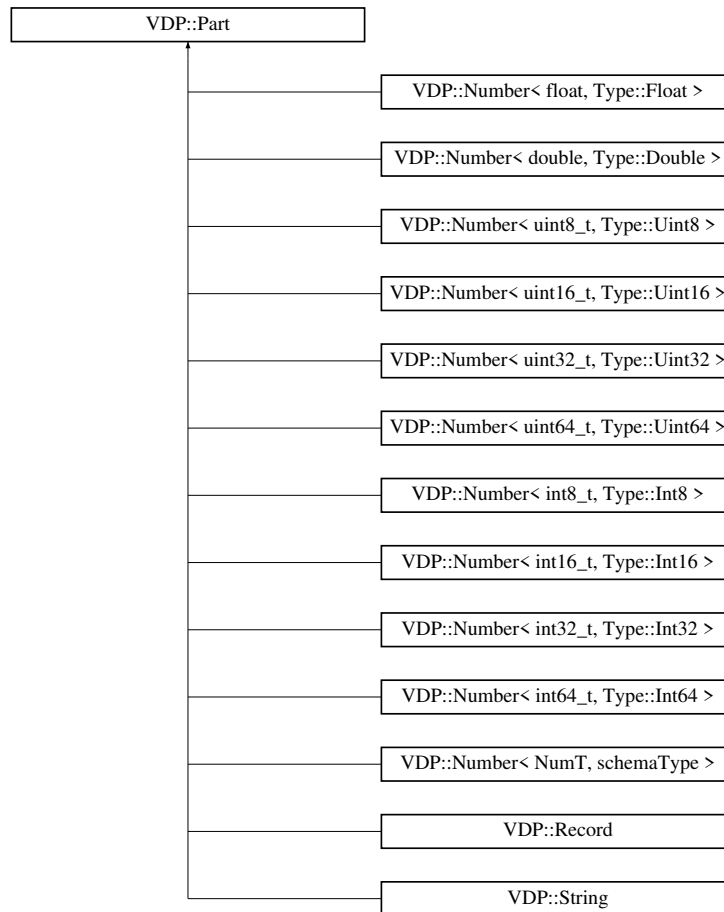
The documentation for this class was generated from the following files:

- auto_command.h
- auto_command.cpp

5.63 VDP::Part Class Reference

```
#include <protocol.hpp>
```

Inheritance diagram for VDP::Part:



Public Member Functions

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const
- virtual void [read_data_from_message](#) (PacketReader &reader)=0

Protected Member Functions

- virtual void [write_schema](#) (PacketWriter &sofar) const =0
- virtual void [write_message](#) (PacketWriter &sofar) const =0
- virtual void [pprint](#) (std::stringstream &ss, size_t indent) const =0
- virtual void [pprint_data](#) (std::stringstream &ss, size_t indent) const =0

5.63.1 Detailed Description

defines a [Part](#), which has a name and contains data essentially defines data formatted so that it can be sent to the debug board

5.63.2 Constructor & Destructor Documentation

Part()

```
VDP::Part::Part (  
    std::string name)
```

Creates a [Part](#) with a name a part is essentially data formatted so that it can be sent to the debug board

Parameters

<i>name</i>	name for the Part
-------------	-----------------------------------

5.63.3 Member Function Documentation

pprint()

```
virtual void VDP::Part::pprint (  
    std::stringstream & ss,  
    size_t indent) const [protected], [pure virtual]
```

changes a stringstream to a specified format, meant to be overridden

Parameters

<i>ss</i>	the stream of strings to change
<i>indent</i>	the amount of double spaced indents to add to the string

Implemented in [VDP::Number< NumT, schemaType >](#), [VDP::Number< double, Type::Double >](#), [VDP::Number< float, Type::Float >](#), [VDP::Number< int16_t, Type::Int16 >](#), [VDP::Number< int32_t, Type::Int32 >](#), [VDP::Number< int64_t, Type::Int64 >](#), [VDP::Number< int8_t, Type::Int8 >](#), [VDP::Number< uint16_t, Type::UInt16 >](#), [VDP::Number< uint32_t, Type::UInt32 >](#), [VDP::Number< uint64_t, Type::UInt64 >](#), [VDP::Number< uint8_t, Type::UInt8 >](#), and [VDP::String](#).

pprint_data()

```
virtual void VDP::Part::pprint_data (  
    std::stringstream & ss,  
    size_t indent) const [protected], [pure virtual]
```

changes a stringstream to the contain the [Part](#)'s data in a specified format, meant to be overridden

Parameters

<i>ss</i>	the stream of strings to change
<i>indent</i>	the amount of double spaced indents to add to the string

Implemented in [VDP::Number< NumT, schemaType >](#), [VDP::Number< double, Type::Double >](#), [VDP::Number< float, Type::Float >](#), [VDP::Number< int16_t, Type::Int16 >](#), [VDP::Number< int32_t, Type::Int32 >](#), [VDP::Number< int64_t, Type::Int64 >](#), [VDP::Number< int8_t, Type::Int8 >](#), [VDP::Number< uint16_t, Type::UInt16 >](#), [VDP::Number< uint32_t, Type::UInt32 >](#), [VDP::Number< uint64_t, Type::UInt64 >](#), [VDP::Number< uint8_t, Type::UInt8 >](#), and [VDP::String](#).

pretty_print()

```
std::string VDP::Part::pretty_print () const
```

Returns

- a string of the [Part](#) with the format "name: string"
- a stringstream of the [Part](#) with the format "name: string"

pretty_print_data()

```
std::string VDP::Part::pretty_print_data () const
```

Returns

- a string of the [Part](#)'s data with the format "name: value"
- a stringstream of the [Part](#)'s data with the format "name: value"

read_data_from_message()

```
virtual void VDP::Part::read_data_from_message (
    PacketReader & reader) [pure virtual]
```

sets the data the part contains to the data from a packet, meant to be overridden

Parameters

<i>reader</i>	the PacketReader to read data from
---------------	------------------------------------

Implemented in [VDP::Number< NumT, schemaType >](#), [VDP::Number< double, Type::Double >](#), [VDP::Number< float, Type::Float >](#), [VDP::Number< int16_t, Type::Int16 >](#), [VDP::Number< int32_t, Type::Int32 >](#), [VDP::Number< int64_t, Type::Int64 >](#), [VDP::Number< int8_t, Type::Int8 >](#), [VDP::Number< uint16_t, Type::UInt16 >](#), [VDP::Number< uint32_t, Type::UInt32 >](#), [VDP::Number< uint64_t, Type::UInt64 >](#), [VDP::Number< uint8_t, Type::UInt8 >](#), [VDP::Record](#), and [VDP::String](#).

write_message()

```
virtual void VDP::Part::write_message (
    PacketWriter &sofar) const [protected], [pure virtual]
```

writes the value of the [Part](#) to a packet so that it can be sent to the debug board

Parameters

<i>sofar</i>	the packet writer to write with
--------------	---------------------------------

Implemented in [VDP::Number< NumT, schemaType >](#), [VDP::Number< double, Type::Double >](#), [VDP::Number< float, Type::Float >](#), [VDP::Number< int16_t, Type::Int16 >](#), [VDP::Number< int32_t, Type::Int32 >](#), [VDP::Number< int64_t, Type::Int64 >](#), [VDP::Number< int8_t, Type::Int8 >](#), [VDP::Number< uint16_t, Type::UInt16 >](#), [VDP::Number< uint32_t, Type::UInt32 >](#), [VDP::Number< uint64_t, Type::UInt64 >](#), [VDP::Number< uint8_t, Type::UInt8 >](#), [VDP::Record](#), and [VDP::String](#).

Generated by Doxygen

write_schema()

Parameters

<i>sofar</i>	the packet writer to write with
--------------	---------------------------------

Implemented in [VDP::Number< NumT, schemaType >](#), [VDP::Number< double, Type::Double >](#), [VDP::Number< float, Type::Float >](#), [VDP::Number< int16_t, Type::Int16 >](#), [VDP::Number< int32_t, Type::Int32 >](#), [VDP::Number< int64_t, Type::Int64 >](#), [VDP::Number< int8_t, Type::Int8 >](#), [VDP::Number< uint16_t, Type::UInt16 >](#), [VDP::Number< uint32_t, Type::UInt32 >](#), [VDP::Number< uint64_t, Type::UInt64 >](#), [VDP::Number< uint8_t, Type::UInt8 >](#), [VDP::Record](#), and [VDP::String](#).

The documentation for this class was generated from the following files:

- [protocol.hpp](#)
- [protocol.cpp](#)

5.64 PurePursuit::Path Class Reference

```
#include <pure_pursuit.h>
```

Public Member Functions

- [Path](#) (std::vector< [Translation2d](#) > points, double radius)
- const std::vector< [Translation2d](#) > [get_points](#) ()
- double [get_radius](#) ()
- bool [is_valid](#) ()

5.64.1 Detailed Description

Wrapper for a vector of points, checking if any of the points are too close for pure pursuit

5.64.2 Constructor & Destructor Documentation**Path()**

```
PurePursuit::Path::Path (
    std::vector< Translation2d > points,
    double radius)
```

Create a [Path](#)

Parameters

<i>points</i>	the points that make up the path
<i>radius</i>	the lookahead radius for pure pursuit

5.64.3 Member Function Documentation

get_points()

```
const std::vector< Translation2d > PurePursuit::Path::get_points ()
```

Get the points associated with this [Path](#)

get_radius()

```
double PurePursuit::Path::get_radius ()
```

Get the radius associated with this [Path](#)

is_valid()

```
bool PurePursuit::Path::is_valid ()
```

Get whether this path will behave as expected

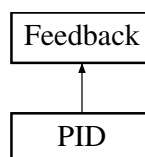
The documentation for this class was generated from the following files:

- pure_pursuit.h
- pure_pursuit.cpp

5.65 PID Class Reference

```
#include <pid.h>
```

Inheritance diagram for PID:



Classes

- struct [pid_config_t](#)

Public Types

- enum [ERROR_TYPE](#)

Public Member Functions

- [PID](#) ([pid_config_t](#) &config)
- void [init](#) (double start_pt, double set_pt) override
- double [update](#) (double sensor_val) override
- double [update](#) (double sensor_val, double v_setpt)
- double [get_sensor_val](#) () const
gets the sensor value that we were last updated with
- double [get](#) () override
- void [set_limits](#) (double lower, double upper) override
- bool [is_on_target](#) () override
- void [reset](#) ()
- double [get_error](#) ()
- double [get_output](#) ()
- double [get_target](#) () const
- void [set_target](#) (double target)

Public Attributes

- [pid_config_t](#) & [config](#)

5.65.1 Detailed Description

[PID](#) Class

Defines a standard feedback loop using the constants kP, kI, kD, deadband, and on_target_time. The formula is:

$$\text{out} = kP * \text{error} + kI * \text{integral}(\text{d Error}) + kD * (\text{dError}/\text{dt})$$

The [PID](#) object will determine it is "on target" when the error is within the deadband, for a duration of on_target_time

Author

Ryan McGee

Date

4/3/2020

5.65.2 Member Enumeration Documentation

ERROR_TYPE

enum [PID::ERROR_TYPE](#)

An enum to distinguish between a linear and angular calculation of [PID](#) error.

5.65.3 Constructor & Destructor Documentation

[PID](#)()

Parameters

<i>config</i>	the configuration data for this controller
---------------	--

Create the [PID](#) object

5.65.4 Member Function Documentation**get()**

```
double PID::get () [override], [virtual]
```

Gets the current [PID](#) out value, from when [update\(\)](#) was last run

Returns

the Out value of the controller (voltage, RPM, whatever the [PID](#) controller is controlling)

Gets the current [PID](#) out value, from when [update\(\)](#) was last run

Implements [Feedback](#).

get_error()

```
double PID::get_error ()
```

Get the delta between the current sensor data and the target

Returns

the error calculated. how it is calculated depends on `error_method` specified in [pid_config_t](#)

Get the delta between the current sensor data and the target

get_output()

```
double PID::get_output ()
```

Get the output calculated from the P, I, D and Error values

Returns

the output calculated from the pid controller specified in [pid_config_t](#)

Get the delta between the current sensor data and the target

get_sensor_val()

```
double PID::get_sensor_val () const
```

gets the sensor value that we were last updated with

Returns

sensor_val

get_target()

```
double PID::get_target () const
```

Get the [PID](#)'s target

Returns

the target the [PID](#) controller is trying to achieve

init()

```
void PID::init (  
    double start_pt,  
    double set_pt) [override], [virtual]
```

Inherited from [Feedback](#) for interoperability. Update the setpoint and reset integral accumulation

start_pt can be safely ignored in this feedback controller

Parameters

<i>start_pt</i>	completely ignored for PID . necessary to satisfy Feedback base
<i>set_pt</i>	sets the target of the PID controller
<i>start_vel</i>	completely ignored for PID . necessary to satisfy Feedback base
<i>end_vel</i>	sets the target end velocity of the PID controller

Implements [Feedback](#).

is_on_target()

```
bool PID::is_on_target () [override], [virtual]
```

Checks if the [PID](#) controller is on target.

Returns

true if the loop is within [deadband] for [on_target_time] seconds

Returns true if the loop is within [deadband] for [on_target_time] seconds

Implements [Feedback](#).

reset()

```
void PID::reset ()
```

Reset the [PID](#) loop by resetting time since 0 and accumulated error.

set_limits()

```
void PID::set_limits (
    double lower,
    double upper) [override], [virtual]
```

Set the limits on the [PID](#) out. The [PID](#) out will "clip" itself to be between the limits.

Parameters

<i>lower</i>	the lower limit. the PID controller will never command the output go below <code>lower</code>
<i>upper</i>	the upper limit. the PID controller will never command the output go higher than <code>upper</code>

Set the limits on the [PID](#) out. The [PID](#) out will "clip" itself to be between the limits.

Implements [Feedback](#).

set_target()

```
void PID::set_target (
    double target)
```

Set the target for the [PID](#) loop, where the robot is trying to end up

Parameters

<i>target</i>	the sensor reading we would like to achieve
---------------	---

Set the target for the [PID](#) loop, where the robot is trying to end up

update() [1/2]

```
double PID::update (
    double sensor_val) [override], [virtual]
```

Update the [PID](#) loop by taking the time difference from last update, and running the [PID](#) formula with the new sensor data

Parameters

<i>sensor_val</i>	the distance, angle, encoder position or whatever it is we are measuring
-------------------	--

Returns

the new output. What would be returned by [PID::get\(\)](#)

Implements [Feedback](#).

update() [2/2]

```
double PID::update (
    double sensor_val,
    double v_setpt)
```

Update the [PID](#) loop by taking the time difference from last update, and running the [PID](#) formula with the new sensor data

Parameters

<i>sensor_val</i>	the distance, angle, encoder position or whatever it is we are measuring
<i>v_setpt</i>	Expected velocity setpoint, to subtract from the D term (for velocity control)

Returns

the new output. What would be returned by [PID::get\(\)](#)

5.65.5 Member Data Documentation**config**

```
pid_config_t& PID::config
```

configuration struct for this controller. see [pid_config_t](#) for information about what this contains

The documentation for this class was generated from the following files:

- [pid.h](#)
- [pid.cpp](#)

5.66 PID::pid_config_t Struct Reference

```
#include <pid.h>
```

Public Attributes

- double **p**
*proportional coefficient p * error()*
- double **i**
*integral coefficient i * integral(error)*
- double **d**
*derivitave coefficient d * derivative(error)*
- double **deadband**
at what threshold are we close enough to be finished
- double [on_target_time](#)
- [ERROR_TYPE](#) [error_method](#)

5.66.1 Detailed Description

[pid_config_t](#) holds the configuration parameters for a pid controller. In addition to the constant of proportional, integral and derivative, these parameters include:

- deadband -
- on_target_time - for how long do we have to be at the target to stop. As well, [pid_config_t](#) holds an error type which determines whether errors should be calculated as if the sensor position is a measure of distance or an angle.

5.66.2 Member Data Documentation

error_method

[ERROR_TYPE](#) PID::pid_config_t::error_method

Linear or angular. whether to do error as a simple subtraction or to wrap

on_target_time

double PID::pid_config_t::on_target_time

the time in seconds that we have to be on target for to say we are officially at the target

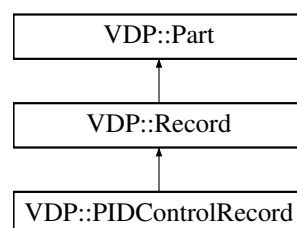
The documentation for this struct was generated from the following file:

- pid.h

5.67 VDP::PIDControlRecord Class Reference

```
#include <builtins.hpp>
```

Inheritance diagram for VDP::PIDControlRecord:



Public Member Functions

- [PIDControlRecord](#) (std::string name, [PID](#) &pid)
- void [response](#) () override

Public Member Functions inherited from [VDP::Record](#)

- [Record](#) (std::string name)
- [Record](#) (std::string name, const std::vector< [Part](#) * > &parts)
- [Record](#) (std::string name, std::vector< PartPtr > parts)
- [Record](#) (std::string name, PacketReader &reader)
- void [set_fields](#) (std::vector< PartPtr > fields)
- void [fetch](#) () override
- void [read_data_from_message](#) (PacketReader &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Additional Inherited Members

Protected Member Functions inherited from [VDP::Record](#)

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.67.1 Detailed Description

Defines a record for setting pid values from the board

5.67.2 Constructor & Destructor Documentation

PIDControlRecord()

```
VDP::PIDControlRecord::PIDControlRecord (
    std::string name,
    PID & pid)
```

Creates a record for setting pid values from the board

Parameters

<i>name</i>	the name of the record to create
<i>pid</i>	the pid to get data from

Defines a record for setting pid values from the board

5.67.3 Member Function Documentation

response()

```
void VDP::PIDControlRecord::response () [override], [virtual]
```

sets the [PID](#) values to the values from the board

Reimplemented from [VDP::Part](#).

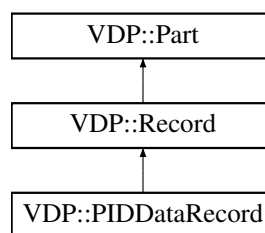
The documentation for this class was generated from the following files:

- builtins.hpp
- builtins.cpp

5.68 VDP::PIDDataRecord Class Reference

```
#include <builtins.hpp>
```

Inheritance diagram for VDP::PIDDataRecord:



Public Member Functions

- [PIDDataRecord](#) (std::string name, [PID](#) &pid)
- void [fetch](#) () override

Public Member Functions inherited from [VDP::Record](#)

- [Record](#) (std::string name)
- [Record](#) (std::string name, const std::vector< [Part](#) * > &parts)
- [Record](#) (std::string name, std::vector< PartPtr > parts)
- [Record](#) (std::string name, PacketReader &reader)
- void [set_fields](#) (std::vector< PartPtr > fields)
- void [fetch](#) () override
- void [read_data_from_message](#) (PacketReader &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Additional Inherited Members

Protected Member Functions inherited from [VDP::Record](#)

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.68.1 Detailed Description

Defines a record that holds pid values to be sent to the board

5.68.2 Constructor & Destructor Documentation

PIDDataRecord()

```
VDP::PIDDataRecord::PIDDataRecord (  
    std::string name,  
    PID & pid)
```

Creates a record that contains a Float of the pid P value Float of the pid I value Float of the pid D value Float of the pid error Float of the pid output [String](#) of the pid type (linear or angular)

Parameters

<i>name</i>	the name of the record to create
<i>pid</i>	the pid to get data from

5.68.3 Member Function Documentation

fetch()

```
void VDP::PIDDataRecord::fetch () [override], [virtual]
```

sets the data that the [PID](#) Parts hold

sets the data that the [PID](#) Parts hold to be sent to the board

Implements [VDP::Part](#).

The documentation for this class was generated from the following files:

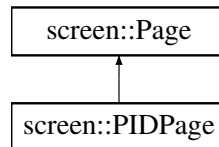
- builtins.hpp
- builtins.cpp

5.69 screen::PIDPage Class Reference

[PIDPage](#) provides a way to tune a pid controller on the screen.

```
#include <screen.h>
```

Inheritance diagram for screen::PIDPage:



Public Member Functions

- [PIDPage](#) ([PID](#) &pid, std::string name, std::function< void(void)> onchange=[]() {})
Create a [PIDPage](#).
- void [update](#) (bool was_pressed, int x, int y) override
- void [draw](#) (vex::brain::lcd &, bool first_draw, unsigned int frame_number) override

5.69.1 Detailed Description

[PIDPage](#) provides a way to tune a pid controller on the screen.

5.69.2 Constructor & Destructor Documentation

PIDPage()

```
screen::PIDPage::PIDPage (
    PID & pid,
    std::string name,
    std::function< void(void)> onchange = []() {})
```

Create a [PIDPage](#).

Parameters

<i>pid</i>	the pid controller we're changing
<i>name</i>	a name to recognize this pid controller if we've got multiple pid screens
<i>onchange</i>	a function that is called when a tuning parameter is changed. If you need to update stuff on that change register a handler here

5.69.3 Member Function Documentation

draw()

```
void screen::PIDPage::draw (
    vex::brain::lcd & scr,
    bool first_draw,
    unsigned int frame_number) [override], [virtual]
```

See also

[Page::draw](#)

Reimplemented from [screen::Page](#).

update()

```
void screen::PIDPage::update (
    bool was_pressed,
    int x,
    int y) [override], [virtual]
```

See also

[Page::update](#)

Reimplemented from [screen::Page](#).

The documentation for this class was generated from the following files:

- screen.h
- screen.cpp

5.70 Pose2d Class Reference

```
#include <pose2d.h>
```

Public Member Functions

- constexpr [Pose2d](#) ()
- [Pose2d](#) (const [Translation2d](#) &translation, const [Rotation2d](#) &rotation)
- [Pose2d](#) (const double &x, const double &y, const [Rotation2d](#) &rotation)
- [Pose2d](#) (const double &x, const double &y, const double &radians)
- [Pose2d](#) (const [Translation2d](#) &translation, const double &radians)
- [Pose2d](#) (const Eigen::Vector3d &pose_vector)
- [Translation2d](#) translation () const
- double x () const
- double y () const
- [Rotation2d](#) rotation () const
- void setRotationRad (double rotRad)
- void setRotationDeg (double rotDeg)
- bool operator== (const [Pose2d](#) other) const
- [Pose2d](#) operator* (const double &scalar) const
- [Pose2d](#) operator/ (const double &scalar) const
- [Pose2d](#) operator+ (const [Transform2d](#) &transform) const
- [Transform2d](#) operator- (const [Pose2d](#) &other) const
- [Pose2d](#) relative_to (const [Pose2d](#) &other) const
- [Pose2d](#) transform_by (const [Transform2d](#) &transform) const
- [Pose2d](#) exp (const [Twist2d](#) &twist) const
- [Twist2d](#) log (const [Pose2d](#) &end_pose) const

Friends

- `std::ostream & operator<< (std::ostream &os, const Pose2d &pose)`

5.70.1 Detailed Description

Class representing a pose in 2d space with x, y, and rotational components

Assumes conventional cartesian coordinate system: Looking down at the coordinate plane, +X is right +Y is up +Theta is counterclockwise

5.70.2 Constructor & Destructor Documentation

Pose2d() [1/6]

```
Pose2d::Pose2d () [inline], [constexpr]
```

Default Constructor for [Pose2d](#)

Pose2d() [2/6]

```
Pose2d::Pose2d (
    const Translation2d & translation,
    const Rotation2d & rotation)
```

Constructs a pose with given translation and rotation components.

Parameters

<i>translation</i>	translational component.
<i>rotation</i>	rotational component.

Pose2d() [3/6]

```
Pose2d::Pose2d (
    const double & x,
    const double & y,
    const Rotation2d & rotation)
```

Constructs a pose with given translation and rotation components.

Parameters

<i>x</i>	x component.
<i>y</i>	y component.
<i>rotation</i>	rotational component.

Parameters

<i>x</i>	x component.
<i>y</i>	y component.
<i>radians</i>	rotational component in radians.

Pose2d() [5/6]

```
Pose2d::Pose2d (
    const Translation2d & translation,
    const double & radians)
```

Constructs a pose with given translation and rotation components.

Parameters

<i>translation</i>	translational component.
<i>radians</i>	rotational component in radians.

Pose2d() [6/6]

```
Pose2d::Pose2d (
    const Eigen::Vector3d & pose_vector)
```

Constructs a pose with given translation and rotation components.

Parameters

<i>pose_vector</i>	vector of the form [x, y, theta].
--------------------	-----------------------------------

5.70.3 Member Function Documentation**exp()**

```
Pose2d Pose2d::exp (
    const Twist2d & twist) const
```

Applies a twist (pose delta) to a pose by including first order dynamics of heading.

When applying a twist, imagine a constant angular velocity, the translational components must be rotated into the global frame at every point along the twist, simply adding the deltas does not do this, and using euler integration results in some error. This is the analytic solution that that problem.

Can also be thought of more simply as applying a twist as following an arc rather than a straight line.

See this document for more information on the pose exponential and its derivation. <https://file.tavsys.net/control/controls-engineering-in-frc.pdf#section.10.2>

Parameters

<i>old_pose</i>	The pose to which the twist will be applied.
<i>twist</i>	The twist, represents a pose delta.

Returns

new pose that has been moved forward according to the twist.

log()

```
Twist2d Pose2d::log (
    const Pose2d & end_pose) const
```

The inverse of the pose exponential.

Determines the twist required to go from this pose to the given end pose. suppose you have [Pose2d](#) a, [Twist2d](#) twist if $a.exp(twist) = b$ then $a.log(b) = twist$

Parameters

<i>end_pose</i>	the end pose to find the mapping to.
-----------------	--------------------------------------

Returns

the twist required to go from this pose to the given end

operator*()

```
Pose2d Pose2d::operator* (
    const double & scalar) const
```

Multiplies this pose by a scalar. Simply multiplies each component.

Parameters

<i>scalar</i>	the scalar value to multiply by.
---------------	----------------------------------

operator+()

```
Pose2d Pose2d::operator+ (
    const Transform2d & transform) const
```

Adds a transform to this pose. Transforms the pose in the pose's frame.

Parameters

<i>transform</i>	the change in pose.
------------------	---------------------

operator-()

```
Transform2d Pose2d::operator- (
    const Pose2d & other) const
```

Subtracts one pose from another to find the transform between them.

Parameters

<i>other</i>	the pose to subtract.
--------------	-----------------------

operator/()

```
Pose2d Pose2d::operator/ (
    const double & scalar) const
```

Divides this pose by a scalar. Simply divides each component.

Parameters

<i>scalar</i>	the scalar value to divide by.
---------------	--------------------------------

operator==()

```
bool Pose2d::operator== (
    const Pose2d other) const
```

Compares this to another pose.

Parameters

<i>other</i>	the other pose to compare to.
--------------	-------------------------------

Returns

true if each of the components are within 1e-9 of each other.

relative_to()

```
Pose2d Pose2d::relative_to (
    const Pose2d & other) const
```

Finds the pose equivalent to this pose relative to another arbitrary pose rather than the origin.

Parameters

<i>other</i>	the pose representing the new origin.
--------------	---------------------------------------

Returns

this pose relative to another pose.

rotation()

```
Rotation2d Pose2d::rotation () const
```

Returns the rotational component.

Returns

the rotational component.

setRotationDeg()

```
void Pose2d::setRotationDeg (  
    double rotDeg)
```

sets the rotation value of the rotational component in Degrees

setRotationRad()

```
void Pose2d::setRotationRad (  
    double rotRad)
```

sets the rotation value of the rotational component in Radians

transform_by()

```
Pose2d Pose2d::transform_by (  
    const Transform2d & transform) const
```

Adds a transform to this pose. Simply adds each component.

Parameters

<i>transform</i>	the change in pose.
------------------	---------------------

Returns

the pose after being transformed.

translation()

```
Translation2d Pose2d::translation () const
```

Returns the translational component.

Returns

the translational component.

x()

```
double Pose2d::x () const
```

Returns the x value of the translational component.

Returns

the x value of the translational component.

y()

```
double Pose2d::y () const
```

Returns the y value of the translational component.

Returns

the y value of the translational component.

5.70.4 Friends And Related Symbol Documentation

operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Pose2d & pose) [friend]
```

Sends a pose to an output stream. Ex. `std::cout << pose;`

prints "Pose2d[x: (value), y: (value), rad: (radians), deg: (degrees)]"

The documentation for this class was generated from the following files:

- pose2d.h
- pose2d.cpp

5.71 PurePursuitCommand Class Reference

```
#include <drive_commands.h>
```

Public Member Functions

- [PurePursuitCommand](#) ([TankDrive](#) &drive_sys, [Feedback](#) &feedback, [PurePursuit::Path](#) path, directionType dir, double max_speed=1, double end_speed=0)
- bool [run](#) () override
- void [on_timeout](#) () override

5.71.1 Detailed Description

Autocommand wrapper class for pure pursuit function in the [TankDrive](#) class

5.71.2 Constructor & Destructor Documentation

PurePursuitCommand()

```
PurePursuitCommand::PurePursuitCommand (
    TankDrive & drive_sys,
    Feedback & feedback,
    PurePursuit::Path path,
    directionType dir,
    double max_speed = 1,
    double end_speed = 0)
```

Construct a Pure Pursuit AutoCommand

Parameters

<i>path</i>	The list of coordinates to follow, in order
<i>dir</i>	Run the bot forwards or backwards
<i>feedback</i>	The feedback controller determining speed
<i>max_speed</i>	Limit the speed of the robot (for pid / pidff feedbacks)

5.71.3 Member Function Documentation

on_timeout()

```
void PurePursuitCommand::on_timeout () [override]
```

Reset the drive system when it times out

run()

```
bool PurePursuitCommand::run () [override]
```

Direct call to [TankDrive::pure_pursuit](#)

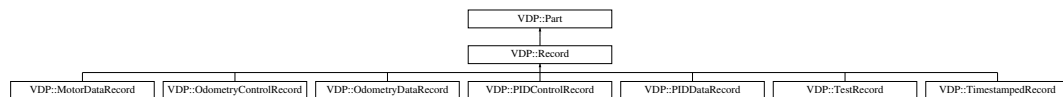
The documentation for this class was generated from the following files:

- drive_commands.h
- drive_commands.cpp

5.72 VDP::Record Class Reference

```
#include <types.hpp>
```

Inheritance diagram for VDP::Record:

**Public Member Functions**

- [Record](#) (std::string name)
- [Record](#) (std::string name, const std::vector< [Part](#) * > &parts)
- [Record](#) (std::string name, std::vector< PartPtr > parts)
- [Record](#) (std::string name, PacketReader &reader)
- void [set_fields](#) (std::vector< PartPtr > fields)
- void [fetch](#) () override
- void [read_data_from_message](#) (PacketReader &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Protected Member Functions

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.72.1 Detailed Description

Defines a [Part](#) that contains another [Part](#) essentially an array of parts that is formatted so that it can be sent to the debug board

Parameters

<i>name</i>	the name for the part
-------------	-----------------------

Record() [2/4]

```
VDP::Record::Record (  
    std::string name,  
    const std::vector< Part * > & parts)
```

Creates a [Record](#) with a name that contains the Parts inside a vector of Parts a [Record](#) is essentially an array of parts that is formatted so that it can be sent to the debug board

Parameters

<i>name</i>	the name for the part
<i>parts</i>	the vector of Parts for the record to hold

Record() [3/4]

```
VDP::Record::Record (  
    std::string name,  
    std::vector< PartPtr > parts)
```

Creates a [Record](#) with a name that contains the Parts inside a vector of [Part](#) Pointers a [Record](#) is essentially an array of parts that is formatted so that it can be sent to the debug board

Parameters

<i>name</i>	the name for the part
<i>parts</i>	the vector of Part Pointers for the record to hold

Creates a [Record](#) with a name that contains the Parts inside a vector of [Part](#) Pointers a [Record](#) is essentially an array of parts that is formatted so that it can be sent to the debug board

Parameters

<i>name</i>	the name for the Record
<i>parts</i>	the vector of Part Pointers for the record to hold

Record() [4/4]

```
VDP::Record::Record (  
    std::string name,  
    PacketReader & reader)
```

Creates a record with a name based off of a packet read by a PacketReader a [Record](#) is essentially an array of parts that is formatted so that it can be sent to the debug board

Parameters

<i>name</i>	
<i>reader</i>	

5.72.3 Member Function Documentation**fetch()**

```
void VDP::Record::fetch () [override], [virtual]
```

sets the values of each [Part](#) the [Record](#) contains

Implements [VDP::Part](#).

Reimplemented in [VDP::TimestampedRecord](#).

read_data_from_message()

```
void VDP::Record::read_data_from_message (  
    PacketReader & reader) [override], [virtual]
```

writes a message to the packet containing part record

Parameters

<i>sofar</i>	the PacketWriter to write with
--------------	--

Implements [VDP::Part](#).

set_fields()

```
void VDP::Record::set_fields (  
    std::vector< PartPtr > fs)
```

sets the [Record](#) to contain Parts from a part Pointer

Parameters

<i>fs</i>	the vector of Part Pointers for the record to hold
-----------	--

write_message()

```
void VDP::Record::write_message (  
    PacketWriter & sofar) const [override], [protected], [virtual]
```

writes a message to the packet containing part record

Parameters

<i>sofar</i>	the PacketWriter to write with
--------------	--

Implements [VDP::Part](#).

write_schema()

```
void VDP::Record::write_schema (
    PacketWriter & sofar) const [override], [protected], [virtual]
```

writes the [Record](#) as the

Implements [VDP::Part](#).

The documentation for this class was generated from the following files:

- types.hpp
- types.cpp

5.73 Rect Struct Reference

```
#include <geometry.h>
```

5.73.1 Detailed Description

Describes a Rectangle with a minimum and maximum point

The documentation for this struct was generated from the following file:

- geometry.h

5.74 VDP::RegistryListener< MutexType > Class Template Reference

```
#include <registry-listener.hpp>
```

Public Member Functions

- [RegistryListener](#) ([AbstractDevice](#) *device)
- void [take_packet](#) (const Packet &pac)
Call this if you are a device who has a packet for the protocol to decode.
- bool [submit_response](#) (PacketType type, ChannelID id, PartPtr data)
Submits a channel to respond to the board with.
- void [install_broadcast_callback](#) (CallbackFn on_broadcastf)
- void [install_data_callback](#) (CallbackFn on_dataf)
- bool [send_data](#) (ChannelID id, PartPtr data)

5.74.1 Detailed Description

```
template<typename MutexType>
class VDP::RegistryListener< MutexType >
```

defines a device registry for sending data or listening to data over a device

5.74.2 Constructor & Destructor Documentation

RegistryListener()

```
template<typename MutexType>
VDP::RegistryListener< MutexType >::RegistryListener (
    AbstractDevice * device) [inline]
```

creates a device registry for sending data or listening to data over the device

Parameters

<i>device</i>	the device to send data to
<i>reg_type</i>	the type of registry it is (Listener or Controller)

5.74.3 Member Function Documentation

install_broadcast_callback()

```
template<typename MutexType>
void VDP::RegistryListener< MutexType >::install_broadcast_callback (
    CallbackFn on_broadcastf) [inline]
```

installs a callback to a function that is called when the registry broadcasts the data schematic

Parameters

<i>on_broadcastf</i>	the callback to run when the registry broadcasts the schematic
----------------------	--

install_data_callback()

```
template<typename MutexType>
void VDP::RegistryListener< MutexType >::install_data_callback (
    CallbackFn on_dataf) [inline]
```

installs a callback to a function that is called when the registry broadcasts data

Parameters

<i>on_dataf</i>	the callback to run when the registry broadcasts data
-----------------	---

send_data()

```
template<typename MutexType>
bool VDP::RegistryListener< MutexType >::send_data (
    ChannelID id,
    PartPtr data) [inline]
```

sets the data at the channel id to a [Part](#) Pointer and sends it to the device

Parameters

<i>id</i>	The id of the channel to hold the data
<i>data</i>	the Part Pointer for the channel to hold and send to the device

submit_response()

```
template<typename MutexType>
bool VDP::RegistryListener< MutexType >::submit_response (
    PacketType type,
    ChannelID id,
    PartPtr data) [inline]
```

Submits a channel to respond to the board with.

Parameters

<i>id</i>	the channel id to respond with
-----------	--------------------------------

Returns

if the channel was submitted successfully or not

take_packet()

```
template<typename MutexType>
void VDP::RegistryListener< MutexType >::take_packet (
    const Packet & pac) [inline]
```

Call this if you are a device who has a packet for the protocol to decode.

Parameters

<i>pac</i>	the packet to take.
------------	---------------------

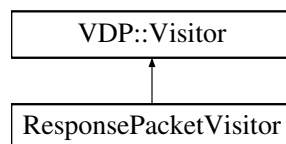
The documentation for this class was generated from the following file:

- registry-listener.hpp

5.75 ResponsePacketVisitor Class Reference

```
#include <visitor.hpp>
```

Inheritance diagram for ResponsePacketVisitor:

**Public Member Functions**

- [ResponsePacketVisitor](#) (VDP::PartPtr from_part)
- void [VisitRecord](#) (VDP::Record *record) override
- void [VisitString](#) (VDP::String *str) override
- void [VisitFloat](#) (VDP::Float *float_part) override
- void [VisitDouble](#) (VDP::Double *double_part) override
- void [VisitInt64](#) (VDP::Int64 *int64_part) override
- void [VisitInt32](#) (VDP::Int32 *int32_part) override
- void [VisitInt16](#) (VDP::Int16 *int16_part) override
- void [VisitInt8](#) (VDP::Int8 *int8_part) override
- void [VisitUInt64](#) (VDP::UInt64 *UInt64_part) override
- void [VisitUInt32](#) (VDP::UInt32 *UInt32_part) override
- void [VisitUInt16](#) (VDP::UInt16 *UInt16_part) override
- void [VisitUInt8](#) (VDP::UInt8 *UInt8_part) override

5.75.1 Detailed Description

Class for visiting parts from a response packet and modifying the parts we have in response to the packet essentially takes the response data and moves it into our data

5.75.2 Constructor & Destructor Documentation**ResponsePacketVisitor()**

```
ResponsePacketVisitor::ResponsePacketVisitor (
    VDP::PartPtr from_part)
```

Class for visiting parts from a response packet and modifying the parts we have in response to the packet essentially takes the response data and moves it into our data

Parameters

<i>from_part</i>	the part with the data we want to merge with our data
------------------	---

5.75.3 Member Function Documentation**VisitDouble()**

```
void ResponsePacketVisitor::VisitDouble (
    VDP::Double * double_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitFloat()

```
void ResponsePacketVisitor::VisitFloat (
    VDP::Float * float_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitInt16()

```
void ResponsePacketVisitor::VisitInt16 (
    VDP::Int16 * int16_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitInt32()

```
void ResponsePacketVisitor::VisitInt32 (
    VDP::Int32 * int32_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitInt64()

```
void ResponsePacketVisitor::VisitInt64 (
    VDP::Int64 * int64_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitInt8()

```
void ResponsePacketVisitor::VisitInt8 (
    VDP::Int8 * int8_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitRecord()

```
void ResponsePacketVisitor::VisitRecord (
    VDP::Record * record) [override], [virtual]
```

essentially just calls the same visitor on all the parts inside the record

Implements [VDP::Visitor](#).

VisitString()

```
void ResponsePacketVisitor::VisitString (
    VDP::String * str) [override], [virtual]
```

Checks if the string has the value N/A, if it does skip it otherwise replace our data with the new string

Implements [VDP::Visitor](#).

VisitUInt16()

```
void ResponsePacketVisitor::VisitUInt16 (
    VDP::UInt16 * UInt16_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitUint32()

```
void ResponsePacketVisitor::VisitUint32 (
    VDP::Uint32 * Uint32_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitUint64()

```
void ResponsePacketVisitor::VisitUint64 (
    VDP::Uint64 * Uint64_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

VisitUint8()

```
void ResponsePacketVisitor::VisitUint8 (
    VDP::Uint8 * Uint8_part) [override], [virtual]
```

checks if the value if the lowest possible number it can be, if it is skip it otherwise replace our datw with the new number

Implements [VDP::Visitor](#).

The documentation for this class was generated from the following files:

- visitor.hpp
- visitor.cpp

5.76 robot_specs_t Struct Reference

```
#include <robot_specs.h>
```

Public Attributes

- double **robot_radius**
if you were to draw a circle with this radius, the robot would be entirely contained within it
- double **odom_wheel_diam**
the diameter of the wheels used for
- double **odom_gear_ratio**
the ratio of the odometry wheel to the encoder reading odometry data
- double **dist_between_wheels**
the distance between centers of the central drive wheels
- double [drive_correction_cutoff](#)
- [Feedback](#) * **drive_feedback**
the default feedback for autonomous driving
- [Feedback](#) * **turn_feedback**
the default feedback for autonomous turning
- [PID::pid_config_t](#) **correction_pid**
the pid controller to keep the robot driving in as straight a line as possible

5.76.1 Detailed Description

Main robot characterization struct. This will be passed to all the major subsystems that require info about the robot. All distance measurements are in inches.

5.76.2 Member Data Documentation

drive_correction_cutoff

```
double robot_specs_t::drive_correction_cutoff
```

the distance at which to stop trying to turn towards the target. If we are less than this value, we can continue driving forward to minimize our distance but will not try to spin around to point directly at the target

The documentation for this struct was generated from the following file:

- robot_specs.h

5.77 Rotation2d Class Reference

```
#include <rotation2d.h>
```

Public Member Functions

- constexpr [Rotation2d](#) ()
- [Rotation2d](#) (const double &radians)
- [Rotation2d](#) (const double &x, const double &y)
- [Rotation2d](#) (const [Translation2d](#) &translation)
- double [radians](#) () const
- double [degrees](#) () const
- double [revolutions](#) () const
- double [f_cos](#) () const
- double [f_sin](#) () const
- double [f_tan](#) () const
- Eigen::Matrix2d [rotation_matrix](#) () const
- double [wrapped_radians_180](#) () const
- double [wrapped_degrees_180](#) () const
- double [wrapped_revolutions_180](#) () const
- double [wrapped_radians_360](#) () const
- double [wrapped_degrees_360](#) () const
- double [wrapped_revolutions_360](#) () const
- [Rotation2d](#) operator+ (const [Rotation2d](#) &other) const
- [Rotation2d](#) operator- (const [Rotation2d](#) &other) const
- [Rotation2d](#) operator- () const
- [Rotation2d](#) operator* (const double &scalar) const
- [Rotation2d](#) operator/ (const double &scalar) const
- bool operator== (const [Rotation2d](#) &other) const

Friends

- `std::ostream & operator<< (std::ostream &os, const Rotation2d &rotation)`

5.77.1 Detailed Description

Class representing a rotation in 2d space. Stores theta in radians, as well as cos and sin.

Internally this angle is stored continuously, however there are functions that return wrapped angles: "180" is from $[-\pi, \pi)$, $[-180, 180)$, $[-0.5, 0.5)$ "360" is from $[0, 2\pi)$, $[0, 360)$, $[0, 1)$

5.77.2 Constructor & Destructor Documentation

Rotation2d() [1/4]

```
Rotation2d::Rotation2d () [inline], [constexpr]
```

Default Constructor for [Rotation2d](#)

Rotation2d() [2/4]

```
Rotation2d::Rotation2d (
    const double & radians)
```

Constructs a rotation with the given value in radians.

Parameters

<i>radians</i>	the value of the rotation in radians.
----------------	---------------------------------------

Rotation2d() [3/4]

```
Rotation2d::Rotation2d (
    const double & x,
    const double & y)
```

Constructs a rotation given x and y values. Does not have to be normalized. The angle from the x axis to the point.

$[\text{theta}] = [\text{atan2}(y, x)]$

Parameters

<i>x</i>	the x value of the point
<i>y</i>	the y value of the point

Rotation2d() [4/4]

Generated by Doxygen

```
Rotation2d::Rotation2d (
    const Translation2d & translation)
```

Parameters

<i>translation</i>	
--------------------	--

5.77.3 Member Function Documentation**degrees()**

```
double Rotation2d::degrees () const
```

Returns the degree angle value.

Returns

the degree angle value.

f_cos()

```
double Rotation2d::f_cos () const
```

Returns the cosine of the angle value.

Returns

the cosine of the angle value

f_sin()

```
double Rotation2d::f_sin () const
```

Returns the sine of the angle value.

Returns

the sine of the angle value.

f_tan()

```
double Rotation2d::f_tan () const
```

Returns the tangent of the angle value.

Returns

the tangent of the angle value.

operator*()

Parameters

<i>scalar</i>	the scalar value to multiply the rotation by.
---------------	---

Returns

the rotation multiplied by the scalar.

operator+()

```
Rotation2d Rotation2d::operator+ (
    const Rotation2d & other) const
```

Adds the values of two rotations using a rotation matrix

$[new_cos] = [other.cos, -other.sin][cos]$ $[new_sin] = [other.sin, other.cos][sin]$ $new_value = atan2(new_sin, new_cos)$

Parameters

<i>other</i>	the other rotation to add to this rotation.
--------------	---

Returns

the sum of the two rotations.

Adds the values of two rotations using a rotation matrix.

$[new_cos] = [other.cos, -other.sin][cos]$ $[new_sin] = [other.sin, other.cos][sin]$ $new_value = atan2(new_sin, new_cos)$

Parameters

<i>other</i>	the other rotation to add to this rotation.
--------------	---

Returns

the sum of the two rotations.

operator-() [1/2]

```
Rotation2d Rotation2d::operator- () const
```

Takes the inverse of this rotation by flipping it. Equivalent to adding 180 degrees.

Returns

this inverse of the rotation.

Takes the inverse of this rotation by flipping it.

Returns

Parameters

<i>other</i>	the other rotation to subtract from this rotation.
--------------	--

Returns

the difference between the two rotations.

operator/()

```
Rotation2d Rotation2d::operator/ (
    const double & scalar) const
```

Divides this rotation by a scalar.

Parameters

<i>scalar</i>	the scalar value to divide the rotation by.
---------------	---

Returns

the rotation divided by the scalar.

operator==()

```
bool Rotation2d::operator== (
    const Rotation2d & other) const
```

Compares two rotations. Returns true if their values are within 1e-9 radians of each other, to account for floating point error.

Parameters

<i>other</i>	the other rotation to compare to
--------------	----------------------------------

Returns

whether the values of the rotations are within 1e-9 radians of each other

radians()

```
double Rotation2d::radians () const
```

Returns the radian angle value.

Returns

the radian angle value.

revolutions()

```
double Rotation2d::revolutions () const
```

Returns the revolution angle value.

Returns

the revolution angle value.

rotation_matrix()

```
Eigen::Matrix2d Rotation2d::rotation_matrix () const
```

Returns the rotation matrix equivalent to this rotation $[\cos, -\sin]$ $R = [\sin, \cos]$

Returns

the rotation matrix equivalent to this rotation

wrapped_degrees_180()

```
double Rotation2d::wrapped_degrees_180 () const
```

Returns the degree angle value, wrapped from $[-180, 180)$.

Returns

the degree angle value, wrapped from $[-180, 180)$

wrapped_degrees_360()

```
double Rotation2d::wrapped_degrees_360 () const
```

Returns the degree angle value, wrapped from $[0, 360)$.

Returns

the degree angle value, wrapped from $[0, 360)$

wrapped_radians_180()

```
double Rotation2d::wrapped_radians_180 () const
```

Returns the radian angle value, wrapped from $[-\pi, \pi)$.

Returns

the radian angle value, wrapped from $[-\pi, \pi)$

wrapped_radians_360()

```
double Rotation2d::wrapped_radians_360 () const
```

Returns the radian angle value, wrapped from [0, 2pi).

Returns

the radian angle value, wrapped from [0, 2pi)

wrapped_revolutions_180()

```
double Rotation2d::wrapped_revolutions_180 () const
```

Returns the revolution angle value, wrapped from [-0.5, 0.5).

Returns

the revolution angle value, wrapped from [-0.5, 0.5)

wrapped_revolutions_360()

```
double Rotation2d::wrapped_revolutions_360 () const
```

Returns the revolution angle value, wrapped from [0, 1).

Returns

the revolution angle value, wrapped from [0, 1)

5.77.4 Friends And Related Symbol Documentation**operator<<**

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Rotation2d & rotation) [friend]
```

Sends a rotation to an output stream. Ex. `std::cout << rotation;`

prints "Rotation2d[rad: (radians), deg: (degrees)]"

The documentation for this class was generated from the following files:

- rotation2d.h
- rotation2d.cpp

5.78 ScaledSphericalSimplexSigmaPoints< STATES > Class Template Reference

```
#include <unscented_kalman_filter.h>
```

Public Member Functions

- [ScaledSphericalSimplexSigmaPoints](#) (double alpha=0.001, double beta=2)
- int [num_sigmas](#) ()
- EMat< STATES, NUM_SIGMAS > [square_root_sigma_points](#) (const EVec< STATES > &x, const EMat< STATES, STATES > &S)
- const EVec< NUM_SIGMAS > & [Wm](#) () const
- const EVec< NUM_SIGMAS > & [Wc](#) () const
- double [Wm](#) (int i) const
- double [Wc](#) (int i) const

5.78.1 Detailed Description

```
template<int STATES>
class ScaledSphericalSimplexSigmaPoints< STATES >
```

Generates sigma points and weights according to the paper [1] This is very different from Wan and Merwe's formulation.

This only requires $N + 2$ sigma points instead of $2N + 1$ sigma points. Rather than generating sigma points symmetrically around the mean, it generates them as vertices of an N -simplex.

The performance of the filter using this reduced set of sigma points is identical to the standard method, so there is no downside to using it here.

[1] A Scaled Spherical Simplex [Filter](#) (S3F) with a decreased $n + 2$ sigma points set size and equivalent $2n + 1$ Unscented Kalman [Filter](#) (UKF) accuracy

Template Parameters

<i>STATES</i>	the dimension of the state. NUM_SIGMAS sigma points and weights will be generated.
---------------	--

5.78.2 Constructor & Destructor Documentation

ScaledSphericalSimplexSigmaPoints()

```
template<int STATES>
ScaledSphericalSimplexSigmaPoints< STATES >::ScaledSphericalSimplexSigmaPoints (
    double alpha = 0.001,
    double beta = 2) [inline]
```

Constructs a sigma point generator for Spherical Simplex sigma points

Parameters

<i>alpha</i>	Determines the spread of the sigma points around the mean. Smaller values are closer to the mean, this is usually a small value.
<i>beta</i>	Incorporates prior knowledge of the distribution of the state. For Gaussian distributions, beta = 2 is optimal.

5.78.3 Member Function Documentation**num_sigmas()**

```
template<int STATES>
int ScaledSphericalSimplexSigmaPoints< STATES >::num_sigmas () [inline]
```

Returns the number of sigma points, for simplex sigma points this is N+2.

square_root_sigma_points()

```
template<int STATES>
EMat< STATES, NUM_SIGMAS > ScaledSphericalSimplexSigmaPoints< STATES >::square_root_sigma_↵
points (
    const EVec< STATES > & x,
    const EMat< STATES, STATES > & S) [inline]
```

Computes the sigma points given a mean (x) and square-root covariance (S).

Parameters

<i>x</i>	Vector of the means.
<i>S</i>	Square-root covariance.

Returns

Matrix containing the sigma points. Each column contains one sigma point in the same space as x. The first column is the same as the mean, with the others arranged around the mean.

Wc() [1/2]

```
template<int STATES>
const EVec< NUM_SIGMAS > & ScaledSphericalSimplexSigmaPoints< STATES >::Wc () const [inline]
```

Returns a vector containing the weights of each sigma point for the covariance.

Wc() [2/2]

```
template<int STATES>
double ScaledSphericalSimplexSigmaPoints< STATES >::Wc (
    int i) const [inline]
```

Returns the weight for the i-th sigma point for the covariance.

Parameters

<i>i</i>	Element of the weights vector to return.
----------	--

Wm() [1/2]

```
template<int STATES>
const EVec< NUM_SIGMAS > & ScaledSphericalSimplexSigmaPoints< STATES >::Wm () const [inline]
```

Returns a vector containing the weights of each sigma point for the mean.

Wm() [2/2]

```
template<int STATES>
double ScaledSphericalSimplexSigmaPoints< STATES >::Wm (
    int i) const [inline]
```

Returns the weight for the i-th sigma point for the mean.

Parameters

<i>i</i>	Element of the weights vector to return.
----------	--

The documentation for this class was generated from the following file:

- unscented_kalman_filter.h

5.79 screen::ScreenData Struct Reference

The [ScreenData](#) class holds the data that will be passed to the screen thread you probably shouldnt have to use it.

5.79.1 Detailed Description

The [ScreenData](#) class holds the data that will be passed to the screen thread you probably shouldnt have to use it.

The documentation for this struct was generated from the following file:

- screen.cpp

5.80 Serializer Class Reference

Serializes Arbitrary data to a file on the SD Card.

```
#include <serializer.h>
```

Public Member Functions

- **~Serializer ()**
Save and close upon destruction (bc of vex, this doesnt always get called when the program ends. To be sure, call save_to_disk).
- **Serializer** (const std::string &filename, bool flush_always=true)
create a [Serializer](#)
- void **save_to_disk ()** const
saves current [Serializer](#) state to disk
- void **set_int** (const std::string &name, int i)
Setters - not saved until save_to_disk is called.
- void **set_bool** (const std::string &name, bool b)
sets a bool by the name of name to b. If flush_always == true, this will save to the sd card
- void **set_double** (const std::string &name, double d)
sets a double by the name of name to d. If flush_always == true, this will save to the sd card
- void **set_string** (const std::string &name, std::string str)
sets a string by the name of name to s. If flush_always == true, this will save to the sd card
- int **int_or** (const std::string &name, int otherwise)
gets a value stored in the serializer. If not found, sets the value to otherwise
- bool **bool_or** (const std::string &name, bool otherwise)
gets a value stored in the serializer. If not, sets the value to otherwise
- double **double_or** (const std::string &name, double otherwise)
gets a value stored in the serializer. If not, sets the value to otherwise
- std::string **string_or** (const std::string &name, std::string otherwise)
gets a value stored in the serializer. If not, sets the value to otherwise

5.80.1 Detailed Description

Serializes Arbitrary data to a file on the SD Card.

5.80.2 Constructor & Destructor Documentation

Serializer()

```
Serializer::Serializer (
    const std::string & filename,
    bool flush_always = true) [inline], [explicit]
```

create a [Serializer](#)

Parameters

<i>filename</i>	the file to read from. If filename does not exist we will create that file
<i>flush_always</i>	If true, after every write flush to a file. If false, you are responsible for calling save_to_disk

5.80.3 Member Function Documentation

bool_or()

```
bool Serializer::bool_or (
    const std::string & name,
    bool otherwise)
```

Parameters

<i>name</i>	name of value
<i>otherwise</i>	value if the name is not specified

Returns

the value if found or otherwise

double_or()

```
double Serializer::double_or (
    const std::string & name,
    double otherwise)
```

gets a value stored in the serializer. If not, sets the value to otherwise

Parameters

<i>name</i>	name of value
<i>otherwise</i>	value if the name is not specified

Returns

the value if found or otherwise

int_or()

```
int Serializer::int_or (
    const std::string & name,
    int otherwise)
```

gets a value stored in the serializer. If not found, sets the value to otherwise

Getters Return value if it exists in the serializer

Parameters

<i>name</i>	name of value
<i>otherwise</i>	value if the name is not specified

Returns

the value if found or otherwise

save_to_disk()

```
void Serializer::save_to_disk () const
```

saves current [Serializer](#) state to disk

forms data bytes then saves to filename this was opened with

set_bool()

```
void Serializer::set_bool (
    const std::string & name,
    bool b)
```

sets a bool by the name of name to b. If flush_always == true, this will save to the sd card

Parameters

<i>name</i>	name of bool
<i>b</i>	value of bool

set_double()

```
void Serializer::set_double (
    const std::string & name,
    double d)
```

sets a double by the name of name to d. If flush_always == true, this will save to the sd card

Parameters

<i>name</i>	name of double
<i>d</i>	value of double

set_int()

```
void Serializer::set_int (
    const std::string & name,
    int i)
```

Setters - not saved until save_to_disk is called.

sets an integer by the name of name to i. If flush_always == true, this will save to the sd card

Parameters

<i>name</i>	name of integer
-------------	-----------------

<i>i</i>	value of integer
----------	------------------

set_string()

```
void Serializer::set_string (
    const std::string & name,
    std::string str)
```

sets a string by the name of name to s. If flush_always == true, this will save to the sd card

Parameters

<i>name</i>	name of string
<i>i</i>	value of string

string_or()

```
std::string Serializer::string_or (
    const std::string & name,
    std::string otherwise)
```

gets a value stored in the serializer. If not, sets the value to otherwise

Parameters

<i>name</i>	name of value
<i>otherwise</i>	value if the name is not specified

Returns

the value if found or otherwise

The documentation for this class was generated from the following files:

- serializer.h
- serializer.cpp

5.81 screen::SliderWidget Class Reference

Widget that updates a double value. Updates by reference so watch out for race conditions cuz the screen stuff lives on another thread.

```
#include <screen.h>
```

Public Member Functions

- [SliderWidget](#) (double &val, double low, double high, [Rect](#) rect, std::string name)
Creates a slider widget.
- bool [update](#) (bool was_pressed, int x, int y)
responds to user input
- void [draw](#) (vex::brain::lcd &, bool first_draw, unsigned int frame_number)
Page::draws the slide to the screen

5.81.1 Detailed Description

Widget that updates a double value. Updates by reference so watch out for race conditions cuz the screen stuff lives on another thread.

5.81.2 Constructor & Destructor Documentation

SliderWidget()

```
screen::SliderWidget::SliderWidget (  
    double & val,  
    double low,  
    double high,  
    Rect rect,  
    std::string name) [inline]
```

Creates a slider widget.

Parameters

<i>val</i>	reference to the value to modify
<i>low</i>	minimum value to go to
<i>high</i>	maximum value to go to
<i>rect</i>	rect to draw it
<i>name</i>	name of the value

5.81.3 Member Function Documentation

update()

```
bool screen::SliderWidget::update (  
    bool was_pressed,  
    int x,  
    int y)
```

responds to user input

Parameters

<i>was_pressed</i>	if the screen is pressed
<i>x</i>	x position if the screen was pressed
<i>y</i>	y position if the screen was pressed

Returns

true if the value updated

The documentation for this class was generated from the following files:

- screen.h
- screen.cpp

5.82 SpinRPMCommand Class Reference

```
#include <flywheel_commands.h>
```

Public Member Functions

- [SpinRPMCommand](#) ([Flywheel](#) &flywheel, int rpm)
- bool [run](#) () override

5.82.1 Detailed Description

File: [flywheel_commands.h](#) Desc: [insert meaningful desc] AutoCommand wrapper class for the spin_rpm function in the [Flywheel](#) class

5.82.2 Constructor & Destructor Documentation**SpinRPMCommand()**

```
SpinRPMCommand::SpinRPMCommand (
    Flywheel & flywheel,
    int rpm)
```

Construct a SpinRPM Command

Parameters

<i>flywheel</i>	the flywheel sys to command
<i>rpm</i>	the rpm that we should spin at

File: flywheel_commands.cpp Desc: [insert meaningful desc]

5.82.3 Member Function Documentation

run()

```
bool SpinRPMCommand::run () [override]
```

Run spin_manual Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- flywheel_commands.h
- flywheel_commands.cpp

5.83 PurePursuit::spline Struct Reference

```
#include <pure_pursuit.h>
```

5.83.1 Detailed Description

Represents a piece of a cubic spline with $s(x) = a(x-x_i)^3 + b(x-x_i)^2 + c(x-x_i) + d$ The `x_start` and `x_end` shows where the equation is valid.

The documentation for this struct was generated from the following file:

- pure_pursuit.h

5.84 State Struct Reference

```
#include <state_machine.h>
```

5.84.1 Detailed Description

Abstract class that all states for this machine must inherit from States MUST override `respond()` and `id()` in order to function correctly (the compiler won't have it any other way)

The documentation for this struct was generated from the following file:

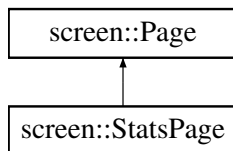
- state_machine.h

5.85 screen::StatsPage Class Reference

Draws motor stats and battery stats to the screen.

```
#include <screen.h>
```

Inheritance diagram for screen::StatsPage:



Public Member Functions

- [StatsPage](#) (std::map< std::string, vex::motor & > motors)
Creates a stats page.
- void [update](#) (bool was_pressed, int x, int y) override
- void [draw](#) (vex::brain::lcd &, bool first_draw, unsigned int frame_number) override

5.85.1 Detailed Description

Draws motor stats and battery stats to the screen.

5.85.2 Constructor & Destructor Documentation

StatsPage()

```
screen::StatsPage::StatsPage (
    std::map< std::string, vex::motor & > motors)
```

Creates a stats page.

Parameters

<i>motors</i>	a map of string to motor that we want to draw on this page
---------------	--

5.85.3 Member Function Documentation

draw()

```
void screen::StatsPage::draw (
    vex::brain::lcd & scr,
    bool first_draw,
    unsigned int frame_number) [override], [virtual]
```

See also

[Page::draw](#)

Reimplemented from [screen::Page](#).

update()

```
void screen::StatsPage::update (
    bool was_pressed,
    int x,
    int y) [override], [virtual]
```

See also[Page::update](#)Reimplemented from [screen::Page](#).

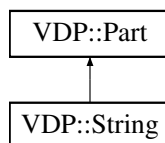
The documentation for this class was generated from the following files:

- screen.h
- screen.cpp

5.86 VDP::String Class Reference

```
#include <types.hpp>
```

Inheritance diagram for VDP::String:

**Public Member Functions**

- [String](#) (std::string name, FetchFunc fetcher=[]) { return "no value";}
- void [fetch](#) () override
- void [response](#) () override
- void [set_value](#) (std::string new_value)
- std::string [get_value](#) ()
- void [read_data_from_message](#) (PacketReader &reader) override
- void [pprint](#) (std::stringstream &ss, size_t indent) const override
- void [pprint_data](#) (std::stringstream &ss, size_t indent) const override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Protected Member Functions

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.86.1 Detailed Description

A string type conveyed as a part

5.86.2 Constructor & Destructor Documentation

String()

```
VDP::String::String (
    std::string field_name,
    FetchFunc fetcher = []() { return "no value"; }) [explicit]
```

creates a string type conveyed as a part with a name and a fetcher

Parameters

<i>name</i>	name of the string to have
<i>fetcher</i>	the fetch function to use when running fetch()

creates a string type conveyed as a part with a name and a fetcher

Parameters

<i>name</i>	name of the string to have
<i>fetcher</i>	the fetcher function to use when assigning it new data

5.86.3 Member Function Documentation

fetch()

```
void VDP::String::fetch () [override], [virtual]
```

function to run when fetching this part, runs the fetch function

used to assign the string new data, runs the fetch function

Implements [VDP::Part](#).

get_value()

```
std::string VDP::String::get_value ()
```

Returns

the currently stored string

Parameters

<i>ss</i>	the stringstream to change
<i>indent</i>	the amount of indents to use

Implements [VDP::Part](#).

pprint_data()

```
void VDP::String::pprint_data (
    std::stringstream & ss,
    size_t indent) const [override], [virtual]
```

changes a stringstream to be formatted as name: value

Parameters

<i>ss</i>	the stringstream to change
<i>indent</i>	the amount of indents to use

Implements [VDP::Part](#).

read_data_from_message()

```
void VDP::String::read_data_from_message (
    PacketReader & reader) [override], [virtual]
```

sets the string part's value to the string read by a packet reader

Parameters

<i>reader</i>	the packet reader to get the string from
---------------	--

sets the string part's value to the string read by a packet reader

Parameters

<i>reader</i>	the part reader to get
---------------	------------------------

Implements [VDP::Part](#).

response()

```
void VDP::String::response () [override], [virtual]
```

function to run when receiving to this part

Reimplemented from [VDP::Part](#).

set_value()

Parameters

<i>new_value</i>	the string to set the value to
------------------	--------------------------------

write_message()

```
void VDP::String::write_message (
    PacketWriter & sofar) const [override], [protected], [virtual]
```

writes the strings data to a packet

Parameters

<i>sofar</i>	the packet writer to write with
--------------	---------------------------------

Implements [VDP::Part](#).

write_schema()

```
void VDP::String::write_schema (
    PacketWriter & sofar) const [override], [protected], [virtual]
```

writes the schematic for the string to a packet

Parameters

<i>sofar</i>	the packet writer to write with
--------------	---------------------------------

Implements [VDP::Part](#).

The documentation for this class was generated from the following files:

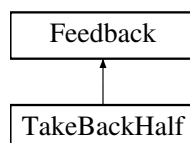
- types.hpp
- types.cpp

5.87 TakeBackHalf Class Reference

A velocity controller.

```
#include <take_back_half.h>
```

Inheritance diagram for TakeBackHalf:



Public Member Functions

- void [init](#) (double start_pt, double set_pt)
- double [update](#) (double val) override
- double [get](#) () override
- void [set_limits](#) (double lower, double upper) override
- bool [is_on_target](#) () override

Public Attributes

- double **TBH_gain**
tuned parameter

5.87.1 Detailed Description

A velocity controller.

Warning

If you try to use this as a position controller, it will fail.

5.87.2 Member Function Documentation

get()

```
double TakeBackHalf::get () [override], [virtual]
```

Returns

the last saved result from the feedback controller

Implements [Feedback](#).

init()

```
void TakeBackHalf::init (  
    double start_pt,  
    double set_pt) [virtual]
```

Initialize the feedback controller for a movement

Parameters

<i>start_pt</i>	the current sensor value
-----------------	--------------------------

<i>set_pt</i>	where the sensor value should be
<i>start_vel</i>	Movement starting velocity (IGNORED)
<i>end_vel</i>	Movement ending velocity (IGNORED)

Implements [Feedback](#).

is_on_target()

```
bool TakeBackHalf::is_on_target () [override], [virtual]
```

Returns

true if the feedback controller has reached it's setpoint

Implements [Feedback](#).

set_limits()

```
void TakeBackHalf::set_limits (
    double lower,
    double upper) [override], [virtual]
```

Clamp the upper and lower limits of the output. If both are 0, no limits should be applied.

Parameters

<i>lower</i>	Upper limit
<i>upper</i>	Lower limit

Implements [Feedback](#).

update()

```
double TakeBackHalf::update (
    double val) [override], [virtual]
```

Iterate the feedback loop once with an updated sensor value

Parameters

<i>val</i>	value from the sensor
------------	-----------------------

Returns

feedback loop result

Implements [Feedback](#).

The documentation for this class was generated from the following files:

- take_back_half.h
- take_back_half.cpp

5.88 TankDrive Class Reference

```
#include <tank_drive.h>
```

Public Types

- enum class [BrakeType](#) { [None](#) , [ZeroVelocity](#) , [Smart](#) , [TurnOnly](#) }

Public Member Functions

- [TankDrive](#) (motor_group &left_motors, motor_group &right_motors, [robot_specs_t](#) &config, [OdometryBase](#) *odom=NULL)
- void [stop](#) ()
- [Pose2d](#) [get_position](#) ()
- void [drive_tank](#) (double left, double right, int power=1, [BrakeType](#) bt=[BrakeType::None](#))
- void [drive_tank_raw](#) (double left, double right)
- void [drive_arcade](#) (double forward_back, double left_right, int power=1, [BrakeType](#) bt=[BrakeType::None](#))
- bool [drive_forward](#) (double inches, directionType dir, [Feedback](#) &feedback, double max_speed=1, double end_speed=0)
- bool [drive_forward](#) (double inches, directionType dir, double max_speed=1, double end_speed=0)
- bool [turn_degrees](#) (double degrees, [Feedback](#) &feedback, double max_speed=1, double end_speed=0)
- bool [turn_degrees](#) (double degrees, double max_speed=1, double end_speed=0)
- bool [drive_to_point](#) (double x, double y, vex::directionType dir, [Feedback](#) &feedback, double max_speed=1, double end_speed=0)
- bool [drive_to_point](#) (double x, double y, vex::directionType dir, double max_speed=1, double end_speed=0)
- bool [turn_to_heading](#) (double heading_deg, [Feedback](#) &feedback, double max_speed=1, double end_speed=0)
- bool [turn_to_heading](#) (double heading_deg, double max_speed=1, double end_speed=0)
- void [reset_auto](#) ()
- bool [pure_pursuit](#) ([PurePursuit::Path](#) path, directionType dir, [Feedback](#) &feedback, double max_speed=1, double end_speed=0)

Static Public Member Functions

- static double [modify_inputs](#) (double input, int power=2)

5.88.1 Detailed Description

[TankDrive](#) is a class to run a tank drive system. A tank drive system, sometimes called differential drive, has a motor (or group of synchronized motors) on the left and right side

5.88.2 Member Enumeration Documentation

BrakeType

```
enum class TankDrive::BrakeType [strong]
```

Enumerator

None	just send 0 volts to the motors
ZeroVelocity	try to bring the robot to rest. But don't try to hold position
Smart	bring the robot to rest and once it's stopped, try to hold that position

5.88.3 Constructor & Destructor Documentation**TankDrive()**

```
TankDrive::TankDrive (
    motor_group & left_motors,
    motor_group & right_motors,
    robot_specs_t & config,
    OdometryBase * odom = NULL)
```

Create the [TankDrive](#) Object

Parameters

<i>left_motors</i>	left side drive motors
<i>right_motors</i>	right side drive motors
<i>config</i>	the configuration specification defining physical dimensions about the robot. See robot_specs_t for more info
<i>odom</i>	an odometry system to track position and rotation. this is necessary to execute autonomous paths

5.88.4 Member Function Documentation**drive_arcade()**

```
void TankDrive::drive_arcade (
    double forward_back,
    double left_right,
    int power = 1,
    BrakeType bt = BrakeType::None)
```

Drive the robot using arcade style controls. forward_back controls the linear motion, left_right controls the turning.

forward_back and left_right are in "percent": -1.0 -> 1.0

Parameters

<i>forward_back</i>	the percent to move forward or backward
<i>left_right</i>	the percent to turn left or right
<i>power</i>	modifies the input velocities $\text{left}^{\text{power}}$, $\text{right}^{\text{power}}$
<i>bt</i>	breaktype. What to do if the driver lets go of the sticks

Drive the robot using arcade style controls. forward_back controls the linear motion, left_right controls the turning.

left_motors and right_motors are in "percent": -1.0 -> 1.0

drive_forward() [1/2]

```
bool TankDrive::drive_forward (
    double inches,
    directionType dir,
    double max_speed = 1,
    double end_speed = 0)
```

Autonomously drive the robot forward a certain distance

Parameters

<i>inches</i>	degrees by which we will turn relative to the robot (+) turns ccw, (-) turns cw
<i>dir</i>	the direction we want to travel forward and backward
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Autonomously drive the robot forward a certain distance

Parameters

<i>inches</i>	degrees by which we will turn relative to the robot (+) turns ccw, (-) turns cw
<i>dir</i>	the direction we want to travel forward and backward
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

true if we have finished driving to our point

drive_forward() [2/2]

```
bool TankDrive::drive_forward (
    double inches,
    directionType dir,
    Feedback & feedback,
    double max_speed = 1,
    double end_speed = 0)
```

Use odometry to drive forward a certain distance using a custom feedback controller

Returns whether or not the robot has reached it's destination.

Parameters

<i>inches</i>	the distance to drive forward
<i>dir</i>	the direction we want to travel forward and backward

<i>feedback</i>	the custom feedback controller we will use to travel. controls the rate at which we accelerate and drive.
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

true when we have reached our target distance

Use odometry to drive forward a certain distance using a custom feedback controller

Returns whether or not the robot has reached it's destination.

Parameters

<i>inches</i>	the distance to drive forward
<i>dir</i>	the direction we want to travel forward and backward
<i>feedback</i>	the custom feedback controller we will use to travel. controls the rate at which we accelerate and drive.
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

drive_tank()

```
void TankDrive::drive_tank (
    double left,
    double right,
    int power = 1,
    BrakeType bt = BrakeType::None)
```

Drive the robot using differential style controls. left_motors controls the left motors, right_motors controls the right motors.

left_motors and right_motors are in "percent": -1.0 -> 1.0

Parameters

<i>left</i>	the percent to run the left motors
<i>right</i>	the percent to run the right motors
<i>power</i>	modifies the input velocities $\text{left}^{\text{power}}$, $\text{right}^{\text{power}}$
<i>bt</i>	breaktype. What to do if the driver lets go of the sticks

drive_tank_raw()

```
void TankDrive::drive_tank_raw (
    double left,
    double right)
```

Generated by Doxygen

Drive the robot raw-ly

Parameters

<i>left</i>	the percent to run the left motors (-1, 1)
<i>right</i>	the percent to run the right motors (-1, 1)

drive_to_point() [1/2]

```
bool TankDrive::drive_to_point (
    double x,
    double y,
    vex::directionType dir,
    double max_speed = 1,
    double end_speed = 0)
```

Use odometry to automatically drive the robot to a point on the field. X and Y is the final point we want the robot. Here we use the default feedback controller from the drive_sys

Returns whether or not the robot has reached it's destination.

Parameters

<i>x</i>	the x position of the target
<i>y</i>	the y position of the target
<i>dir</i>	the direction we want to travel forward and backward
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Use odometry to automatically drive the robot to a point on the field. X and Y is the final point we want the robot. Here we use the default feedback controller from the drive_sys

Returns whether or not the robot has reached it's destination.

Parameters

<i>x</i>	the x position of the target
<i>y</i>	the y position of the target
<i>dir</i>	the direction we want to travel forward and backward
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

true if we have reached our target point

drive_to_point() [2/2]

```
bool TankDrive::drive_to_point (
    double x,
    double y,
    vex::directionType dir,
    Feedback & feedback,
    double max_speed = 1,
    double end_speed = 0)
```

Use odometry to automatically drive the robot to a point on the field. X and Y is the final point we want the robot.

Returns whether or not the robot has reached it's destination.

Parameters

<i>x</i>	the x position of the target
<i>y</i>	the y position of the target
<i>dir</i>	the direction we want to travel forward and backward
<i>feedback</i>	the feedback controller we will use to travel. controls the rate at which we accelerate and drive.
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Use odometry to automatically drive the robot to a point on the field. X and Y is the final point we want the robot.

Returns whether or not the robot has reached it's destination.

Parameters

<i>x</i>	the x position of the target
<i>y</i>	the y position of the target
<i>dir</i>	the direction we want to travel forward and backward
<i>feedback</i>	the feedback controller we will use to travel. controls the rate at which we accelerate and drive.
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

true if we have reached our target point

get_position()

```
Pose2d TankDrive::get_position ()
```

Returns the Robot position as a [Pose2d](#)

modify_inputs()

Generated by Doxygen

```
double TankDrive::modify_inputs (
    double input,
    int power = 2) [static]
```

Parameters

<i>input</i>	the input before modification
<i>power</i>	the power to raise input to

Returns

$\text{input}^{\text{power}}$ (accounts for negative inputs and odd numbered powers)

Modify the inputs from the controller by squaring / cubing, etc Allows for better control of the robot at slower speeds

Parameters

<i>input</i>	the input signal -1 -> 1
<i>power</i>	the power to raise the signal to

Returns

$\text{input}^{\text{power}}$ accounting for any sign issues that would arise with this naive solution

pure_pursuit()

```
bool TankDrive::pure_pursuit (
    PurePursuit::Path path,
    directionType dir,
    Feedback & feedback,
    double max_speed = 1,
    double end_speed = 0)
```

Drive the robot autonomously using a pure-pursuit algorithm - Input path with a set of waypoints - the robot will attempt to follow the points while cutting corners (radius) to save time (compared to stop / turn / start)

Parameters

<i>path</i>	The list of coordinates to follow, in order
<i>dir</i>	Run the bot forwards or backwards
<i>feedback</i>	The feedback controller determining speed
<i>max_speed</i>	Limit the speed of the robot (for pid / pidff feedbacks)
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

True when the path is complete

Drive the robot autonomously using a pure-pursuit algorithm - Input path with a set of waypoints - the robot will attempt to follow the points while cutting corners (radius) to save time (compared to stop / turn / start)

Parameters

<i>path</i>	The list of coordinates to follow, in order
<i>dir</i>	Run the bot forwards or backwards
<i>feedback</i>	The feedback controller determining speed
<i>max_speed</i>	Limit the speed of the robot (for pid / pidff feedbacks)

Returns

True when the path is complete

reset_auto()

```
void TankDrive::reset_auto ()
```

Reset the initialization for autonomous drive functions

stop()

```
void TankDrive::stop ()
```

Stops rotation of all the motors using their "brake mode"

turn_degrees() [1/2]

```
bool TankDrive::turn_degrees (
    double degrees,
    double max_speed = 1,
    double end_speed = 0)
```

Autonomously turn the robot X degrees to counterclockwise (negative for clockwise), with a maximum motor speed of percent_speed (-1.0 -> 1.0)

Uses the default turning feedback of the drive system.

Parameters

<i>degrees</i>	degrees by which we will turn relative to the robot (+) turns ccw, (-) turns cw
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Autonomously turn the robot X degrees to counterclockwise (negative for clockwise), with a maximum motor speed of percent_speed (-1.0 -> 1.0)

Uses the default turning feedback of the drive system.

Parameters

<i>degrees</i>	degrees by which we will turn relative to the robot (+) turns ccw, (-) turns cw
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

true if we turned te target number of degrees

turn_degrees() [2/2]

```
bool TankDrive::turn_degrees (
    double degrees,
    Feedback & feedback,
    double max_speed = 1,
    double end_speed = 0)
```

Autonomously turn the robot X degrees counterclockwise (negative for clockwise), with a maximum motor speed of percent_speed (-1.0 -> 1.0)

Uses PID + Feedforward for it's control.

Parameters

<i>degrees</i>	degrees by which we will turn relative to the robot (+) turns ccw, (-) turns cw
<i>feedback</i>	the feedback controller we will use to travel. controls the rate at which we accelerate and drive.
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power

Autonomously turn the robot X degrees to counterclockwise (negative for clockwise), with a maximum motor speed of percent_speed (-1.0 -> 1.0)

Uses the specified feedback for it's control.

Parameters

<i>degrees</i>	degrees by which we will turn relative to the robot (+) turns ccw, (-) turns cw
<i>feedback</i>	the feedback controller we will use to travel. controls the rate at which we accelerate and drive.
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

true if we have turned our target number of degrees

turn_to_heading() [1/2]

```
bool TankDrive::turn_to_heading (
    double heading_deg,
    double max_speed = 1,
    double end_speed = 0)
```

Parameters

<i>heading_deg</i>	the heading to which we will turn
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Turn the robot in place to an exact heading relative to the field. 0 is forward. Uses the default turn feedback of the drive system

Parameters

<i>heading_deg</i>	the heading to which we will turn
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

true if we have reached our target heading

turn_to_heading() [2/2]

```
bool TankDrive::turn_to_heading (
    double heading_deg,
    Feedback & feedback,
    double max_speed = 1,
    double end_speed = 0)
```

Turn the robot in place to an exact heading relative to the field. 0 is forward.

Parameters

<i>heading_deg</i>	the heading to which we will turn
<i>feedback</i>	the feedback controller we will use to travel. controls the rate at which we accelerate and drive.
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Turn the robot in place to an exact heading relative to the field. 0 is forward.

Parameters

<i>heading_deg</i>	the heading to which we will turn
<i>feedback</i>	the feedback controller we will use to travel. controls the rate at which we accelerate and drive.
<i>max_speed</i>	the maximum percentage of robot speed at which the robot will travel. 1 = full power
<i>end_speed</i>	the movement profile will attempt to reach this velocity by its completion

Returns

true if we have reached our target heading

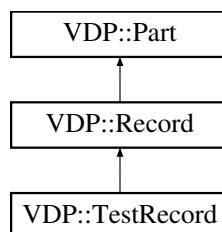
The documentation for this class was generated from the following files:

- tank_drive.h
- tank_drive.cpp

5.89 VDP::TestRecord Class Reference

```
#include <builtins.hpp>
```

Inheritance diagram for VDP::TestRecord:

**Public Member Functions**

- [TestRecord](#) (std::string name, double test_float, int64_t test_int64)

Public Member Functions inherited from [VDP::Record](#)

- [Record](#) (std::string name)
- [Record](#) (std::string name, const std::vector< [Part](#) * > &parts)
- [Record](#) (std::string name, std::vector< PartPtr > parts)
- [Record](#) (std::string name, PacketReader &reader)
- void [set_fields](#) (std::vector< PartPtr > fields)
- void [read_data_from_message](#) (PacketReader &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Additional Inherited Members**Protected Member Functions inherited from [VDP::Record](#)**

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.89.1 Detailed Description

Defines a record for testing purposes, currently tests a float and int64

5.89.2 Constructor & Destructor Documentation

TestRecord()

```
VDP::TestRecord::TestRecord (
    std::string name,
    double test_float,
    int64_t test_int64)
```

Defines a record for testing purposes, currently tests a float and int64

=

Defines a record for testing purposes, currently tests a float and int64

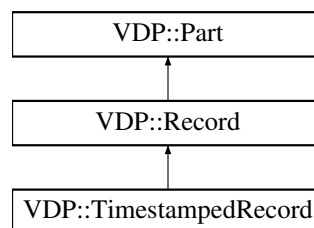
The documentation for this class was generated from the following files:

- builtins.hpp
- builtins.cpp

5.90 VDP::TimestampedRecord Class Reference

```
#include <builtins.hpp>
```

Inheritance diagram for VDP::TimestampedRecord:



Public Member Functions

- [TimestampedRecord](#) (std::string name, [Part](#) *data)
- void [fetch](#) ()

Public Member Functions inherited from [VDP::Record](#)

- [Record](#) (std::string name)
- [Record](#) (std::string name, const std::vector< [Part](#) * > &parts)
- [Record](#) (std::string name, std::vector< [PartPtr](#) > parts)
- [Record](#) (std::string name, [PacketReader](#) &reader)
- void [set_fields](#) (std::vector< [PartPtr](#) > fields)
- void [read_data_from_message](#) ([PacketReader](#) &reader) override

Public Member Functions inherited from [VDP::Part](#)

- [Part](#) (std::string name)
- std::string [pretty_print](#) () const
- std::string [pretty_print_data](#) () const

Additional Inherited Members

Protected Member Functions inherited from [VDP::Record](#)

- void [write_schema](#) (PacketWriter &sofar) const override
- void [write_message](#) (PacketWriter &sofar) const override

5.90.1 Detailed Description

Defines a record that holds a timestamp and data

5.90.2 Constructor & Destructor Documentation

TimestampedRecord()

```
VDP::TimestampedRecord::TimestampedRecord (  
    std::string name,  
    Part * data)
```

Creates a record that contains a Float of a timestamp [Part](#) of data

Parameters

<i>name</i>	the name of the record to create
<i>data</i>	the data to put into the record

5.90.3 Member Function Documentation

fetch()

```
void VDP::TimestampedRecord::fetch () [virtual]
```

sets the data that the Timestamp Parts hold

Reimplemented from [VDP::Record](#).

The documentation for this class was generated from the following files:

- builtins.hpp
- builtins.cpp

5.91 tracking_wheel_cfg_t Struct Reference

```
#include <odometry_nwheel.h>
```

Public Attributes

- double `x`
- double `y`
- double `theta_rad`
- double `radius`

5.91.1 Detailed Description

OdometryNWheel

This class handles the code for an N-pod odometry setup, where there are N <WHEELS> free spinning omni wheels (dead wheels) placed in any known configuration on the robot.

Example of a possible wheel configuration:

```
+Y ----- ^ | === ||||| O ||||| === || ----- | +-----> + X
```

Where O is the center of rotation. The robot will monitor the changes in rotation of these wheels, use this to calculate a pose delta, then integrate the deltas over time to determine the robot's position.

This is a "set and forget" class, meaning once the object is created, the robot will immediately begin tracking it's movement in the background.

<https://rit.enterprise.slack.com/files/U04112Y5RB6/F080M01KPA5/predictperpindiculars2.pdf> 2024-2025 Notebook: Entries/Software Entries/Localization/N-Pod Odometry

Author

Jack Cammarata, Richie Sommers

Date

Nov 14 2024 `tracking_wheel_cfg_t` holds all the specifications for a single tracking wheel The units for x, y, and radius will determine the units of the position estimate

5.91.2 Member Data Documentation

radius

```
double tracking_wheel_cfg_t::radius
```

radius of the wheel

theta_rad

```
double tracking_wheel_cfg_t::theta_rad
```

angle between wheel direction and x axis in the robot frame

x

```
double tracking_wheel_cfg_t::x
```

x position of the center of the wheel

y

```
double tracking_wheel_cfg_t::y
```

y position of the center of the wheel

The documentation for this struct was generated from the following file:

- `odometry_nwheel.h`

5.92 Transform2d Class Reference

```
#include <transform2d.h>
```

Public Member Functions

- constexpr [Transform2d](#) ()
- [Transform2d](#) (const [Translation2d](#) &translation, const [Rotation2d](#) &rotation)
- [Transform2d](#) (const double &x, const double &y, const [Rotation2d](#) &rotation)
- [Transform2d](#) (const double &x, const double &y, const double &radians)
- [Transform2d](#) (const [Translation2d](#) &translation, const double &radians)
- [Transform2d](#) (const Eigen::Vector3d &transform_vector)
- [Transform2d](#) (const [Pose2d](#) &start, const [Pose2d](#) &end)
- [Translation2d](#) translation () const
- double x () const
- double y () const
- [Rotation2d](#) rotation () const
- [Transform2d](#) inverse () const
- [Transform2d](#) operator* (const double &scalar) const
- [Transform2d](#) operator/ (const double &scalar) const
- [Transform2d](#) operator- () const
- bool operator== (const [Transform2d](#) &other) const

Friends

- std::ostream & operator<< (std::ostream &os, const [Transform2d](#) &transform)

5.92.1 Detailed Description

Class representing a transformation of a pose2d, or a linear difference between the components of poses.

Assumes conventional cartesian coordinate system: Looking down at the coordinate plane, +X is right +Y is up
+Theta is counterclockwise

5.92.2 Constructor & Destructor Documentation

Transform2d() [1/7]

```
Transform2d::Transform2d () [constexpr]
```

Default Constructor for [Transform2d](#)

Transform2d() [2/7]

```
Transform2d::Transform2d (
    const Translation2d & translation,
    const Rotation2d & rotation)
```

Constructs a transform given translation and rotation components.

Parameters

<i>translation</i>	the translational component of the transform.
<i>rotation</i>	the rotational component of the transform.

Transform2d() [3/7]

```
Transform2d::Transform2d (
    const double & x,
    const double & y,
    const Rotation2d & rotation)
```

Constructs a transform given translation and rotation components.

Parameters

<i>x</i>	the x component of the transform.
<i>y</i>	the y component of the transform.
<i>rotation</i>	the rotational component of the transform.

Transform2d() [4/7]

```
Transform2d::Transform2d (
    const double & x,
    const double & y,
    const double & radians)
```

Constructs a transform given translation and rotation components

Parameters

<i>x</i>	the x component of the transform.
<i>y</i>	the y component of the transform.
<i>radians</i>	the rotational component of the transform in radians.

Transform2d() [5/7]

```
Transform2d::Transform2d (  
    const Translation2d & translation,  
    const double & radians)
```

Constructs a transform given translation and rotation components.

Parameters

<i>translation</i>	the translational component of the transform.
<i>radians</i>	the rotational component of the transform in radians.

Transform2d() [6/7]

```
Transform2d::Transform2d (  
    const Eigen::Vector3d & transform_vector)
```

Constructs a transform given translation and rotation components given as a vector.

Parameters

<i>transform_vector</i>	vector of the form [x, y, theta]
-------------------------	----------------------------------

Transform2d() [7/7]

```
Transform2d::Transform2d (  
    const Pose2d & start,  
    const Pose2d & end)
```

Constructs a transform given translation and rotation components.

Parameters

<i>translation</i>	the translational component of the transform.
<i>rotation</i>	the rotational component of the transform.

5.92.3 Member Function Documentation

inverse()

```
Transform2d Transform2d::inverse () const
```

Inverts the transform.

Returns

the inverted transform.

operator*()

```
Transform2d Transform2d::operator* (  
    const double & scalar) const
```

Multiplies this transform by a scalar.

Parameters

<i>scalar</i>	the scalar to multiply this transform by.
---------------	---

operator-()

```
Transform2d Transform2d::operator- () const
```

Inverts the transform.

Returns

the inverted transform.

operator/()

```
Transform2d Transform2d::operator/ (  
    const double & scalar) const
```

Divides this transform by a scalar.

Parameters

<i>scalar</i>	the scalar to divide this transform by.
---------------	---

operator==(

```
Generated by Doxygen  
Transform2d Transform2d::operator== (  
    const Transform2d & other) const
```

Compares this to another transform.

Parameters

<i>other</i>	the other transform to compare to.
--------------	------------------------------------

Returns

true if the components are within 1e-9 of each other.

rotation()

```
Rotation2d Transform2d::rotation () const
```

Returns the rotational component of the transform.

Returns

the rotational component of the transform.

translation()

```
Translation2d Transform2d::translation () const
```

Returns the translational component of the transform.

Returns

the translational component of the transform.

x()

```
double Transform2d::x () const
```

Returns the x component of the transform.

Returns

the x component of the transform.

y()

```
double Transform2d::y () const
```

Returns the y component of the transform.

Returns

the y component of the transform.

5.92.4 Friends And Related Symbol Documentation

operator<<

```
std::ostream & operator<< (
    std::ostream & os,
    const Transform2d & transform) [friend]
```

Sends a transform to an output stream. Ex. `std::cout << transform;`

prints "Transform2d[dx: (value), dy: (value), drad: (radians), ddeg: (degrees)]"

Sends a transform to an output stream. Ex. `std::cout << transform;`

prints "Transform2d[x: (value), y: (value), rad: (radians), deg: (degrees)]"

The documentation for this class was generated from the following files:

- transform2d.h
- transform2d.cpp

5.93 Translation2d Class Reference

```
#include <translation2d.h>
```

Public Member Functions

- constexpr [Translation2d](#) ()
- [Translation2d](#) (const double &x, const double &y)
- [Translation2d](#) (const Eigen::Vector2d &vector)
- [Translation2d](#) (const double &r, const [Rotation2d](#) &theta)
- double [x](#) () const
- void [setX](#) (double x)
- double [y](#) () const
- void [setY](#) (double y)
- [Rotation2d](#) [theta](#) () const
- Eigen::Vector2d [as_vector](#) () const
- double [norm](#) () const
- [Translation2d](#) [normalize](#) () const
- double [distance](#) (const [Translation2d](#) &other) const
- [Translation2d](#) [rotate_by](#) (const [Rotation2d](#) &rotation) const
- [Translation2d](#) [rotate_around](#) (const [Translation2d](#) &other, const [Rotation2d](#) &rotation) const
- [Translation2d](#) [operator+](#) (const [Translation2d](#) &other) const
- [Translation2d](#) [operator-](#) (const [Translation2d](#) &other) const
- [Translation2d](#) [operator-](#) () const
- [Translation2d](#) [operator*](#) (const double &scalar) const
- [Translation2d](#) [operator/](#) (const double &scalar) const
- double [operator*](#) (const [Translation2d](#) &other) const
- bool [operator==](#) (const [Translation2d](#) &other) const

Friends

- `std::ostream & operator<< (std::ostream &os, const Translation2d &translation)`

5.93.1 Detailed Description

Class representing a point in 2d space with x and y coordinates.

Assumes conventional cartesian coordinate system: Looking down at the coordinate plane, +X is right +Y is up +Theta is counterclockwise

5.93.2 Constructor & Destructor Documentation

Translation2d() [1/4]

```
Translation2d::Translation2d () [inline], [constexpr]
```

Default Constructor for [Translation2d](#)

Translation2d() [2/4]

```
Translation2d::Translation2d (  
    const double & x,  
    const double & y)
```

Constructs a [Translation2d](#) with the given x and y values.

Parameters

<i>x</i>	The x component of the translation.
<i>y</i>	The y component of the translation.

Translation2d() [3/4]

```
Translation2d::Translation2d (  
    const Eigen::Vector2d & vector)
```

Constructs a [Translation2d](#) with the values from the given vector.

Parameters

<i>vector</i>	The vector whose values will be used.
---------------	---------------------------------------

Translation2d() [4/4]

```
Translation2d::Translation2d (  
    const double & r,  
    const Rotation2d & theta)
```

Constructs a [Translation2d](#) given polar coordinates of the form (r, theta).

Parameters

<i>r</i>	The radius (magnitude) of the vector.
<i>theta</i>	The angle (direction) of the vector.

5.93.3 Member Function Documentation**as_vector()**

```
Eigen::Vector2d Translation2d::as_vector () const
```

Returns the vector as an Eigen::Vector2d.

Returns

Eigen::Vector2d with the same values as the translation.

distance()

```
double Translation2d::distance (
    const Translation2d & other) const
```

Returns the distance between two translations.

Returns

the distance between two translations.

norm()

```
double Translation2d::norm () const
```

Returns the norm/radius/magnitude/distance from origin.

Returns

the norm of the translation.

normalize()

```
Translation2d Translation2d::normalize () const
```

returns a translation as if it were a vector with a magnitude of 1

Returns

the norm of the translation.

Generated by Doxygen

Returns a translation so that it has a vector magnitude of 1

Parameters

<i>scalar</i>	the scalar to multiply by.
---------------	----------------------------

Returns

this translation multiplied by a scalar.

operator*() [2/2]

```
double Translation2d::operator* (
    const Translation2d & other) const
```

Returns the dot product of two translations.

$[scalar] = [x][otherx] + [y][othery]$

Parameters

<i>other</i>	the other translation to find the dot product with.
--------------	---

Returns

the scalar valued dot product.

operator+()

```
Translation2d Translation2d::operator+ (
    const Translation2d & other) const
```

Returns the sum of two translations.

$[x] = [x] + [otherx]; [y] = [y] + [othery];$

Parameters

<i>other</i>	the other translation to add to this translation.
--------------	---

Returns

the sum of the two translations.

operator-() [1/2]

```
Translation2d Translation2d::operator- () const
```

Returns the inverse of this translation. Equivalent to flipping the vector across the origin.

$$[x] = [-x] \quad [y] = [-y]$$
Returns

the inverse of this translation.

operator-() [2/2]

```
Translation2d Translation2d::operator- (
    const Translation2d & other) const
```

Returns the difference of two translations.

$$[x] = [x] - [otherx] \quad [y] = [y] - [othery]$$
Parameters

<i>other</i>	the translation to subtract from this translation.
--------------	--

Returns

the difference of the two translations.

operator/()

```
Translation2d Translation2d::operator/ (
    const double & scalar) const
```

Returns this translation divided by a scalar.

$$[x] = [x] / [scalar] \quad [y] = [y] / [scalar]$$
Parameters

<i>scalar</i>	the scalar to divide by.
---------------	--------------------------

Returns

this translation divided by a scalar.

operator==(

Generated by Doxygen

```
bool Translation2d::operator== (
    const Translation2d & other) const
```

Parameters

<i>other</i>	the translation to compare to.
--------------	--------------------------------

Returns

whether the two translations are equal.

rotate_around()

```
Translation2d Translation2d::rotate_around (  
    const Translation2d & other,  
    const Rotation2d & rotation) const
```

Applies a rotation to this translation around another given point.

$[x] = [\cos, -\sin][x - \text{otherx}] + [\text{otherx}]$ $[y] = [\sin, \cos][y - \text{othersy}] + [\text{othersy}]$

Parameters

<i>other</i>	the center of rotation.
<i>rotation</i>	the angle amount the translation will be rotated.

Returns

the translation that has been rotated.

rotate_by()

```
Translation2d Translation2d::rotate_by (  
    const Rotation2d & rotation) const
```

Applies a rotation to this translation around the origin.

Equivalent to multiplying a vector by a rotation matrix: $x = [\cos, -\sin][x]$ $y = [\sin, \cos][y]$

Parameters

<i>rotation</i>	the angle amount the translation will be rotated.
-----------------	---

Returns

the new translation that has been rotated around the origin.

setX()

```
void Translation2d::setX (  
    double x)
```

Sets the x value of the translation.

setY()

```
void Translation2d::setY (  
    double y)
```

Sets the y value of the translation.

theta()

```
Rotation2d Translation2d::theta () const
```

Returns the angle of the translation.

Returns

the angle of the translation.

x()

```
double Translation2d::x () const
```

Returns the x value of the translation.

Returns

the x value of the translation.

y()

```
double Translation2d::y () const
```

Returns the y value of the translation.

Returns

the y value of the translation.

5.93.4 Friends And Related Symbol Documentation

operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Translation2d & translation) [friend]
```

Sends a translation to an output stream. Ex. `std::cout << translation;`

prints "Translation2d[x: (value), y: (value)]"

Sends a translation to an output stream. Ex. `std::cout << translation;`

prints "Translation2d[x: (value), y: (value), rad: (radians), deg: (degrees)]"

The documentation for this class was generated from the following files:

- translation2d.h
- translation2d.cpp

5.94 TrapezoidProfile Class Reference

```
#include <trapezoid_profile.h>
```

Public Member Functions

- [TrapezoidProfile](#) (const double &x_initial, const double &x_target, const double &v_max, const double &accel, const double &decel)
- `motion_t` [calculate](#) (double t)
- double [total_time](#) ()

5.94.1 Detailed Description

Class representing a trapezoidal motion profile. This consists of either two or three stages. First, an acceleration stage where the velocity increases to a maximum. Then a cruise stage where the velocity is constant, then a deceleration stage where the velocity decreases to zero.

This implementation allows for different acceleration and deceleration rates.

This is best used with LQR, as it tracks both velocity and position at the same time, however it can also be used with [PID](#) and feedforward.

Author

Jack Cammarata

Date

3/28/2025

5.94.2 Constructor & Destructor Documentation

Parameters

<i>x_initial</i>	The initial position.
<i>x_target</i>	The target position.
<i>v_max</i>	The maximum velocity.
<i>accel</i>	The acceleration.
<i>decel</i>	The deceleration.

5.94.3 Member Function Documentation**calculate()**

```
motion_t TrapezoidProfile::calculate (  
    double t)
```

Calculate the state along the motion profile at some given time.

Parameters

<i>t</i>	The time in seconds.
----------	----------------------

Returns

the state.

Calculate the state along the motion profile after some given time.

Parameters

<i>t</i>	The time in seconds.
----------	----------------------

Returns

the state.

total_time()

```
double TrapezoidProfile::total_time ()
```

Returns the total time that the motion profile takes to complete.

Returns

the total time that the motion profile takes to complete.

The documentation for this class was generated from the following files:

- trapezoid_profile.h
- trapezoid_profile.cpp

5.95 TurnDegreesCommand Class Reference

```
#include <drive_commands.h>
```

Public Member Functions

- [TurnDegreesCommand](#) ([TankDrive](#) &drive_sys, [Feedback](#) &feedback, double degrees, double max_speed=1, double end_speed=0)
- bool [run](#) () override
- void [on_timeout](#) () override

5.95.1 Detailed Description

AutoCommand wrapper class for the turn_degrees function in the [TankDrive](#) class

5.95.2 Constructor & Destructor Documentation

TurnDegreesCommand()

```
TurnDegreesCommand::TurnDegreesCommand (
    TankDrive & drive_sys,
    Feedback & feedback,
    double degrees,
    double max_speed = 1,
    double end_speed = 0)
```

Construct a [TurnDegreesCommand](#) Command

Parameters

<i>drive_sys</i>	the drive system we are commanding
<i>feedback</i>	the feedback controller we are using to execute the turn
<i>degrees</i>	how many degrees to rotate
<i>max_speed</i>	0 -> 1 percentage of the drive systems speed to drive at

5.95.3 Member Function Documentation

on_timeout()

```
void TurnDegreesCommand::on_timeout () [override]
```

Cleans up drive system if we time out before finishing

reset the drive system if we timeout

run()

```
bool TurnDegreesCommand::run () [override]
```

Run turn_degrees Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- drive_commands.h
- drive_commands.cpp

5.96 TurnToHeadingCommand Class Reference

```
#include <drive_commands.h>
```

Public Member Functions

- [TurnToHeadingCommand](#) ([TankDrive](#) &drive_sys, [Feedback](#) &feedback, double heading_deg, double speed=1, double end_speed=0)
- bool [run](#) () override
- void [on_timeout](#) () override

5.96.1 Detailed Description

AutoCommand wrapper class for the turn_to_heading() function in the [TankDrive](#) class

5.96.2 Constructor & Destructor Documentation**TurnToHeadingCommand()**

```
TurnToHeadingCommand::TurnToHeadingCommand (
    TankDrive & drive_sys,
    Feedback & feedback,
    double heading_deg,
    double max_speed = 1,
    double end_speed = 0)
```

Construct a [TurnToHeadingCommand](#) Command

Parameters

<i>drive_sys</i>	the drive system we are commanding
------------------	------------------------------------

<i>feedback</i>	the feedback controller we are using to execute the drive
<i>heading_deg</i>	the heading to turn to in degrees
<i>max_speed</i>	0 -> 1 percentage of the drive systems speed to drive at

5.96.3 Member Function Documentation

on_timeout()

```
void TurnToHeadingCommand::on_timeout () [override]
```

Cleans up drive system if we time out before finishing

reset the drive system if we don't hit our target

run()

```
bool TurnToHeadingCommand::run () [override]
```

Run turn_to_heading Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- drive_commands.h
- drive_commands.cpp

5.97 Twist2d Class Reference

```
#include <twist2d.h>
```

Public Member Functions

- constexpr [Twist2d](#) ()
- [Twist2d](#) (const double &dx, const double &dy, const double &dtheta)
- [Twist2d](#) (const Eigen::Vector3d &twist_vector)
- double [dx](#) () const
- double [dy](#) () const
- double [dtheta](#) () const
- bool [operator==](#) (const [Twist2d](#) &other) const
- [Twist2d operator*](#) (const double &scalar) const
- [Twist2d operator/](#) (const double &scalar) const

Friends

- `std::ostream & operator<< (std::ostream &os, const Twist2d &twist)`

5.97.1 Detailed Description

Class representing a difference between two poses, more specifically a distance along an arc from a pose.

Assumes conventional cartesian coordinate system: Looking down at the coordinate plane, +X is right +Y is up +Theta is counterclockwise

5.97.2 Constructor & Destructor Documentation

Twist2d() [1/3]

```
Twist2d::Twist2d () [constexpr]
```

Default Constructor for [Twist2d](#)

Twist2d() [2/3]

```
Twist2d::Twist2d (
    const double & dx,
    const double & dy,
    const double & dtheta)
```

Constructs a twist with given translation and angle deltas.

Parameters

<i>dx</i>	the linear dx component.
<i>dy</i>	the linear dy component.
<i>dtheta</i>	the angular dtheta component.

Twist2d() [3/3]

```
Twist2d::Twist2d (
    const Eigen::Vector3d & twist_vector)
```

Constructs a twist with given translation and angle deltas.

Parameters

<i>twist_vector</i>	vector of the form [dx, dy, dtheta]
---------------------	-------------------------------------

5.97.3 Member Function Documentation

dtheta()

```
double Twist2d::dtheta () const
```

Returns the angular dtheta component.

Returns

the angular dtheta component.

dx()

```
double Twist2d::dx () const
```

Returns the linear dx component.

Returns

the linear dx component.

dy()

```
double Twist2d::dy () const
```

Returns the linear dy component.

Returns

the linear dy component.

operator*()

```
Twist2d Twist2d::operator* (
    const double & scalar) const
```

Multiplies this twist by a scalar.

Parameters

<i>scalar</i>	the scalar value to multiply by.
---------------	----------------------------------

operator/()

```
Twist2d Twist2d::operator/ (
    const double & scalar) const
```

Divides this twist by a scalar.

Parameters

<i>scalar</i>	the scalar value to divide by.
---------------	--------------------------------

operator==()

```
bool Twist2d::operator== (
    const Twist2d & other) const
```

Compares this to another twist.

Parameters

<i>other</i>	the other twist to compare to.
--------------	--------------------------------

Returns

true if each of the components are within 1e-9 of each other.

5.97.4 Friends And Related Symbol Documentation**operator<<**

```
std::ostream & operator<< (
    std::ostream & os,
    const Twist2d & twist) [friend]
```

Sends a twist to an output stream. Ex. `std::cout << twist;`

prints "Twist2d[dx: (value), dy: (value), drad: (radians)]"

Sends a twist to an output stream. Ex. `std::cout << twist;`

prints "Twist2d[x: (value), y: (value), rad: (radians), deg: (degrees)]"

The documentation for this class was generated from the following files:

- twist2d.h
- twist2d.cpp

5.98 UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS > Class Template Reference

```
#include <unscented_kalman_filter.h>
```

Public Member Functions

- [UnscentedKalmanFilter](#) (const std::function< StateVector(const StateVector &, const InputVector &)> &f, const std::function< OutputVector(const StateVector &, const InputVector &)> &h, const WithInputIntegrator &integrator, const StateVector &state_stddevs, const OutputVector &measurement_stddevs)
- [UnscentedKalmanFilter](#) (const std::function< StateVector(const StateVector &, const InputVector &)> &f, const std::function< OutputVector(const StateVector &, const InputVector &)> &h, const WithInputIntegrator &integrator, const StateVector &state_stddevs, const OutputVector &measurement_stddevs, const std::function< StateVector(const EMat< STATES, NUM_SIGMAS > &, const EVec< NUM_SIGMAS > &)> &mean_func_X, const std::function< OutputVector(const EMat< OUTPUTS, NUM_SIGMAS > &, const EVec< NUM_SIGMAS > &)> &mean_func_Y, const std::function< StateVector(const StateVector &, const StateVector &)> &residual_func_X, const std::function< OutputVector(const OutputVector &, const OutputVector &)> &residual_func_Y, const std::function< StateVector(const StateVector &, const StateVector &)> &add_func_X)
- StateMatrix [S](#) () const
- double [S](#) (int i, int j) const
- void [set_S](#) (const StateMatrix &S)
- StateMatrix [P](#) () const
- void [set_P](#) (const StateMatrix &P)
- StateVector [xhat](#) () const
- double [xhat](#) (int i) const
- void [set_xhat](#) (const StateVector &xhat)
- void [set_xhat](#) (int i, double value)
- void [reset](#) ()
- void [predict](#) (const InputVector &u, double dt)
- void [correct](#) (const InputVector &u, const OutputVector &y)
- void [correct](#) (const InputVector &u, const OutputVector &y, const EVec< OUTPUTS > &measurement_stddevs)
- template<int ROWS>
void [correct](#) (const InputVector &u, const EVec< ROWS > &y, const std::function< EVec< ROWS >(const StateVector &, const InputVector &)> &h, const EVec< ROWS > &measurement_stddevs)
- template<int ROWS>
void [correct](#) (const InputVector &u, const EVec< ROWS > &y, const std::function< EVec< ROWS >(const StateVector &, const InputVector &)> &h, const EVec< ROWS > &measurement_stddevs, const std::function< EVec< ROWS >(const EMat< ROWS, NUM_SIGMAS > &, const EVec< NUM_SIGMAS > &)> &mean_func_Y, const std::function< EVec< ROWS >(const EVec< ROWS > &, const EVec< ROWS > &)> &residual_func_Y, const std::function< StateVector(const StateVector &, const StateVector &)> &residual_func_X, const std::function< StateVector(const StateVector &, const StateVector &)> &add_func_X)

5.98.1 Detailed Description

```
template<int STATES, int INPUTS, int OUTPUTS>
class UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >
```

Kalman filters combine predictions from a model and measurements to estimate a system's true state.

The Unscented Kalman [Filter](#) is a nonlinear estimator, meaning that the model used to predict how the state changes over time can be nonlinear. The model that determines the expected measurement given the current state can also be nonlinear.

At each timestep, sigma points are generated close to the mean, they are all propagated forward in time according to the nonlinear model. The Unscented Transform uses the propagated sigma points to compute the prior state and covariance.

When correcting the state and covariance with a measurement, sigma points are again generated, but are transformed into the measurement space using the measurement function. A Kalman gain matrix K is then computed, and used to update the state and covariance.

To read more about Kalman filters and the standard UKF read: <https://github.com/rlabbe/Kalman-and-Bayesian-Filters-in-Python>

This implementation is somewhat non-standard. The square-root form of the UKF (SR-UKF) is used, and the way the sigma points are generated is different than most implementations. The square-root form is used to ensure that the covariance matrix remains positive definite.

To learn more about the SR-UKF, and see the exact formulation that most of this implementation follows, read: <https://www.researchgate.net/publication/3908304>

The sigma points are not generated symmetrically around the mean, instead they are generated as vertices of a simplex. Using $N = \#$ of states, this method uses $N + 2$ sigma points instead of the standard $2N + 1$ sigma points. This reduces computation up to 50%. To learn more about this method, read: <https://www.sciencedirect.com/science/article/pii/S0888327020308190>

Template Parameters

<i>STATES</i>	Dimension of the state vector.
<i>INPUTS</i>	Dimension of the control input vector.
<i>OUTPUTS</i>	Dimension of the measurement vector.

5.98.2 Constructor & Destructor Documentation

UnscentedKalmanFilter() [1/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::UnscentedKalmanFilter (
    const std::function< StateVector(const StateVector &, const InputVector &)> & f,
    const std::function< OutputVector(const StateVector &, const InputVector &)> & h,
    const WithInputIntegrator & integrator,
    const StateVector & state_stddevs,
    const OutputVector & measurement_stddevs) [inline]
```

Constructs an Unscented Kalman [Filter](#).

Parameters

<i>f</i>	A vector valued function of x and u that returns the derivative of the state vector with respect to time.
<i>h</i>	A vector valued function of x and u that returns the expected measurement at the given state.
<i>integrator</i>	A function from "numerical_integration.h" that integrates a differential equation of the form $f(x, u)$.
<i>state_stddevs</i>	Standard deviations of the states in the model.
<i>measurement_stddevs</i>	Standard deviations of the measurements.

UnscentedKalmanFilter() [2/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::UnscentedKalmanFilter (
    const std::function< StateVector(const StateVector &, const InputVector &)> & f,
    const std::function< OutputVector(const StateVector &, const InputVector &)> & h,
    const WithInputIntegrator & integrator,
    const StateVector & state_stddevs,
    const OutputVector & measurement_stddevs,
    const std::function< StateVector(const EMat< STATES, NUM_SIGMAS > &, const EVec<
NUM_SIGMAS > &)> & mean_func_X,
    const std::function< OutputVector(const EMat< OUTPUTS, NUM_SIGMAS > &, const
EVec< NUM_SIGMAS > &)> & mean_func_Y,
    const std::function< StateVector(const StateVector &, const StateVector &)> &
residual_func_X,
    const std::function< OutputVector(const OutputVector &, const OutputVector &)> &
residual_func_Y,
    const std::function< StateVector(const StateVector &, const StateVector &)> &
add_func_X) [inline]
```

Constructs an Unscented Kalman [Filter](#) with custom mean, residual, and addition functions. The most common use for these functions is when you are estimating angles whose arithmetic operations need to be wrapped.

Parameters

<i>f</i>	A vector valued function of x and u that returns the derivative of the state vector with respect to time.
<i>h</i>	A vector valued function of x and u that returns the expected measurement at the given state.
<i>integrator</i>	A function from "numerical_integration.h" that integrates a differential equation of the form f(x, u).
<i>state_stddevs</i>	Standard deviations of the states in the model.
<i>measurement_stddevs</i>	Standard deviations of the measurements.
<i>mean_func_X</i>	A function that computes the mean of a matrix containing NUM_SIGMAS state sigma points with a set of weights for each.
<i>mean_func_Y</i>	A function that computes the mean of a matrix containing NUM_SIGMAS measurement sigma points with a set of weights for each.
<i>residual_func_X</i>	A function that computes the residual of two state vectors, usually by simple subtraction.
<i>residual_func_Y</i>	A function that computes the residual of two measurement vectors, usually by simple subtraction.
<i>add_func_X</i>	A function that adds two state vectors.

5.98.3 Member Function Documentation**correct()** [1/4]

```
template<int STATES, int INPUTS, int OUTPUTS>
template<int ROWS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::correct (
```

```

        const InputVector & u,
        const EVec< ROWS > & y,
        const std::function< EVec< ROWS >(const StateVector &, const InputVector &)> &
h,

        const EVec< ROWS > & measurement_stddevs) [inline]

```

Correct the state estimate using the measurements in *y*, a custom measurement function, and custom standard deviations. This is useful for when a different set of measurements are being applied.

Parameters

<i>u</i>	The control input used in the last predict step.
<i>y</i>	The vector of measurements.
<i>h</i>	A vector valued function of <i>x</i> and <i>u</i> that returns the expected measurement at the given state.
<i>measurement_stddevs</i>	The vector of standard deviations for each measurement to be used for this correct step.

correct() [2/4]

```

template<int STATES, int INPUTS, int OUTPUTS>
template<int ROWS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::correct (
    const InputVector & u,
    const EVec< ROWS > & y,
    const std::function< EVec< ROWS >(const StateVector &, const InputVector &)> &
h,

    const EVec< ROWS > measurement_stddevs,
    const std::function< EVec< ROWS >(const EMat< ROWS, NUM_SIGMAS > &, const EVec<
NUM_SIGMAS > &)> & mean_func_Y,
    const std::function< EVec< ROWS >(const EVec< ROWS > &, const EVec< ROWS > &)>
& residual_func_Y,
    const std::function< StateVector(const StateVector &, const StateVector &)> &
residual_func_X,
    const std::function< StateVector(const StateVector &, const StateVector &)> &
add_func_X) [inline]

```

Correct the state estimate using the measurements in *y*, a custom measurement function, custom standard deviations, and custom mean, residual, and addition functions. This is useful for when a different set of measurements are being applied, and they require custom arithmetic functions.

Parameters

<i>u</i>	The control input used in the last predict step.
<i>y</i>	The vector of measurements.
<i>h</i>	A vector valued function of <i>x</i> and <i>u</i> that returns the expected measurement at the given state.
<i>measurement_stddevs</i>	The vector of standard deviations for each measurement to be used for this correct step.
<i>mean_func_Y</i>	A function that computes the mean of a matrix containing NUM_SIGMAS measurement sigma points with a set of weights for each.

<i>residual_func_X</i>	A function that computes the residual of two state vectors, usually by simple subtraction.
<i>residual_func_Y</i>	A function that computes the residual of two measurement vectors, usually by simple subtraction.
<i>add_func_X</i>	A function that adds two state vectors.

correct() [3/4]

```
template<int STATES, int INPUTS, int OUTPUTS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::correct (
    const InputVector & u,
    const OutputVector & y) [inline]
```

Correct the state estimate using the measurements in *y*.

Parameters

<i>u</i>	The control input used in the last predict step.
<i>y</i>	The vector of measurements.

correct() [4/4]

```
template<int STATES, int INPUTS, int OUTPUTS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::correct (
    const InputVector & u,
    const OutputVector & y,
    const EVec< OUTPUTS > & measurement_stddevs) [inline]
```

Correct the state estimate using the measurements in *y*, and custom standard deviations. This is useful for when the noise in the measurements vary.

Parameters

<i>u</i>	The control input used in the last predict step.
<i>y</i>	The vector of measurements.
<i>measurement_stddevs</i>	The vector of standard deviations for each measurement to be used for this correct step.

P()

```
template<int STATES, int INPUTS, int OUTPUTS>
StateMatrix UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::P () const [inline]
```

Returns the reconstructed covariance matrix *P*.

predict()

```
template<int STATES, int INPUTS, int OUTPUTS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::predict (
    const InputVector & u,
    double dt) [inline]
```

Projects the state into the future by dt seconds with control input u.

Parameters

u	The control input.
dt	The timestep in seconds.

reset()

```
template<int STATES, int INPUTS, int OUTPUTS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::reset () [inline]
```

Resets the filter.

S() [1/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
StateMatrix UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::S () const [inline]
```

Returns the square-root covariance matrix S.

S() [2/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
double UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::S (
    int i,
    int j) const [inline]
```

Returns one element of the square-root covariance matrix S.

Parameters

i	Row of S.
j	Column of S.

set_P()

```
template<int STATES, int INPUTS, int OUTPUTS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::set_P (
    const StateMatrix & P) [inline]
```

Parameters

P	The covariance matrix P .
-----	-----------------------------

set_S()

```
template<int STATES, int INPUTS, int OUTPUTS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::set_S (
    const StateMatrix & S) [inline]
```

Set the current square-root covariance matrix S .

Parameters

S	The new square-root covariance matrix S .
-----	---

set_xhat() [1/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::set_xhat (
    const StateVector & xhat) [inline]
```

Set the current state estimate x -hat.

set_xhat() [2/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
void UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::set_xhat (
    int i,
    double value) [inline]
```

Set one element of the current state estimate x -hat.

Parameters

i	Row of x -hat.
-----	------------------

xhat() [1/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
StateVector UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::xhat () const [inline]
```

Returns the current state estimate x -hat.

xhat() [2/2]

```
template<int STATES, int INPUTS, int OUTPUTS>
double UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >::xhat (
    int i) const [inline]
```

Parameters

<i>i</i>	Row of x-hat.
----------	---------------

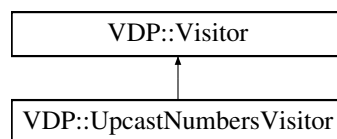
The documentation for this class was generated from the following file:

- unscented_kalman_filter.h

5.99 VDP::UpcastNumbersVisitor Class Reference

```
#include <types.hpp>
```

Inheritance diagram for VDP::UpcastNumbersVisitor:

**5.99.1 Detailed Description**

A class for broadly visiting a part and doing some action based on the upcast type of the part

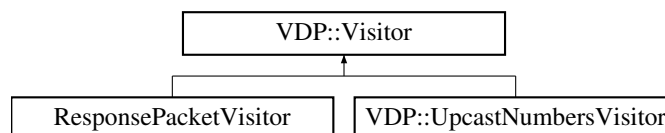
The documentation for this class was generated from the following files:

- types.hpp
- types.cpp

5.100 VDP::Visitor Class Reference

```
#include <types.hpp>
```

Inheritance diagram for VDP::Visitor:

**5.100.1 Detailed Description**

A class for broadly visiting a part and doing some action based on the type of part

The documentation for this class was generated from the following file:

- types.hpp

5.101 WaitUntilCondition Class Reference

Waits until the condition is true.

```
#include <auto_command.h>
```

5.101.1 Detailed Description

Waits until the condition is true.

The documentation for this class was generated from the following file:

- `auto_command.h`

5.102 WaitUntilUpToSpeedCommand Class Reference

```
#include <flywheel_commands.h>
```

Public Member Functions

- [WaitUntilUpToSpeedCommand](#) ([Flywheel](#) &flywheel, int threshold_rpm)
- bool [run](#) () override

5.102.1 Detailed Description

AutoCommand that listens to the [Flywheel](#) and waits until it is at its target speed +/- the specified threshold

5.102.2 Constructor & Destructor Documentation

WaitUntilUpToSpeedCommand()

```
WaitUntilUpToSpeedCommand::WaitUntilUpToSpeedCommand (
    Flywheel & flywheel,
    int threshold_rpm)
```

Create a [WaitUntilUpToSpeedCommand](#)

Parameters

<i>flywheel</i>	the flywheel system we are commanding
<i>threshold_rpm</i>	the threshold over and under the flywheel target RPM that we define to be acceptable

5.102.3 Member Function Documentation

run()

```
bool WaitUntilUpToSpeedCommand::run () [override]
```

Run spin_manual Overrides run from AutoCommand

Returns

true when execution is complete, false otherwise

The documentation for this class was generated from the following files:

- `flywheel_commands.h`
- `flywheel_commands.cpp`

Index

- ~AbstractDevice
 - VDP::AbstractDevice, [12](#)
 - A
 - LinearSystem< STATES, INPUTS, OUTPUTS >, [71](#)
 - accel
 - OdometryBase, [100](#)
 - add
 - CommandController, [21](#), [22](#)
 - GenericAuto, [50](#)
 - add_async
 - GenericAuto, [51](#)
 - add_cancel_func
 - CommandController, [22](#)
 - add_delay
 - CommandController, [22](#)
 - GenericAuto, [51](#)
 - add_entry
 - ExponentialMovingAverage, [35](#)
 - MovingAverage, [88](#)
 - ang_accel_deg
 - OdometryBase, [100](#)
 - ang_speed_deg
 - OdometryBase, [100](#)
 - as_vector
 - Translation2d, [199](#)
 - Async, [13](#)
 - auto_drive
 - MecanumDrive, [78](#)
 - auto_turn
 - MecanumDrive, [79](#)
 - AutoChooser, [13](#)
 - AutoChooser, [14](#)
 - choice, [14](#)
 - get_choice, [14](#)
 - list, [14](#)
 - AutoChooser::entry_t, [34](#)
 - name, [34](#)
- B
 - LinearSystem< STATES, INPUTS, OUTPUTS >, [71](#)
- background_task
 - OdometryBase, [97](#)
- BasicSolenoidSet, [15](#)
 - BasicSolenoidSet, [15](#)
 - run, [15](#)
- BasicSpinCommand, [16](#)
 - BasicSpinCommand, [16](#)
 - run, [17](#)
- BasicStopCommand, [17](#)
 - BasicStopCommand, [17](#)
 - run, [18](#)
- bool_or
 - Serializer, [164](#)
- BrakeType
 - TankDrive, [178](#)
- Branch, [18](#)
- ButtonWidget
 - screen::ButtonWidget, [19](#)
- C
 - LinearSystem< STATES, INPUTS, OUTPUTS >, [71](#)
- calculate
 - CRC32, [24](#)
 - FeedForward, [40](#)
 - LinearPlantInversionFeedforward< STATES, INPUTS >, [65](#), [66](#)
 - LinearQuadraticRegulator< STATES, INPUTS >, [69](#)
 - TrapezoidProfile, [205](#)
- choice
 - AutoChooser, [14](#)
- clear
 - InterpolatingMap< KEY, VALUE >, [53](#)
 - VDP::PacketWriter, [116](#)
- cobs_decode
 - OdometrySerial, [108](#)
- cobs_encode
 - OdometrySerial, [108](#)
- CommandController, [20](#)
 - add, [21](#), [22](#)
 - add_cancel_func, [22](#)
 - add_delay, [22](#)
 - CommandController, [21](#)
 - last_command_timed_out, [23](#)
 - run, [23](#)
- compute_X
 - LinearSystem< STATES, INPUTS, OUTPUTS >, [71](#)
- compute_Y
 - LinearSystem< STATES, INPUTS, OUTPUTS >, [71](#)
- Condition, [23](#)
- config
 - PID, [130](#)
- control_continuous
 - Lift< T >, [60](#)
- control_manual
 - Lift< T >, [60](#)
- control_setpoints
 - Lift< T >, [60](#)
- Core, [2](#)
- correct
 - KalmanFilter< STATES, INPUTS, OUTPUTS >, [56](#)
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, [214–216](#)
- CRC32, [24](#)
 - calculate, [24](#)

- finalize, [25](#)
 - update, [25](#)
- current_pos
 - OdometryBase, [100](#)
- CustomEncoder, [26](#)
 - CustomEncoder, [26](#)
 - position, [26](#)
 - rotation, [27](#)
 - setPosition, [27](#)
 - setRotation, [27](#)
 - velocity, [28](#)
- D
 - LinearSystem< STATES, INPUTS, OUTPUTS >, [72](#)
- degrees
 - Rotation2d, [156](#)
- delay_ms
 - VDB, [11](#)
- DelayCommand, [28](#)
 - DelayCommand, [28](#)
 - run, [29](#)
- discAB
 - LinearSystem< STATES, INPUTS, OUTPUTS >, [72](#)
- distance
 - Translation2d, [199](#)
- double_or
 - Serializer, [165](#)
- draw
 - screen::FunctionPage, [49](#)
 - screen::OdometryPage, [106](#)
 - screen::Page, [119](#)
 - screen::PIDPage, [136](#)
 - screen::StatsPage, [171](#)
- drive
 - MecanumDrive, [79](#)
- drive_arcade
 - TankDrive, [179](#)
- drive_correction_cutoff
 - robot_specs_t, [154](#)
- drive_forward
 - TankDrive, [179](#), [180](#)
- drive_raw
 - MecanumDrive, [80](#)
- drive_tank
 - TankDrive, [181](#)
- drive_tank_raw
 - TankDrive, [181](#)
- drive_to_point
 - TankDrive, [182](#)
- DriveForwardCommand, [29](#)
 - DriveForwardCommand, [30](#)
 - on_timeout, [30](#)
 - run, [30](#)
- DriveStopCommand, [31](#)
 - DriveStopCommand, [31](#)
 - run, [32](#)
- DriveToPointCommand, [32](#)
 - DriveToPointCommand, [32](#), [33](#)
 - run, [33](#)
- dtheta
 - Twist2d, [210](#)
- dx
 - Twist2d, [210](#)
- dy
 - Twist2d, [210](#)
- end_async
 - OdometryBase, [98](#)
- error_method
 - PID::pid_config_t, [131](#)
- ERROR_TYPE
 - PID, [126](#)
- exp
 - Pose2d, [138](#)
- ExponentialMovingAverage, [34](#)
 - add_entry, [35](#)
 - ExponentialMovingAverage, [35](#)
 - get_size, [36](#)
 - get_value, [36](#)
- f_cos
 - Rotation2d, [156](#)
- f_sin
 - Rotation2d, [156](#)
- f_tan
 - Rotation2d, [156](#)
- Feedback, [36](#)
 - get, [37](#)
 - init, [37](#)
 - is_on_target, [38](#)
 - set_limits, [38](#)
 - update, [38](#)
- FeedForward, [39](#)
 - calculate, [40](#)
 - FeedForward, [39](#)
- FeedForward::ff_config_t, [40](#)
 - kA, [41](#)
 - kG, [41](#)
 - kS, [41](#)
 - kV, [41](#)
- fetch
 - VDP::MotorDataRecord, [86](#)
 - VDP::Number< NumT, schemaType >, [90](#)
 - VDP::OdometryDataRecord, [104](#)
 - VDP::PIDDataRecord, [134](#)
 - VDP::Record, [146](#)
 - VDP::String, [173](#)
 - VDP::TimestampedRecord, [190](#)
- FetchFunc
 - VDP::Number< NumT, schemaType >, [90](#)
- Filter, [41](#)
- finalize
 - CRC32, [25](#)
- Flywheel, [42](#)
 - Flywheel, [42](#)
 - get_motors, [43](#)

- get_target, 43
- getRPM, 43
- is_on_target, 43
- spin_manual, 43
- spin_rpm, 44
- SpinRpmCmd, 44
- spinRPMTask, 45
- stop, 45
- WaitUntilUpToSpeedCmd, 45
- FlywheelStopCommand, 45
 - FlywheelStopCommand, 46
 - run, 46
- FlywheelStopMotorsCommand, 46
 - FlywheelStopMotorsCommand, 47
 - run, 47
- FlywheelStopNonTasksCommand, 47
- FunctionCommand, 48
- FunctionCondition, 48
- FunctionPage
 - screen::FunctionPage, 49
- GenericAuto, 50
 - add, 50
 - add_async, 51
 - add_delay, 51
 - run, 51
- get
 - Feedback, 37
 - MotionController, 82
 - PID, 127
 - TakeBackHalf, 176
- get_accel
 - OdometryBase, 98
- get_angular_accel_deg
 - OdometryBase, 98
- get_angular_speed_deg
 - OdometryBase, 98
- get_async
 - Lift< T >, 61
- get_choice
 - AutoChooser, 14
- get_error
 - PID, 127
- get_motion
 - MotionController, 82
- get_motors
 - Flywheel, 43
- get_output
 - PID, 127
- get_packet
 - VDP::PacketWriter, 116
- get_points
 - PurePursuit::Path, 125
- get_pose2d
 - OdometrySerial, 109
- get_position
 - OdometryBase, 98
 - OdometrySerial, 109
 - TankDrive, 183
- get_radius
 - PurePursuit::Path, 125
- get_sensor_val
 - PID, 127
- get_setpoint
 - Lift< T >, 61
- get_size
 - ExponentialMovingAverage, 36
 - MovingAverage, 88
- get_speed
 - OdometryBase, 99
- get_target
 - Flywheel, 43
 - PID, 128
- get_value
 - ExponentialMovingAverage, 36
 - MovingAverage, 88
 - VDP::Number< NumT, schemaType >, 90
 - VDP::String, 173
- getRPM
 - Flywheel, 43
- handle
 - OdometryBase, 101
- has_message
 - MaybeMessage, 77
- hold
 - Lift< T >, 61
- home
 - Lift< T >, 61
- IfTimePassed, 52
- init
 - Feedback, 37
 - MotionController, 83
 - PID, 128
 - TakeBackHalf, 176
- InOrder, 52
- insert
 - InterpolatingMap< KEY, VALUE >, 53
- install_broadcast_callback
 - VDP::RegistryListener< MutexType >, 148
- install_data_callback
 - VDP::RegistryListener< MutexType >, 148
- int_or
 - Serializer, 165
- InterpolatingMap< KEY, VALUE >, 53
 - clear, 53
 - insert, 53
 - operator[], 54
- inverse
 - Transform2d, 195
- is_on_target
 - Feedback, 38
 - Flywheel, 43
 - MotionController, 83
 - PID, 128
 - TakeBackHalf, 177
- is_valid

- PurePursuit::Path, 125
- kA
 - FeedForward::ff_config_t, 41
- KalmanFilter
 - KalmanFilter< STATES, INPUTS, OUTPUTS >, 55
- KalmanFilter< STATES, INPUTS, OUTPUTS >, 54
 - correct, 56
 - KalmanFilter, 55
 - P, 57
 - predict, 57
 - reset, 57
 - set_P, 57
 - set_xhat, 58
 - xhat, 58
- kG
 - FeedForward::ff_config_t, 41
- kS
 - FeedForward::ff_config_t, 41
- kV
 - FeedForward::ff_config_t, 41
- last_command_timed_out
 - CommandController, 23
- latency_compensate
 - LinearQuadraticRegulator< STATES, INPUTS >, 69
- Lift
 - Lift< T >, 59
- Lift< T >, 59
 - control_continuous, 60
 - control_manual, 60
 - control_setpoints, 60
 - get_async, 61
 - get_setpoint, 61
 - hold, 61
 - home, 61
 - Lift, 59
 - set_async, 61
 - set_position, 62
 - set_sensor_function, 62
 - set_sensor_reset, 62
 - set_setpoint, 62
- Lift< T >::lift_cfg_t, 63
- LinearPlantInversionFeedforward
 - LinearPlantInversionFeedforward< STATES, INPUTS >, 64
- LinearPlantInversionFeedforward< STATES, INPUTS >, 63
 - calculate, 65, 66
 - LinearPlantInversionFeedforward, 64
 - reset, 66
 - set_r, 66
- LinearQuadraticRegulator
 - LinearQuadraticRegulator< STATES, INPUTS >, 68
- LinearQuadraticRegulator< STATES, INPUTS >, 67
 - calculate, 69
 - latency_compensate, 69
 - LinearQuadraticRegulator, 68
- LinearSystem
 - LinearSystem< STATES, INPUTS, OUTPUTS >, 70
- LinearSystem< STATES, INPUTS, OUTPUTS >, 70
 - A, 71
 - B, 71
 - C, 71
 - compute_X, 71
 - compute_Y, 71
 - D, 72
 - discAB, 72
 - LinearSystem, 70
- list
 - AutoChooser, 14
- Log
 - Logger, 73, 74
- log
 - Pose2d, 139
- Logf
 - Logger, 74
- Logger, 72
 - Log, 73, 74
 - Logf, 74
 - Logger, 73
 - LogIn, 74, 75
- LogIn
 - Logger, 74, 75
- MaybeMessage, 76
 - has_message, 77
 - MaybeMessage, 76
 - message, 77
- MecanumDrive, 77
 - auto_drive, 78
 - auto_turn, 79
 - drive, 79
 - drive_raw, 80
 - MecanumDrive, 78
- MecanumDrive::mecanumdrive_config_t, 80
- message
 - MaybeMessage, 77
- modify_inputs
 - TankDrive, 183
- MotionController, 81
 - get, 82
 - get_motion, 82
 - init, 83
 - is_on_target, 83
 - MotionController, 82
 - set_limits, 83
 - tune_feedforward, 84
 - update, 84
- MotionController::m_profile_cfg_t, 75
- MotorDataRecord
 - VDP::MotorDataRecord, 86
- MovingAverage, 87
 - add_entry, 88
 - get_size, 88

- get_value, 88
- MovingAverage, 87
- mut
 - OdometryBase, 101
- name
 - AutoChooser::entry_t, 34
- None
 - TankDrive, 179
- norm
 - Translation2d, 199
- normalize
 - Translation2d, 199
- num_sigmas
 - ScaledSphericalSimplexSigmaPoints< STATES >, 162
- Number
 - VDP::Number< NumT, schemaType >, 90
- Odometry3Wheel, 92
 - Odometry3Wheel, 94
 - tune, 94
 - update, 95
- Odometry3Wheel::odometry3wheel_cfg_t, 95
 - off_axis_center_dist, 95
 - wheel_diam, 95
 - wheelbase_dist, 96
- OdometryBase, 96
 - accel, 100
 - ang_accel_deg, 100
 - ang_speed_deg, 100
 - background_task, 97
 - current_pos, 100
 - end_async, 98
 - get_accel, 98
 - get_angular_accel_deg, 98
 - get_angular_speed_deg, 98
 - get_position, 98
 - get_speed, 99
 - handle, 101
 - mut, 101
 - OdometryBase, 97
 - set_position, 99
 - smallest_angle, 99
 - speed, 101
 - update, 100
- OdometryControlRecord
 - VDP::OdometryControlRecord, 102
- OdometryDataRecord
 - VDP::OdometryDataRecord, 104
- OdometryPage
 - screen::OdometryPage, 105
- OdometrySerial, 106
 - cobs_decode, 108
 - cobs_encode, 108
 - get_pose2d, 109
 - get_position, 109
 - OdometrySerial, 108
 - receive_cobs_packet, 109
 - send_config, 110
 - set_position, 110
 - update, 110
- OdometryTank, 111
 - OdometryTank, 112, 113
 - set_position, 113
 - update, 113
- OdomSetPosition, 114
 - OdomSetPosition, 114
 - run, 115
- off_axis_center_dist
 - Odometry3Wheel::odometry3wheel_cfg_t, 95
- on_target_time
 - PID::pid_config_t, 131
- on_timeout
 - DriveForwardCommand, 30
 - PurePursuitCommand, 143
 - TurnDegreesCommand, 206
 - TurnToHeadingCommand, 208
- operator<<
 - Pose2d, 142
 - Rotation2d, 160
 - Transform2d, 197
 - Translation2d, 204
 - Twist2d, 211
- operator+
 - Pose2d, 139
 - Rotation2d, 157
 - Translation2d, 200
- operator-
 - Pose2d, 140
 - Rotation2d, 157
 - Transform2d, 195
 - Translation2d, 200, 201
- operator/
 - Pose2d, 140
 - Rotation2d, 158
 - Transform2d, 195
 - Translation2d, 201
 - Twist2d, 210
- operator==
 - Pose2d, 140
 - Rotation2d, 158
 - Transform2d, 195
 - Translation2d, 201
 - Twist2d, 211
- operator[]
 - InterpolatingMap< KEY, VALUE >, 54
- operator*
 - Pose2d, 139
 - Rotation2d, 156
 - Transform2d, 195
 - Translation2d, 199, 200
 - Twist2d, 210
- P
 - KalmanFilter< STATES, INPUTS, OUTPUTS >, 57
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, 216

- PacketWriter
 - VDP::PacketWriter, 116
- Parallel, 120
- Part
 - VDP::Part, 122
- Path
 - PurePursuit::Path, 124
- PID, 125
 - config, 130
 - ERROR_TYPE, 126
 - get, 127
 - get_error, 127
 - get_output, 127
 - get_sensor_val, 127
 - get_target, 128
 - init, 128
 - is_on_target, 128
 - PID, 126
 - reset, 128
 - set_limits, 129
 - set_target, 129
 - update, 129
- PID::pid_config_t, 130
 - error_method, 131
 - on_target_time, 131
- PIDControlRecord
 - VDP::PIDControlRecord, 132
- PIDDataRecord
 - VDP::PIDDataRecord, 134
- PIDPage
 - screen::PIDPage, 135
- Pose2d, 136
 - exp, 138
 - log, 139
 - operator<<, 142
 - operator+, 139
 - operator-, 140
 - operator/, 140
 - operator==, 140
 - operator*, 139
 - Pose2d, 137, 138
 - relative_to, 140
 - rotation, 141
 - setRotationDeg, 141
 - setRotationRad, 141
 - transform_by, 141
 - translation, 141
 - x, 142
 - y, 142
- position
 - CustomEncoder, 26
- pprint
 - VDP::Number< NumT, schemaType >, 90
 - VDP::Part, 122
 - VDP::String, 173
- pprint_data
 - VDP::Number< NumT, schemaType >, 91
 - VDP::Part, 122
- VDP::String, 174
- predict
 - KalmanFilter< STATES, INPUTS, OUTPUTS >, 57
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, 216
- pretty_print
 - VDP::Part, 122
- pretty_print_data
 - VDP::Part, 123
- pure_pursuit
 - TankDrive, 184
- PurePursuit::hermite_point, 52
- PurePursuit::Path, 124
 - get_points, 125
 - get_radius, 125
 - is_valid, 125
 - Path, 124
- PurePursuit::spline, 170
- PurePursuitCommand, 143
 - on_timeout, 143
 - PurePursuitCommand, 143
 - run, 143
- radians
 - Rotation2d, 158
- radius
 - tracking_wheel_cfg_t, 191
- read_data_from_message
 - VDP::Number< NumT, schemaType >, 91
 - VDP::Part, 123
 - VDP::Record, 146
 - VDP::String, 174
- receive_cobs_packet
 - OdometrySerial, 109
- Record
 - VDP::Record, 144, 145
- Rect, 147
- register_receive_callback
 - VDP::AbstractDevice, 12
- RegistryListener
 - VDP::RegistryListener< MutexType >, 148
- relative_to
 - Pose2d, 140
- reset
 - KalmanFilter< STATES, INPUTS, OUTPUTS >, 57
 - LinearPlantInversionFeedforward< STATES, INPUTS >, 66
 - PID, 128
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, 217
- reset_auto
 - TankDrive, 185
- response
 - VDP::OdometryControlRecord, 103
 - VDP::PIDControlRecord, 133
 - VDP::String, 174
- ResponsePacketVisitor, 150
 - ResponsePacketVisitor, 150
 - VisitDouble, 151

- VisitFloat, [151](#)
- VisitInt16, [151](#)
- VisitInt32, [151](#)
- VisitInt64, [151](#)
- VisitInt8, [152](#)
- VisitRecord, [152](#)
- VisitString, [152](#)
- VisitUInt16, [152](#)
- VisitUInt32, [152](#)
- VisitUInt64, [153](#)
- VisitUInt8, [153](#)
- revolutions
 - Rotation2d, [158](#)
- robot_specs_t, [153](#)
 - drive_correction_cutoff, [154](#)
- rotate_around
 - Translation2d, [202](#)
- rotate_by
 - Translation2d, [202](#)
- rotation
 - CustomEncoder, [27](#)
 - Pose2d, [141](#)
 - Transform2d, [196](#)
- Rotation2d, [154](#)
 - degrees, [156](#)
 - f_cos, [156](#)
 - f_sin, [156](#)
 - f_tan, [156](#)
 - operator<=, [160](#)
 - operator+, [157](#)
 - operator-, [157](#)
 - operator/, [158](#)
 - operator==, [158](#)
 - operator*, [156](#)
 - radians, [158](#)
 - revolutions, [158](#)
 - Rotation2d, [155](#)
 - rotation_matrix, [159](#)
 - wrapped_degrees_180, [159](#)
 - wrapped_degrees_360, [159](#)
 - wrapped_radians_180, [159](#)
 - wrapped_radians_360, [159](#)
 - wrapped_revolutions_180, [160](#)
 - wrapped_revolutions_360, [160](#)
- rotation_matrix
 - Rotation2d, [159](#)
- run
 - BasicSolenoidSet, [15](#)
 - BasicSpinCommand, [17](#)
 - BasicStopCommand, [18](#)
 - CommandController, [23](#)
 - DelayCommand, [29](#)
 - DriveForwardCommand, [30](#)
 - DriveStopCommand, [32](#)
 - DriveToPointCommand, [33](#)
 - FlywheelStopCommand, [46](#)
 - FlywheelStopMotorsCommand, [47](#)
 - GenericAuto, [51](#)
 - OdomSetPosition, [115](#)
 - PurePursuitCommand, [143](#)
 - SpinRPMCommand, [170](#)
 - TurnDegreesCommand, [206](#)
 - TurnToHeadingCommand, [208](#)
 - WaitUntilUpToSpeedCommand, [220](#)
- S
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, [217](#)
- save_to_disk
 - Serializer, [165](#)
- ScaledSphericalSimplexSigmaPoints
 - ScaledSphericalSimplexSigmaPoints< STATES >, [161](#)
- ScaledSphericalSimplexSigmaPoints< STATES >, [161](#)
 - num_sigmas, [162](#)
 - ScaledSphericalSimplexSigmaPoints, [161](#)
 - square_root_sigma_points, [162](#)
 - Wc, [162](#)
 - Wm, [163](#)
- screen::ButtonWidget, [19](#)
 - ButtonWidget, [19](#)
 - update, [20](#)
- screen::FunctionPage, [48](#)
 - draw, [49](#)
 - FunctionPage, [49](#)
 - update, [49](#)
- screen::OdometryPage, [105](#)
 - draw, [106](#)
 - OdometryPage, [105](#)
 - update, [106](#)
- screen::Page, [119](#)
 - draw, [119](#)
 - update, [120](#)
- screen::PIDPage, [135](#)
 - draw, [136](#)
 - PIDPage, [135](#)
 - update, [136](#)
- screen::ScreenData, [163](#)
- screen::SliderWidget, [167](#)
 - SliderWidget, [168](#)
 - update, [168](#)
- screen::StatsPage, [171](#)
 - draw, [171](#)
 - StatsPage, [171](#)
 - update, [171](#)
- send_config
 - OdometrySerial, [110](#)
- send_data
 - VDP::RegistryListener< MutexType >, [149](#)
- send_packet
 - VDP::AbstractDevice, [12](#)
- Serializer, [163](#)
 - bool_or, [164](#)
 - double_or, [165](#)
 - int_or, [165](#)
 - save_to_disk, [165](#)
 - Serializer, [164](#)

- set_bool, 166
 - set_double, 166
 - set_int, 166
 - set_string, 167
 - string_or, 167
- set_async
 - Lift< T >, 61
- set_bool
 - Serializer, 166
- set_double
 - Serializer, 166
- set_fields
 - VDP::Record, 146
- set_int
 - Serializer, 166
- set_limits
 - Feedback, 38
 - MotionController, 83
 - PID, 129
 - TakeBackHalf, 177
- set_P
 - KalmanFilter< STATES, INPUTS, OUTPUTS >, 57
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, 217
- set_position
 - Lift< T >, 62
 - OdometryBase, 99
 - OdometrySerial, 110
 - OdometryTank, 113
- set_r
 - LinearPlantInversionFeedforward< STATES, INPUTS >, 66
- set_S
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, 218
- set_sensor_function
 - Lift< T >, 62
- set_sensor_reset
 - Lift< T >, 62
- set_setpoint
 - Lift< T >, 62
- set_string
 - Serializer, 167
- set_target
 - PID, 129
- set_value
 - VDP::Number< NumT, schemaType >, 91
 - VDP::String, 174
- set_xhat
 - KalmanFilter< STATES, INPUTS, OUTPUTS >, 58
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, 218
- setPosition
 - CustomEncoder, 27
- setRotation
 - CustomEncoder, 27
- setRotationDeg
 - Pose2d, 141
- setRotationRad
 - Pose2d, 141
- setX
 - Translation2d, 202
- setY
 - Translation2d, 203
- size
 - VDP::PacketWriter, 116
- SliderWidget
 - screen::SliderWidget, 168
- smallest_angle
 - OdometryBase, 99
- Smart
 - TankDrive, 179
- speed
 - OdometryBase, 101
- spin_manual
 - Flywheel, 43
- spin_rpm
 - Flywheel, 44
- SpinRpmCmd
 - Flywheel, 44
- SpinRPMCommand, 169
 - run, 170
 - SpinRPMCommand, 169
- spinRPMTask
 - Flywheel, 45
- square_root_sigma_points
 - ScaledSphericalSimplexSigmaPoints< STATES >, 162
- State, 170
- StatsPage
 - screen::StatsPage, 171
- stop
 - Flywheel, 45
 - TankDrive, 185
- String
 - VDP::String, 173
- string_or
 - Serializer, 167
- submit_response
 - VDP::RegistryListener< MutexType >, 149
- take_packet
 - VDP::RegistryListener< MutexType >, 149
- TakeBackHalf, 175
 - get, 176
 - init, 176
 - is_on_target, 177
 - set_limits, 177
 - update, 177
- TankDrive, 178
 - BrakeType, 178
 - drive_arcade, 179
 - drive_forward, 179, 180
 - drive_tank, 181
 - drive_tank_raw, 181
 - drive_to_point, 182
 - get_position, 183

- modify_inputs, 183
 - None, 179
 - pure_pursuit, 184
 - reset_auto, 185
 - Smart, 179
 - stop, 185
 - TankDrive, 179
 - turn_degrees, 185, 186
 - turn_to_heading, 186, 187
 - ZeroVelocity, 179
- TestRecord
 - VDP::TestRecord, 189
- theta
 - Translation2d, 203
- theta_rad
 - tracking_wheel_cfg_t, 191
- time_ms
 - VDB, 11
- TimestampedRecord
 - VDP::TimestampedRecord, 190
- total_time
 - TrapezoidProfile, 205
- tracking_wheel_cfg_t, 191
 - radius, 191
 - theta_rad, 191
 - x, 192
 - y, 192
- Transform2d, 192
 - inverse, 195
 - operator<<, 197
 - operator-, 195
 - operator/, 195
 - operator==, 195
 - operator*, 195
 - rotation, 196
 - Transform2d, 193, 194
 - translation, 196
 - x, 196
 - y, 196
- transform_by
 - Pose2d, 141
- translation
 - Pose2d, 141
 - Transform2d, 196
- Translation2d, 197
 - as_vector, 199
 - distance, 199
 - norm, 199
 - normalize, 199
 - operator<<, 204
 - operator+, 200
 - operator-, 200, 201
 - operator/, 201
 - operator==, 201
 - operator*, 199, 200
 - rotate_around, 202
 - rotate_by, 202
 - setX, 202
 - setY, 203
 - theta, 203
 - Translation2d, 198
 - x, 203
 - y, 203
- TrapezoidProfile, 204
 - calculate, 205
 - total_time, 205
 - TrapezoidProfile, 204
- tune
 - Odometry3Wheel, 94
- tune_feedforward
 - MotionController, 84
- turn_degrees
 - TankDrive, 185, 186
- turn_to_heading
 - TankDrive, 186, 187
- TurnDegreesCommand, 206
 - on_timeout, 206
 - run, 206
 - TurnDegreesCommand, 206
- TurnToHeadingCommand, 207
 - on_timeout, 208
 - run, 208
 - TurnToHeadingCommand, 207
- Twist2d, 208
 - dtheta, 210
 - dx, 210
 - dy, 210
 - operator<<, 211
 - operator/, 210
 - operator==, 211
 - operator*, 210
 - Twist2d, 209
- UnscentedKalmanFilter
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, 213
- UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, 211
 - correct, 214–216
 - P, 216
 - predict, 216
 - reset, 217
 - S, 217
 - set_P, 217
 - set_S, 218
 - set_xhat, 218
 - UnscentedKalmanFilter, 213
 - xhat, 218
- update
 - CRC32, 25
 - Feedback, 38
 - MotionController, 84
 - Odometry3Wheel, 95
 - OdometryBase, 100
 - OdometrySerial, 110
 - OdometryTank, 113
 - PID, 129

- screen::ButtonWidget, 20
- screen::FunctionPage, 49
- screen::OdometryPage, 106
- screen::Page, 120
- screen::PIDPage, 136
- screen::SliderWidget, 168
- screen::StatsPage, 171
- TakeBackHalf, 177
- VDB, 11
 - delay_ms, 11
 - time_ms, 11
- VDP::AbstractDevice, 11
 - ~AbstractDevice, 12
 - register_receive_callback, 12
 - send_packet, 12
- VDP::MotorDataRecord, 85
 - fetch, 86
 - MotorDataRecord, 86
- VDP::Number< NumT, schemaType >, 89
 - fetch, 90
 - FetchFunc, 90
 - get_value, 90
 - Number, 90
 - pprint, 90
 - pprint_data, 91
 - read_data_from_message, 91
 - set_value, 91
 - write_message, 92
 - write_schema, 92
- VDP::OdometryControlRecord, 101
 - OdometryControlRecord, 102
 - response, 103
- VDP::OdometryDataRecord, 103
 - fetch, 104
 - OdometryDataRecord, 104
- VDP::PacketHeader, 115
- VDP::PacketWriter, 115
 - clear, 116
 - get_packet, 116
 - PacketWriter, 116
 - size, 116
 - write_byte, 117
 - write_channel_acknowledge, 117
 - write_channel_broadcast, 117
 - write_data_message, 117
 - write_number, 118
 - write_request, 118
 - write_response, 118
 - write_string, 118
 - write_type, 119
- VDP::Part, 121
 - Part, 122
 - pprint, 122
 - pprint_data, 122
 - pretty_print, 122
 - pretty_print_data, 123
 - read_data_from_message, 123
 - write_message, 123
 - write_schema, 123
- VDP::PIDControlRecord, 131
 - PIDControlRecord, 132
 - response, 133
- VDP::PIDDataRecord, 133
 - fetch, 134
 - PIDDataRecord, 134
- VDP::Record, 144
 - fetch, 146
 - read_data_from_message, 146
 - Record, 144, 145
 - set_fields, 146
 - write_message, 146
 - write_schema, 147
- VDP::RegistryListener< MutexType >, 147
 - install_broadcast_callback, 148
 - install_data_callback, 148
 - RegistryListener, 148
 - send_data, 149
 - submit_response, 149
 - take_packet, 149
- VDP::String, 172
 - fetch, 173
 - get_value, 173
 - pprint, 173
 - pprint_data, 174
 - read_data_from_message, 174
 - response, 174
 - set_value, 174
 - String, 173
 - write_message, 175
 - write_schema, 175
- VDP::TestRecord, 188
 - TestRecord, 189
- VDP::TimestampedRecord, 189
 - fetch, 190
 - TimestampedRecord, 190
- VDP::UpcastNumbersVisitor, 219
- VDP::Visitor, 219
- velocity
 - CustomEncoder, 28
- VisitDouble
 - ResponsePacketVisitor, 151
- VisitFloat
 - ResponsePacketVisitor, 151
- VisitInt16
 - ResponsePacketVisitor, 151
- VisitInt32
 - ResponsePacketVisitor, 151
- VisitInt64
 - ResponsePacketVisitor, 151
- VisitInt8
 - ResponsePacketVisitor, 152
- VisitRecord
 - ResponsePacketVisitor, 152
- VisitString
 - ResponsePacketVisitor, 152
- VisitUInt16

- ResponsePacketVisitor, [152](#)
- VisitUInt32
 - ResponsePacketVisitor, [152](#)
- VisitUInt64
 - ResponsePacketVisitor, [153](#)
- VisitUInt8
 - ResponsePacketVisitor, [153](#)
- WaitUntilCondition, [220](#)
- WaitUntilUpToSpeedCmd
 - Flywheel, [45](#)
- WaitUntilUpToSpeedCommand, [220](#)
 - run, [220](#)
 - WaitUntilUpToSpeedCommand, [220](#)
- Wc
 - ScaledSphericalSimplexSigmaPoints< STATES >, [162](#)
- wheel_diam
 - Odometry3Wheel::odometry3wheel_cfg_t, [95](#)
- wheelbase_dist
 - Odometry3Wheel::odometry3wheel_cfg_t, [96](#)
- Wm
 - ScaledSphericalSimplexSigmaPoints< STATES >, [163](#)
- wrapped_degrees_180
 - Rotation2d, [159](#)
- wrapped_degrees_360
 - Rotation2d, [159](#)
- wrapped_radians_180
 - Rotation2d, [159](#)
- wrapped_radians_360
 - Rotation2d, [159](#)
- wrapped_revolutions_180
 - Rotation2d, [160](#)
- wrapped_revolutions_360
 - Rotation2d, [160](#)
- write_byte
 - VDP::PacketWriter, [117](#)
- write_channel_acknowledge
 - VDP::PacketWriter, [117](#)
- write_channel_broadcast
 - VDP::PacketWriter, [117](#)
- write_data_message
 - VDP::PacketWriter, [117](#)
- write_message
 - VDP::Number< NumT, schemaType >, [92](#)
 - VDP::Part, [123](#)
 - VDP::Record, [146](#)
 - VDP::String, [175](#)
- write_number
 - VDP::PacketWriter, [118](#)
- write_request
 - VDP::PacketWriter, [118](#)
- write_response
 - VDP::PacketWriter, [118](#)
- write_schema
 - VDP::Number< NumT, schemaType >, [92](#)
 - VDP::Part, [123](#)
 - VDP::Record, [147](#)
- VDP::String, [175](#)
- write_string
 - VDP::PacketWriter, [118](#)
- write_type
 - VDP::PacketWriter, [119](#)
- x
 - Pose2d, [142](#)
 - tracking_wheel_cfg_t, [192](#)
 - Transform2d, [196](#)
 - Translation2d, [203](#)
- xhat
 - KalmanFilter< STATES, INPUTS, OUTPUTS >, [58](#)
 - UnscentedKalmanFilter< STATES, INPUTS, OUTPUTS >, [218](#)
- y
 - Pose2d, [142](#)
 - tracking_wheel_cfg_t, [192](#)
 - Transform2d, [196](#)
 - Translation2d, [203](#)
- ZeroVelocity
 - TankDrive, [179](#)